SISTEMAS OPERATIVOS

UNIDAD III

Contenidos

Espacio de direcciones de memoria. 4
Introducción. 4
Sin abstracción de memoria. 4
Abstracción de memoria y espacio de direcciones. 7
La noción de un espacio de direcciones. 8
Referencias y bibliografía de la sección. 9
Compilación, enlace y carga. 9
Preprocesamiento. 11
Compilación. 11
Ensamblado. 12
Enlace (Linking). 13
Editor de enlace 15
Montador/enlazador dinámico. 15
Montaje/enlace dinámico y bibliotecas compartidas. 17
Carga. 19
Carga absoluta. 20
Carga reubicable 21
Carga dinámica en tiempo real. 23
Referencias y bibliografía de la sección. 25
Direccionamiento: Direcciones físicas, lógicas y virtuales. 25
Referencias y bibliografía de la sección. 29
CPU, memoria principal y reubicación simple. 29
Hardware básico 30
Reubicación simple con registros base y límite. 32
Reasignación de direcciones. 35
Referencias y bibliografía de la sección. 37
Asignación de memoria y mapeo. 38
Mapeo de memoria y protección. 39
Asignación de memoria contigua. 41
Referencias y bibliografía de la sección. 45

Más sobre algoritmos para carga de procesos en memoria. 45
Referencias y bibliografía de la sección. 47
Fragmentación interna, fragmentación externa y compactación. 48
Referencias y bibliografía de la sección. 51
Otros esquemas de administración de memoria. 51
Paginación. 51
Método básico. 52
Soporte de hardware. 58
Protección. 60
Páginas compartidas. 63
Segmentación. 64
Método básico. 64
Soporte de hardware. 66
Ejemplo: Intel Pentium. 68
Mecanismo de segmentación en el Pentium. 69
Mecanismo de paginación en el Pentium. 70
Referencias y bibliografía de la sección. 71
Tablas de páginas y segmentos. 71
Tablas de páginas. 71
Tablas de segmentos. 74
Referencias y bibliografía de la sección. 77
Estructura de las tablas de páginas. 78
Paginación jerárquica. 78
Tablas de páginas hash. 79
Tablas de páginas invertidas. 80
Referencias y bibliografía de la sección. 82
Algoritmos de reemplazo de páginas. 83
El algoritmo de reemplazo de páginas óptimo. 83
El algoritmo de reemplazo de páginas no usadas recientemente. 84
El algoritmo de reemplazo de páginas primera en entrar, primera en salir (FIFO). 85
El algoritmo de reemplazo de páginas segunda oportunidad. 86
El algoritmo de reemplazo de páginas reloj. 86
El algoritmo de reemplazo de páginas menos usadas recientemente (LRU). 87
El algoritmo de reemplazo de páginas de conjunto de trabajo. 88
El algoritmo de reemplazo de páginas WSClock. 91
Referencias y bibliografía de la sección. 93

Administración de memoria libre. 94
Administración de memoria con mapas de bits..... 94
Administración de memoria con listas ligadas..... 95
Referencias y bibliografía de la sección. 96
Intercambio. 96
Referencias y bibliografía de la sección. 99
Memoria virtual..... 99
Referencias y bibliografía de la sección. 105
Paginación bajo demanda. 105
Conceptos básicos. 106
Rendimiento de la paginación bajo demanda. 109
Referencias y bibliografía de la sección. 110

Espacio de direcciones de memoria.

Introducción.

La ***memoria principal*** es un importante recurso que debe administrarse con cuidado. Aunque actualmente una computadora doméstica tiene miles de veces más memoria que la IBM 7094, la computadora más grande en el mundo a principios de la década de 1960, los programas han ido creciendo con más rapidez que las memorias.

Lo que todo programador quisiera es una memoria privada, de tamaño y rapidez infinitas, que también sea no volátil, es decir, que no pierda su contenido cuando se desconecta de la potencia eléctrica. Por desgracia, la tecnología no proporciona tales memorias en estos momentos.

A través de los años se ha elaborado el concepto de ***jerarquía de memoria***, de acuerdo con el cual, las computadoras tienen ***memoria caché***, muy rápida, costosa y volátil, unos cuantos gigabytes de ***memoria principal***, de mediana velocidad, a precio mediano y volátil, unos cuantos terabytes de almacenamiento no volátil (HDD, SSD) económico y más lento, y el almacenamiento removible, como las memorias USB. El trabajo del sistema operativo es ***abstraer*** esta jerarquía en un modelo útil y después administrarla.



La parte del sistema operativo que administra (parte de) la jerarquía de memoria se conoce como ***administrador de memoria***. Su trabajo es administrar la memoria con eficiencia: llevar el registro de cuáles partes de la memoria están en uso, asignar memoria a los procesos cuando la necesiten y desasignarla cuando terminen.

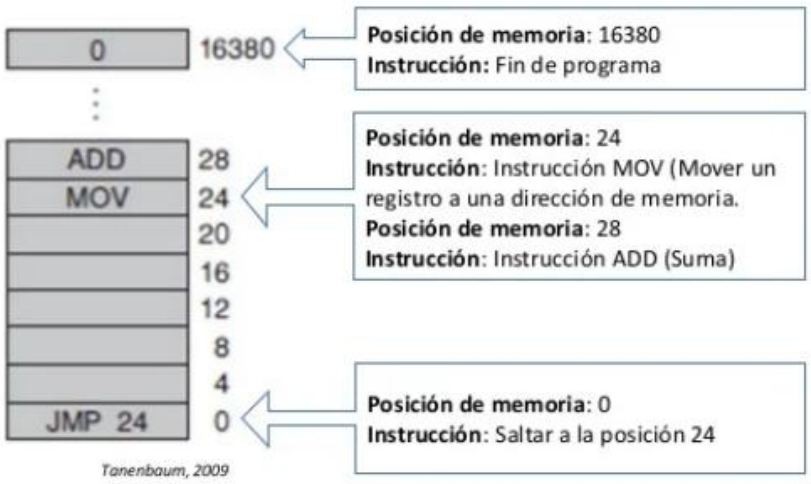
Sin abstracción de memoria.

Las primeras computadoras mainframe (antes de 1960), las primeras minicomputadoras (antes de 1970) y las primeras computadoras personales (antes de 1980) no tenían ***abstracción de memoria***. Cada programa veía simplemente la memoria física.



Ilustración 1 - La IBM 650 (presentado en 1953 y comercializado masivamente a partir de 1954) utilizaba memoria de tambor.

Cuando un programa ejecutaba una instrucción, la computadora sólo movía el contenido de la ubicación de memoria física al registro correspondiente. Así, el modelo de programación que se presentaba al programador era simplemente la memoria física, un conjunto de direcciones desde 0 hasta cierto valor máximo, en donde cada dirección correspondía a una celda que contenía cierto número de bits, comúnmente ocho.



Bajo estas condiciones, no era posible tener dos programas ejecutándose en memoria al mismo tiempo. Si el primer programa escribía un nuevo valor en, por ejemplo, la ubicación 2000, esto borraría cualquier valor que el segundo programa estuviera almacenando ahí. Ambos programas fallarían de inmediato.

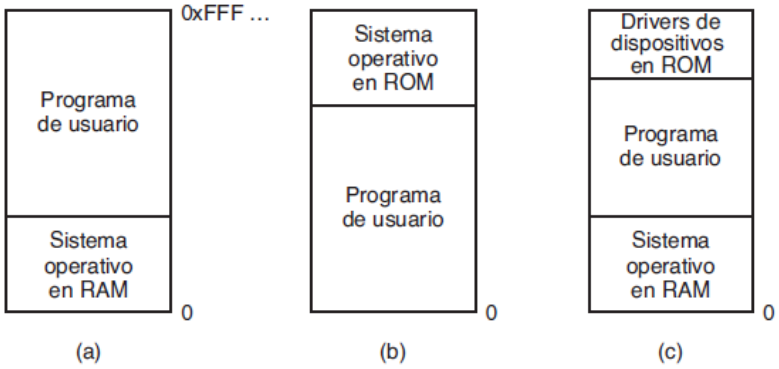


Ilustración 2 - Tres formas simples de organizar la memoria con un sistema operativo y un proceso de usuario. También existen otras posibilidades.

Incluso cuando el modelo de memoria consiste en sólo la memoria física hay varias opciones posibles. El sistema operativo puede estar en la parte inferior de la memoria en la ***RAM*** (*Random Access Memory*, Memoria de acceso aleatorio), puede estar en la ***ROM*** (*Read Only Memory*, Memoria de sólo lectura) en la parte superior de la memoria, o los controladores de dispositivos pueden estar en la parte superior de la memoria en una ROM y el resto del sistema en RAM más abajo.

El primer modelo se utilizó antes en las mainframes y minicomputadoras, pero actualmente casi no se emplea. El segundo modelo se utiliza en algunas computadoras de bolsillo y sistemas integrados. El tercer modelo fue utilizado por las primeras computadoras personales (por ejemplo, las que ejecutaban MS-DOS), donde la porción del sistema en la ROM se conoce como ***BIOS*** (*Basic Input Output System*, Sistema básico de entrada y salida).

Los modelos (a) y (c) de la figura anterior tienen la desventaja de que un error en el programa de usuario puede borrar el sistema operativo, posiblemente con resultados desastrosos (la información del disco podría quedar ininteligible).

Cuando el sistema se organiza de esta forma (sin abstracción), por lo general se puede ejecutar sólo un proceso a la vez. Tan pronto como el usuario teclea un comando, el sistema operativo copia el programa solicitado del disco a la memoria y lo ejecuta. Cuando termina el proceso, el sistema operativo muestra un carácter indicador de comando y espera un nuevo comando. Cuando recibe el comando, carga un nuevo programa en memoria, sobrescribiendo el primero.

El sistema operativo, en estos casos, era sustancialmente más simple porque no tenía que administrar la complejidad de la multiprogramación en memoria principal.

Podríamos decir que el sistema operativo funcionaba esencialmente como una simple biblioteca de rutinas de (E/S) encargada de cargar un archivo desde el disco a una dirección fija de memoria y cederle el control de la CPU.

Direccionamiento	**Físico Directo:** El proceso accede a las celdas reales de la **memoria principal**.	El programador debe conocer el mapa de direcciones físicas del hardware.
Aislamiento	**Nulo:** No existen fronteras de protección entre procesos.	Un error en una dirección de memoria puede sobrescribir al Kernel.
Capacidad	**Un solo proceso:** Solo un programa reside en la **memoria principal** a la vez.	La CPU se desperdicia si el proceso espera por E/S.

Una forma de obtener cierto grado de paralelismo en un sistema, sin abstracción de memoria, es programar con múltiples hilos. Como se supone que todos los hilos en un proceso ven la misma imagen de memoria, el hecho de que se vean obligados a hacerlo no es un problema. Aunque esta idea funciona, es de uso limitado ya que lo que las personas desean a menudo es que los programas *no relacionados* se ejecuten al mismo tiempo, algo que la abstracción de los hilos no provee. Además, es muy poco probable que un sistema tan primitivo como para no proporcionar una abstracción de memoria proporcione una abstracción de hilos.

Hilo	Pila	Variables locales
H1	Direcciones 10-19	Direcciones 20-29
H2	Direcciones 30-39	Direcciones 40-49

Abstracción de memoria y espacio de direcciones.

Exponer la memoria física a los procesos tiene varias desventajas. En primer lugar, si los programas de usuario pueden direccionar cada byte de memoria, pueden estropear el sistema operativo con facilidad, ya sea intencional o accidentalmente, con lo cual el sistema se detendría en forma súbita (a menos que haya hardware especial como el esquema de bloqueo y llaves). Este problema existe aun cuando sólo haya un programa de usuario (aplicación) en ejecución.

En segundo lugar, con este modelo es difícil tener varios programas en ejecución a la vez (tomando turnos, si sólo hay una CPU). En las computadoras personales es común tener varios programas abiertos a la vez (un procesador de texto, un programa de correo electrónico y un navegador Web, donde uno de ellos tiene el enfoque actual, pero los demás se reactivan con el clic de un mouse). Como esta situación era difícil de lograr cuando no hay una abstracción de la memoria física, se tuvo que hacer algo.

La noción de un espacio de direcciones.

Hay que resolver dos problemas para permitir que haya varias aplicaciones en memoria al mismo tiempo sin que interfieran entre sí: ***protección y reubicación***.

Una solución es inventar una abstracción para la memoria: el ***espacio de direcciones***. Así como el concepto del proceso crea un tipo de CPU abstracta para ejecutar programas, el espacio de direcciones crea un tipo de memoria abstracta para que los programas vivan ahí.

Recurso Físico	Abstracción	Beneficio para el Programa
CPU	**Proceso**	El programa siente que tiene un procesador dedicado solo para él.
Memoria Principal	**Espacio de Direcciones**	El programa siente que tiene un mapa de memoria privado y único.

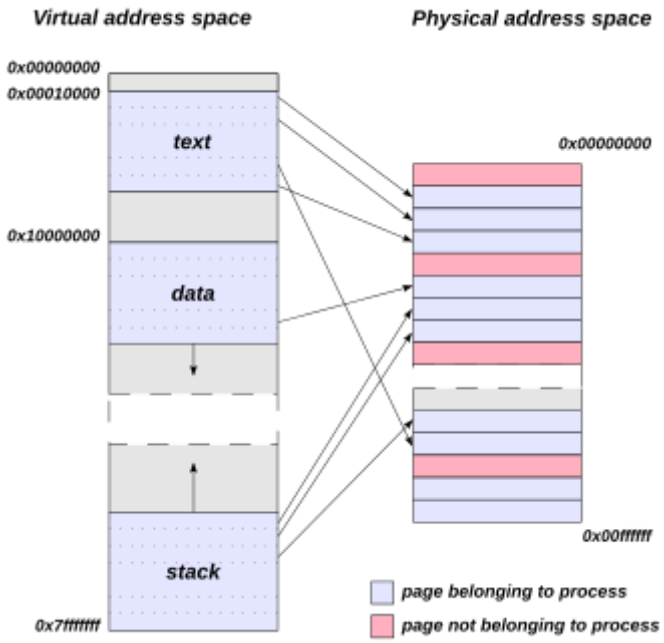
Un ***espacio de direcciones*** (address space) es el conjunto de direcciones que puede utilizar un proceso para direccionar la memoria. Cada proceso tiene su propio espacio de direcciones, independiente de los que pertenecen a otros procesos (excepto en ciertas circunstancias especiales en donde los procesos desean compartir sus espacios de direcciones).

Situación	Estado de los Espacios de Direcciones	Impacto en la **Memoria Principal**
Normal	**Aislamiento Total:** Cada proceso tiene restringido el acceso exclusivamente a su propio rango de direcciones.	Ningún proceso puede leer o alterar los datos almacenados por otro en la **memoria principal**.
Excepción	**Memoria Compartida:** Se habilita un área común de direcciones físicas accesible para múltiples procesos.	Dos o más procesos pueden visualizar y modificar los mismos datos.

El concepto de un espacio de direcciones es muy general y ocurre en muchos contextos. Consideremos los números telefónicos. En los Estados Unidos y en muchos otros países, un número de teléfono local es comúnmente un número de 7 dígitos. En consecuencia, el espacio de direcciones para los números telefónicos varía desde 0,000,000 hasta 9,999,999, aunque algunos números, como los que empiezan con 000, no se utilizan. Con el crecimiento de los teléfonos celulares y módems este espacio se está volviendo demasiado pequeño, en cuyo caso habrá qué utilizar más dígitos.

Podríamos pensar también en el espacio de direcciones para los puertos de E/S en el Pentium varía desde 0 hasta 16383.

Los espacios de direcciones no tienen que ser numéricos. El conjunto de dominios .com de Internet es también un espacio de direcciones. Este espacio de direcciones consiste de todas las cadenas de longitud de 2 a 63 caracteres que se puedan formar utilizando letras, números y guiones cortos, seguidas de *.com*.



Referencias y bibliografía de la sección.

- Tanenbaum 3-1 / 3-2.

Compilación, enlace y carga.

El primer paso en la creación de un proceso activo es cargar un programa en memoria principal y crear una imagen del proceso.

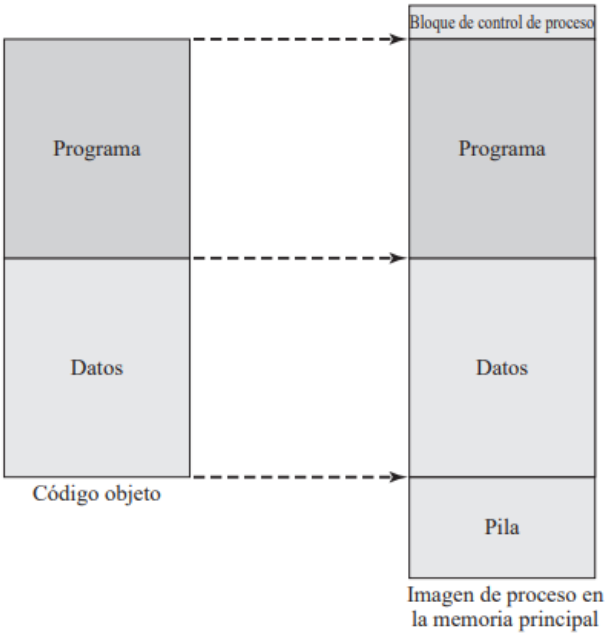


Ilustración 3 - La función de carga.

En un escenario típico para la mayoría de los sistemas, la aplicación está formada por varios módulos ***compilados*** o ***ensamblados*** en formato de código objeto[1]. Éstos son ***enlazados*** para resolver todas las referencias entre los módulos. Al mismo tiempo, se resuelven las referencias a rutinas de biblioteca.

Las rutinas de biblioteca pueden incorporarse al programa o hacerle referencia como código compartido que el sistema operativo proporciona en tiempo de ejecución.

[1] El código objeto es la salida de un compilador o traductor, en formato binario o código máquina, que es comprensible directamente por la máquina. Es el resultado de la traducción del código fuente y forma parte del programa final, específico de la plataforma de ejecución.

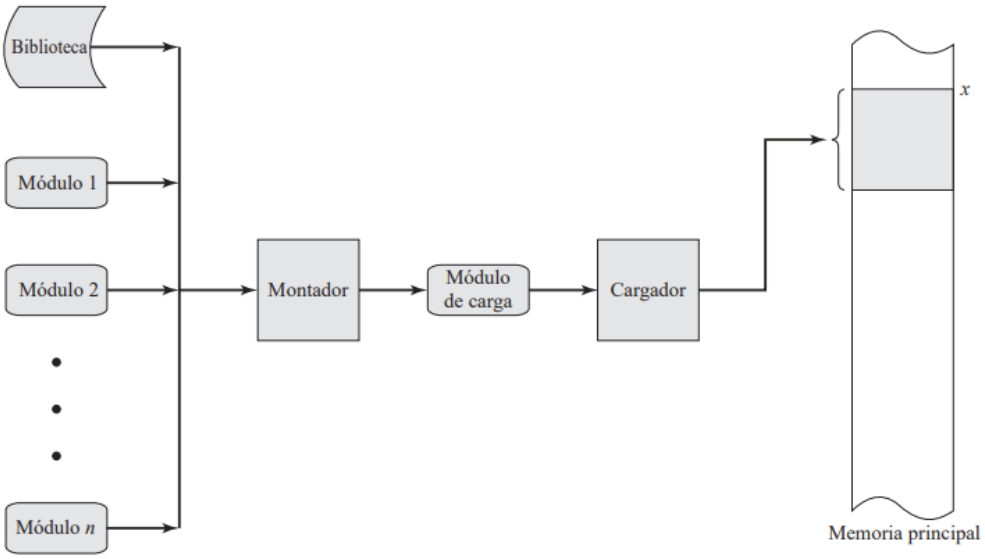


Ilustración 4 - Un escenario de carga.

Preprocesamiento.

El ***preprocesador*** atiende las directivas del código fuente. Su función es realizar sustituciones de texto, expandir macros e incluir archivos de cabecera para generar un archivo de código fuente expandido.

Podríamos decir que el preprocesamiento es la "fase cero" del ciclo de compilación. Aunque solemos admitir que el compilador hace todo el trabajo, en realidad recibe un código que ya fue filtrado y expandido por el preprocesador.

Entonces, el preprocesador es un manipulador de texto que no entiende la lógica del programa, sino que simplemente obedece órdenes que empiezan con el símbolo (***#*** en lenguajes como C o C++). Su salida es una unidad de traducción pura que el compilador podrá puede procesar.

El actuar del preprocesador definen el tamaño final del ejecutable. Si se escribe un programa de 10 líneas, pero se incluyen librerías (como ***stdio*** en C), el preprocesador le entrega al compilador un archivo que puede tener más de 500 líneas.

Compilación.

El ***compilador*** toma el código fuente y lo traduce a lenguaje ***ensamblador***. En esta fase se verifica la sintaxis y se realizan optimizaciones para que las instrucciones sean eficientes para el procesador.

Vale aclarar que en la arquitectura de la mayoría de compiladores modernos, el compilador puede traducir el código fuente directamente a código máquina binario o generar una fase intermedia en lenguaje ensamblador para que luego un ensamblador (assembler) produzca el código objeto binario.

```
gcc -S programa.c
```

Esto genera un archivo llamado *programa.s* que contiene únicamente el código en lenguaje ensamblador.

```
gcc -c programa.c
```

Esto genera un archivo *programa.o* que contiene directamente código máquina binario.

La compilación es el corazón del proceso de traducción. El compilador es un software más sofisticado que el preprocesador; este analiza la lógica, la estructura y la semántica del código para traducirlo a un lenguaje que el procesador pueda entender.

El compilador entrega un archivo en lenguaje ensamblador (habitualmente con la extensión ***.s***). Podríamos decir que este archivo contiene texto que un humano experto puede comprender, sin embargo, ya se encuentra estructurado como instrucciones puras para la CPU (como MOV, ADD, PUSH).

Fase	Tarea Principal	Error Común Detectado
Léxica	Identificar palabras y símbolos.	Carácter ilegal (ej. un símbolo extraño).
Sintáctica	Validar la estructura (gramática).	Falta un `;` o una llave `}`.
Semántica	Validar el sentido y tipos de datos.	Usar una variable no declarada.
Optimización	Hacer el código más eficiente.	(No detecta errores, mejora el rendimiento).

Ensamblado.

El ***ensamblador*** o ***montador*** traduce el código en lenguaje de ensamblador a código de máquina (binario). El resultado es un ***archivo objeto*** (frecuentemente ***.o*** o ***.obj***). Este archivo contiene instrucciones binarias, pero aún tiene referencias externas no resueltas (por ejemplo, llamadas a funciones de bibliotecas).

El ensamblado es la etapa donde el código deja de ser legible para los humanos y se convierte en instrucciones que el hardware puede procesar. Es el puente final hacia el lenguaje binario.

A diferencia del compilador, que debe realizar un análisis lógico complejo, el ensamblador realiza una traducción casi directa "uno a uno". Cada instrucción mnemotécnica[2] (como MOV o ADD) se convierte en un patrón específico de bits.

Aspecto	Detalle
Entrada	Archivo en Ensamblador (texto mnemotécnico, ej: `MOV AL, 61h`).
Salida	Archivo Objeto (binario no ejecutable, ej: `B0 61`).
Naturaleza	Traducción mecánica y directa de instrucciones.
Dependencia	Totalmente ligado a la arquitectura de la CPU (x86, ARM, etc.).

Dentro de la amplia bibliografía de Sistemas Operativos podemos encontrar referencias a dos tipos de ensambladores/montadores: ***montador ensamblador*** (assembler o traductor de ensamblador) y ***montador enlazador*** (montador final o linker).

También es común que se trate al ensamblador como un único modulo que cumple las funciones de ensamblar y enlazar.

Enlace (Linking).

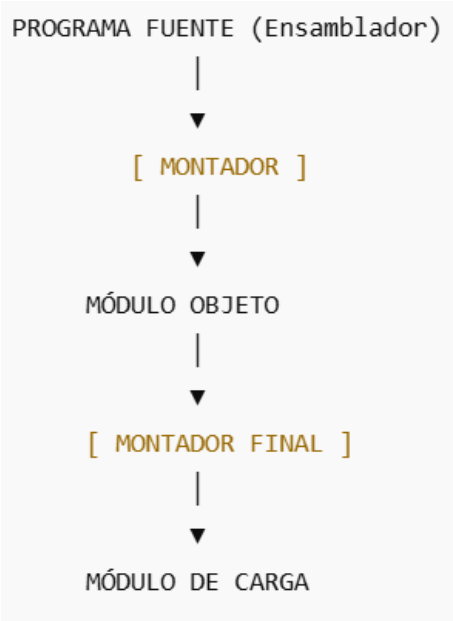
Como ya vimos, el ***ensamblador*** o montador es un traductor de lenguaje de bajo nivel (también llamado ensamblador) a lenguaje de máquina (código binario). Convierte el código de ensamblador a código objeto y genera el ***módulo objeto***.

La función de un ***montador final*** o ***enlazador*** (linker) es tomar como entrada una colección de módulos objeto y producir un ***módulo de carga***, formado por un conjunto integrado de programa y módulos de datos, que se pasará al ***cargador***.

En la gran mayoría de los casos, este módulo de carga es un archivo ejecutable.

En esta etapa, el ***enlazador*** combina los diversos archivos objeto con las bibliotecas del sistema necesarias.

[2] La mnemotécnica se refiere a códigos simbólicos (mnemónicos) que sustituyen instrucciones binarias complejas, facilitando la programación en lenguaje ensamblador.



En cada módulo objeto, podría haber referencias a direcciones de otros módulos. Cada una de estas referencias sólo se puede expresar simbólicamente en un módulo objeto no enlazado. El enlazador crea un único módulo de carga que une de forma contigua a todos los módulos objeto.

Cada referencia entre módulos debe cambiarse: una dirección simbólica debe convertirse en una referencia a una ubicación dentro del módulo de carga.

Entonces, el ***enlace*** es el proceso mediante el cual se unen varios ***módulos objeto*** para formar un programa completo que pueda ejecutarse en memoria.

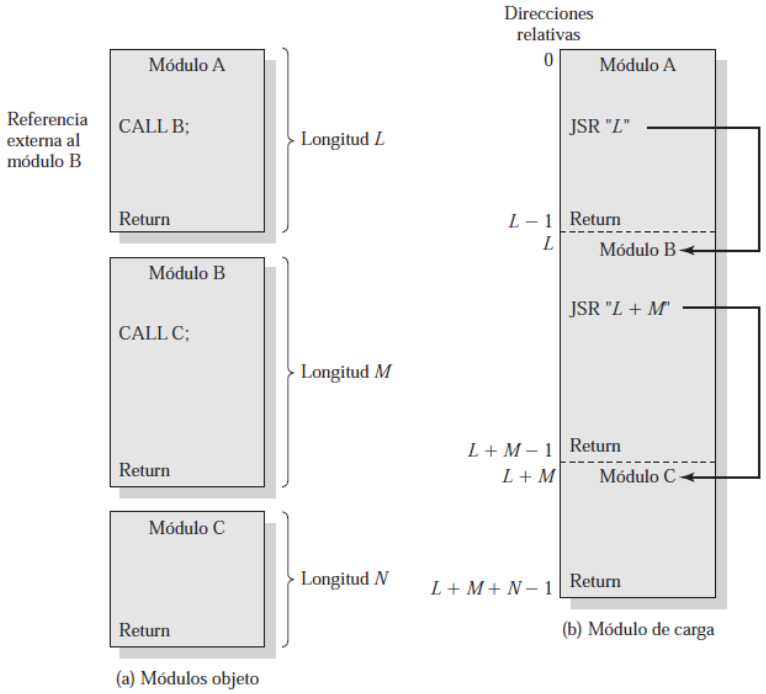
Tiempo de montaje	Función
Tiempo de programación	No se permiten referencias a programas o datos externos. El programador debe colocar en el programa el código fuente de todos los subprogramas que invoque.
Tiempo de compilación o ensamblado	El ensamblador debe traer el código fuente de cada subrutina que se referencia y ensamblarlo como una unidad.
Creación de módulo de carga	Todos los módulos objeto se han ensamblado utilizando direcciones relativas. Estos módulos se enlazan juntos y todas las referencias se restablecen en relación al origen del módulo de carga final.
Tiempo de carga	Las referencias externas no se resuelven hasta que el módulo de carga se carga en memoria principal. En ese momento, los módulos con enlace dinámico referenciados se adjuntan al módulo de carga y el paquete completo se carga en memoria principal o virtual.
Tiempo de ejecución	Las referencias externas no se resuelven hasta que el procesador ejecuta la llamada externa. En ese momento, el proceso se interrumpe y el módulo deseado se enlaza al programa que lo invoca.

Editor de enlace.

La naturaleza del enlace de direcciones dependerá del tipo de módulo de carga que se cree y cuando se lleve a cabo el proceso de enlace.

Si se desea un módulo de carga reubicable, como suele ser lo habitual, el enlace se hace normalmente de la siguiente forma. Cada módulo objeto compilado o ensamblado se crea con referencias relativas al comienzo del módulo objeto. Todos estos módulos se colocan juntos en un único módulo de carga reubicable con todas las referencias relativas al origen del módulo de carga. Este módulo se puede utilizar como entrada para la carga reubicable o carga dinámica en tiempo de ejecución.

Un montador/enlazador que produce un módulo de carga reubicable se denomina frecuentemente ***editor de enlace***.



Montador/enlazador dinámico.

Al igual que con la carga, también es posible posponer algunas funciones relativas al enlace. El término ***enlace dinámico*** se utiliza para denominar la práctica de posponer el enlace de algunos módulos externos hasta después de que el módulo de carga se cree. Por lo tanto, el módulo de carga contiene referencias sin resolver a otros programas. Estas referencias se pueden resolver o bien en tiempo de carga o bien en tiempo de ejecución.

Para el ***enlace dinámico en tiempo de carga***, se deben seguir los siguientes pasos. El módulo de carga (módulo de aplicación) debe llevarse a memoria para cargarlo. Cualquier referencia a un módulo externo (módulo destino) provoca que el cargador encuentre al módulo destino, lo

cargue y altere la referencia a una dirección relativa a la memoria desde el comienzo del módulo de aplicación. Hay varias ventajas de esta técnica frente al enlace estático:

- Se facilita incorporar versiones modificadas o actualizadas del módulo destino, el cual puede ser una utilidad del sistema operativo o algunas otras rutinas de propósito general. Con el enlace estático, un cambio a ese módulo requeriría el reenlace del módulo de aplicación completo. No sólo es ineficiente, sino que puede ser imposible en algunas circunstancias. Por ejemplo, en el campo de los ordenadores personales, la mayoría del software comercial es entregado en el formato del módulo de carga; no se entregan las versiones fuente y objeto.
- Tener el código destino en un fichero con enlace dinámico facilita el camino para compartir código de forma automática. El sistema operativo puede reconocer que más de una aplicación utiliza el mismo código destino porque carga y enlaza dicho código. Puede utilizar esta información para cargar una única copia del código destino y enlazarlo a ambas aplicaciones, en lugar de tener que cargar una copia para cada aplicación. Facilita a los desarrolladores de software independientes extender la funcionalidad de un sistema operativo ampliamente utilizado, tal como Linux. Un desarrollador puede implementar una nueva función que puede ser útil a una variedad de aplicaciones y empaquetarla como un módulo de enlace dinámico.

Con ***enlace dinámico en tiempo de ejecución***, algunos de los enlaces son pospuestos hasta el tiempo de ejecución. Las referencias externas a los módulos destino quedan en el programa cargado.

Cuando se realiza una llamada a un módulo ausente, el sistema operativo localiza el módulo, lo carga y lo enlaza al módulo llamante.

Se ha visto que la carga dinámica permite que se pueda mover un módulo de carga entero; sin embargo, la estructura del módulo es estática, permaneciendo sin cambios durante la ejecución del proceso y de una ejecución a otra. Sin embargo, en algunos casos, no es posible determinar antes de la ejecución qué módulos objeto se necesitarán. Esta situación es tipificada por las aplicaciones de procesamiento de transacciones, como las de un sistema de reserva de vuelos o una aplicación bancaria.

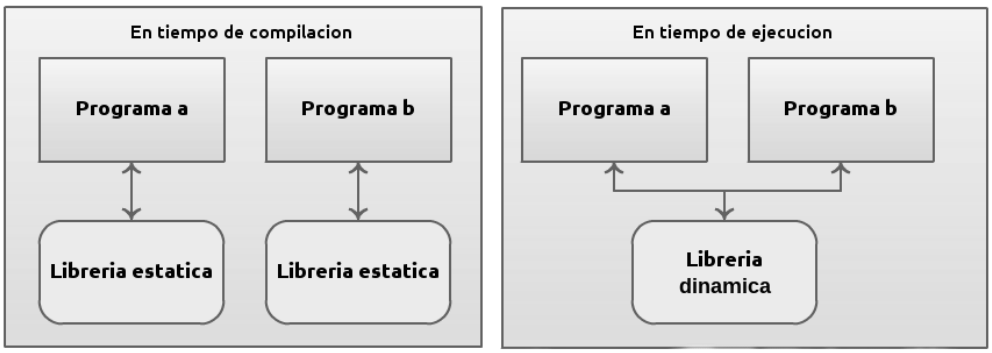
La naturaleza de la transacción especifica qué módulos de programa se requieren, y éstos son cargados y enlazados con el programa principal. La ventaja del uso de un montador dinámico es que no es necesario asignar memoria a unidades del programa a menos que dichas unidades se referencien. Esta capacidad se utiliza para dar soporte a sistemas de segmentación.

Una mejora adicional es posible: una aplicación no necesita conocer los nombres de todos los módulos o puntos de entrada que pueden llamarse. Por ejemplo, puede escribirse un programa de dibujo para trabajar con una gran variedad de trazadores, cada uno de los cuales se gestiona por un controlador diferente. La aplicación puede aprender el nombre del trazador que está actualmente instalado en el sistema por otro proceso o buscando en un fichero de configuración. Esto permite al usuario de la aplicación instalar un nuevo trazador que no exista en el tiempo en que la aplicación se escribió.

Montaje/enlace dinámico y bibliotecas compartidas.

Algunos sistemas operativos sólo permiten el ***montaje estático***[3], mediante el cual las bibliotecas de lenguaje del sistema se tratan como cualquier otro módulo objeto y son integradas por el cargador dentro de la imagen binaria del programa.

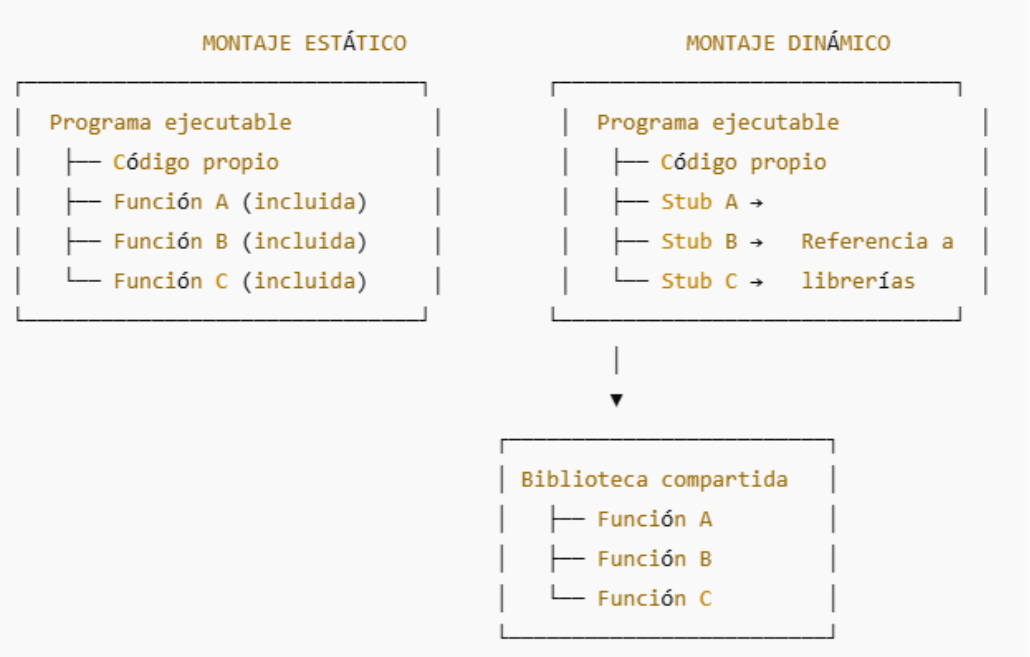
Utilizando este mecanismo, cada programa de un sistema deberá incluir una copia de su biblioteca de lenguaje (o al menos de las rutinas a las que haga referencia el programa) dentro de la imagen ejecutable. Este requisito hace que se desperdicie tanto espacio de disco como memoria principal.



Con el montaje dinámico, se incluye un ***stub*** dentro de la imagen binaria para cada referencia a una rutina de biblioteca. El stub es un pequeño fragmento de código que indica cómo localizar la rutina adecuada de biblioteca residente en memoria o cómo cargar la biblioteca si esa rutina no está todavía presente.

Cuando se ejecuta el stub, éste comprueba si la rutina necesaria ya se encuentra en memoria; si no es así, el programa carga en memoria la rutina. En cualquiera de los casos, el stub se sustituye así mismo por la dirección de la rutina y ejecuta la rutina. Así, la siguiente vez que se ejecute ese segmento de código concreto, se ejecutará directamente la rutina de biblioteca, sin tener que realizar de nuevo el montaje dinámico.

[3] El montaje estático de memoria se refiere a la asignación de espacio de memoria a objetos, variables, instrucciones y constantes en tiempo de compilación, donde su tamaño y ubicación permanecen fijos durante la ejecución del programa. En contraste con la asignación dinámica, la estática no requiere gestión explícita de memoria durante la ejecución.



Con este mecanismo, todos los procesos que utilicen una determinada biblioteca de lenguaje sólo necesitan ejecutar una copia del código de la biblioteca.

Esta funcionalidad puede ampliarse a las actualizaciones de las bibliotecas de código (como por ejemplo las destinadas a corregir errores). Puede sustituirse una biblioteca por una nueva versión y todos los programas que hagan referencia a la biblioteca emplearán automáticamente la versión más reciente.

Sin el mecanismo de montaje dinámico, sería necesario volver a montar todos esos programas para poder acceder a la nueva biblioteca.

Para que los programas no ejecuten accidentalmente versiones nuevas e incompatibles de las bibliotecas, suele incluirse información de versión tanto en el programa como en la biblioteca. Puede haber más de una versión de una biblioteca cargada en memoria y cada programa utilizará su información de versión para decidir qué copia de la biblioteca hay que utilizar. Los cambios de menor entidad retendrán el mismo número de versión, mientras que para los cambios de mayor entidad se incrementará ese número.

De este modo, sólo los programas que se compilen con la nueva versión de la biblioteca se verán afectados por los cambios incompatibles incorporados en ella. Otros programas montados antes de que se instalara la nueva biblioteca continuarán utilizando la antigua. Este sistema se conoce también con el nombre de mecanismo de bibliotecas compartidas.

A diferencia de la carga dinámica, el montaje dinámico suele requerir algo de ayuda por parte del sistema operativo. Si los procesos de la memoria están protegidos unos de otros, entonces el sistema operativo será la única entidad que pueda comprobar si la rutina necesaria se encuentra dentro del espacio de memoria de otro proceso y será también la única entidad que pueda permitir a múltiples procesos acceder a las mismas direcciones de memoria.

Instancia Única	Múltiples procesos acceden a una sola copia de la biblioteca en **memoria principal**.	Optimización del espacio en RAM.
Versionado	El cargador identifica y asigna la versión específica requerida por cada binario.	Prevención de errores por incompatibilidad.
Coexistencia	El sistema permite mantener diferentes versiones de una biblioteca en el espacio físico.	Soporte para aplicaciones con distintas dependencias.

Carga.

Aunque el ciclo de compilación y enlace termina con la creación del ejecutable, el proceso se completa con el ***cargador*** (loader). Este componente del sistema operativo toma el ejecutable del disco, lo sitúa en la memoria principal y establece el espacio de direcciones necesario para que la CPU pueda comenzar la ejecución.

En este escenario, el ***cargador (loader)*** coloca el ***módulo de carga***[4] en la memoria principal, comenzando en la ubicación *x*.

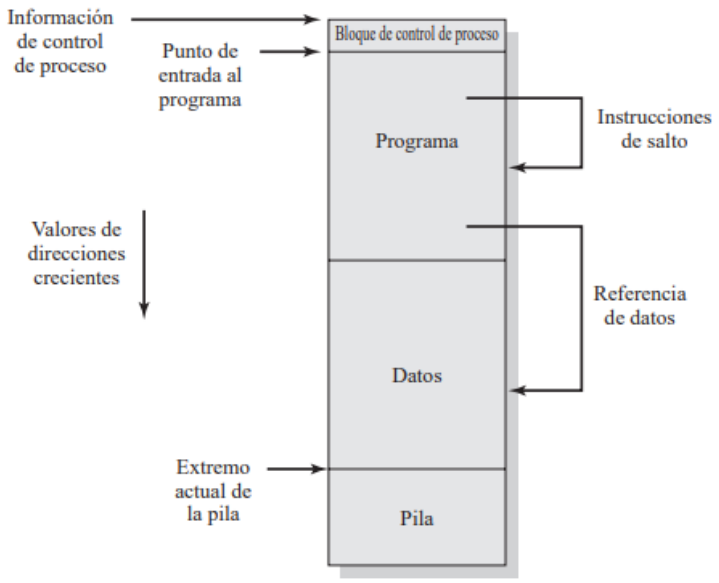


Ilustración 5 - Requisitos de direccionamiento para un proceso.

[4] Es el insumo directo del cargador (loader), que se copia en memoria principal. Contiene código máquina listo para ejecutarse (direcciones resueltas o con información suficiente para el loader) y cabeceras con información para el sistema operativo (dónde ubicar secciones, puntos de entrada, permisos, etc.).

En la carga del programa, se debe satisfacer el requisito de direccionamiento mostrado en la figura anterior. En general, se pueden seguir tres técnicas diferentes:

- Carga absoluta.
- Carga reubicable.
- Carga dinámica en tiempo real.

Carga absoluta.

La ***carga absoluta*** (absolute loading) es el método más simple de carga de programas en memoria. Consiste en cargar el programa en una dirección fija de memoria que ya está especificada.

Un cargador absoluto requiere que un módulo de carga dado debe cargarse siempre en la misma ubicación de la memoria principal. Por lo tanto, en el módulo de carga presentado al cargador, todas las referencias a direcciones deben ser direcciones de memoria principal específicas o absolutas (también llamadas direcciones reales o físicas).

```
CARGA ABSOLUTA
--------------
Programa con direcciones fijas (ej. 0x2000)
↓
Loader lo copia exactamente en 0x2000
↓
Se ejecuta sin reubicación
```

La asignación de valores de direcciones específicas a referencias de programa dentro de un programa lo puede hacer el programador o se hacen en tiempo de compilación o ensamblado.

Tiempo de asociación	Función
Tiempo de programación	El programador especifica directamente en el propio programa todas las direcciones físicas reales.
Tiempo de compilación o ensamblado	El programa contiene referencias a direcciones simbólicas y el compilador o ensamblador las convierte a direcciones físicas reales.
Tiempo de carga	El compilador o ensamblador produce direcciones relativas. El cargador las traduce a direcciones absolutas cuando se carga el programa.
Tiempo de ejecución	El programa cargador retiene direcciones relativas. El hardware del procesador las convierte dinámicamente a direcciones absolutas.

La primera opción tiene varias desventajas. Primero, cada programador debe conocer la estrategia de asignación para colocar los módulos en memoria principal. Segundo, si se hace

cualquier modificación al programa que implique inserciones o borrados en el cuerpo del módulo, entonces todas las direcciones deben alterarse.

Por lo tanto, es preferible permitir que las referencias de memoria dentro de los programas se expresen simbólicamente, y entonces resolver dichas referencias simbólicas en tiempo de compilación o ensamblado.

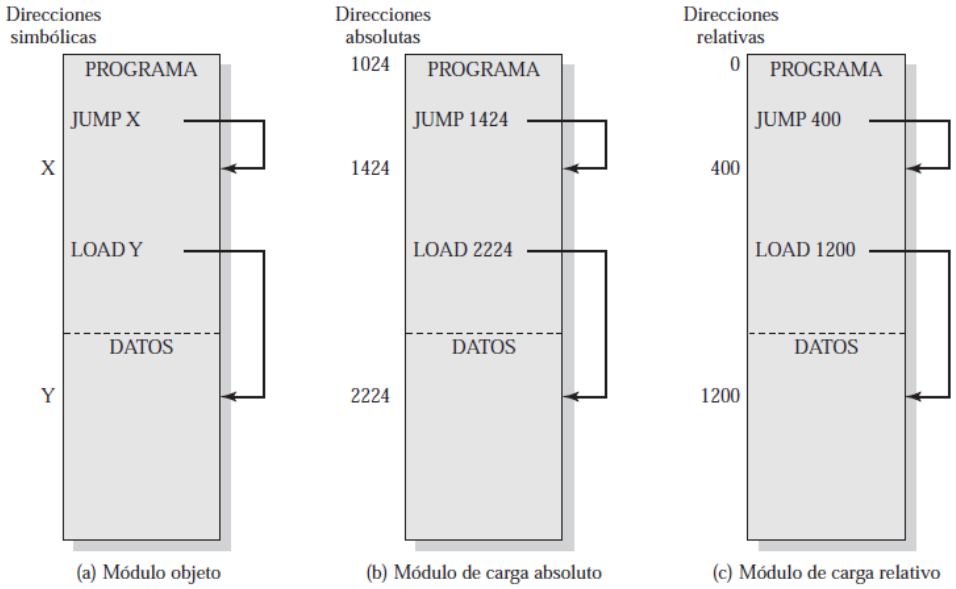


Ilustración 6 - Módulos de carga absolutos y reubicables.

Cada referencia a una instrucción o elemento de datos se representa inicialmente como un símbolo. A la hora de preparar el módulo para su entrada a un cargador absoluto, el ensamblador o compilador convertirá todas estas referencias a direcciones específicas (en el ejemplo de la figura, el módulo se carga en la dirección inicial de 1024.

Con la carga absoluta, normalmente, el compilador/ensamblador genera un ejecutable con direcciones absolutas (no relativas ni reubicables). El cargador simplemente copia el programa tal cual en la dirección indicada. No hay ningún tipo de reubicación, ni traducción, ni relocalización en tiempo de carga.

Carga reubicable.

La desventaja de enlazar referencias de memoria a direcciones específicas antes de la carga es que el módulo de carga resultante sólo se puede colocar en una región específica de memoria principal. Cuando muchos programas comparten memoria principal, podría no ser deseable decidir al principio en qué región de la memoria se debe cargar un módulo particular. Es mejor tomar esta decisión en tiempo de carga.

```
VENTAJAS DE LA CARGA REUBICABLE:
✓ El mismo programa puede cargarse en distintas posiciones de memoria.
✓ Flexibilidad: evita conflictos de direcciones fijas.
✓ Permite usar la memoria libre de manera eficiente.
✓ Facilita la multitarea al mover programas según convenga.
```

Por lo tanto, necesitamos un módulo de carga que pueda ubicarse en cualquier lugar de la memoria principal.

Para satisfacer este nuevo requisito, el ensamblador o compilador no produce direcciones de memoria reales (direcciones absolutas), sino direcciones relativas a algún punto conocido, tal como el inicio del programa.

El comienzo del módulo de carga se asigna a la dirección relativa 0, y el resto de las referencias de memoria dentro del módulo se expresan relativas al comienzo del módulo.

Con todas las referencias de la memoria expresadas en formato relativo, colocar el módulo en el lugar adecuado se convierte en una tarea simple para el cargador. Si el módulo se carga al comienzo de la ubicación *x*, entonces el cargador debe simplemente añadir *x* a cada referencia de la memoria cuando carga el módulo en memoria.

Para asistir en esta tarea, el módulo cargado debe incluir información que dice al cargador donde están las referencias de memoria y cómo se van a interpretar (normalmente relativo al origen del programa, pero también es posible relativo a algún otro punto del programa, tal como la ubicación actual). El compilador o ensamblador prepara este conjunto de información, lo que se denomina normalmente diccionario de reubicación.

```
Programa (direcciones relativas)
0 ──> instrucción1
1 ──> instrucción2
2 ──> instrucción3

Si lo cargamos en memoria empezando en 2000:
2000 ──> instrucción1
2001 ──> instrucción2
2002 ──> instrucción3

Si lo cargamos en memoria empezando en 5000:
5000 ──> instrucción1
5001 ──> instrucción2
5002 ──> instrucción3
```

Carga dinámica en tiempo real.

En las explicaciones que hemos dado hasta el momento, todo el programa y todos los datos de un proceso deben encontrase en memoria física para que ese proceso pueda ejecutarse. En consecuencia, el tamaño de un proceso está limitado por el tamaño de la memoria física. Para obtener una mejor utilización del espacio de memoria, podemos utilizar un mecanismo de ***carga dinámica***.

Los cargadores reubicables son comunes y proporcionan beneficios obvios si se comparan con los cargadores absolutos. Sin embargo, en un entorno de multiprogramación, incluso en uno que no dependa de la memoria virtual, el esquema de carga reubicable no es adecuado.

Para maximizar la utilización de la memoria principal, sería importante poder intercambiar las imágenes de los procesos en diferentes localizaciones en diferentes momentos.

Por lo tanto, un programa, una vez cargado, debe poder intercambiarse a disco y a memoria en diferentes ubicaciones. Esto sería imposible si las referencias de la memoria se limitan a direcciones absolutas en tiempo de carga inicial.

La alternativa es posponer el cálculo de una dirección absoluta hasta que se necesite realmente en tiempo de ejecución. Para este propósito, el módulo de carga se carga en la memoria principal con todas las referencias de la memoria en formato relativo. Hasta que una instrucción no se ejecuta realmente, no se calcula la dirección absoluta.

Con la carga dinámica, una rutina no se carga hasta que se la invoca; todas las rutinas se mantienen en disco en un formato de carga reubicable[5]. Según este método, el programa principal se carga en la memoria y se ejecuta. Cuando una rutina necesita llamar a otra rutina, la rutina que realiza la invocación comprueba primero si la otra ya ha sido cargada, si no es así, se invoca el cargador de montaje reubicable para que cargue en memoria la rutina deseada y para que actualice las tablas de direcciones del programa con el fin de reflejar este cambio. Después, se pasa el control a la rutina recién cargada.

[5] Carga reubicable se refiere a un programa o módulo que puede ser cargado en cualquier ubicación disponible en la memoria, sin necesidad de ser modificado previamente. A diferencia de las cargas absolutas, que deben ser cargadas en una dirección específica y no pueden moverse, las cargas reubicables permiten una mayor flexibilidad en la asignación de memoria, lo que es crucial para la gestión eficiente de recursos.

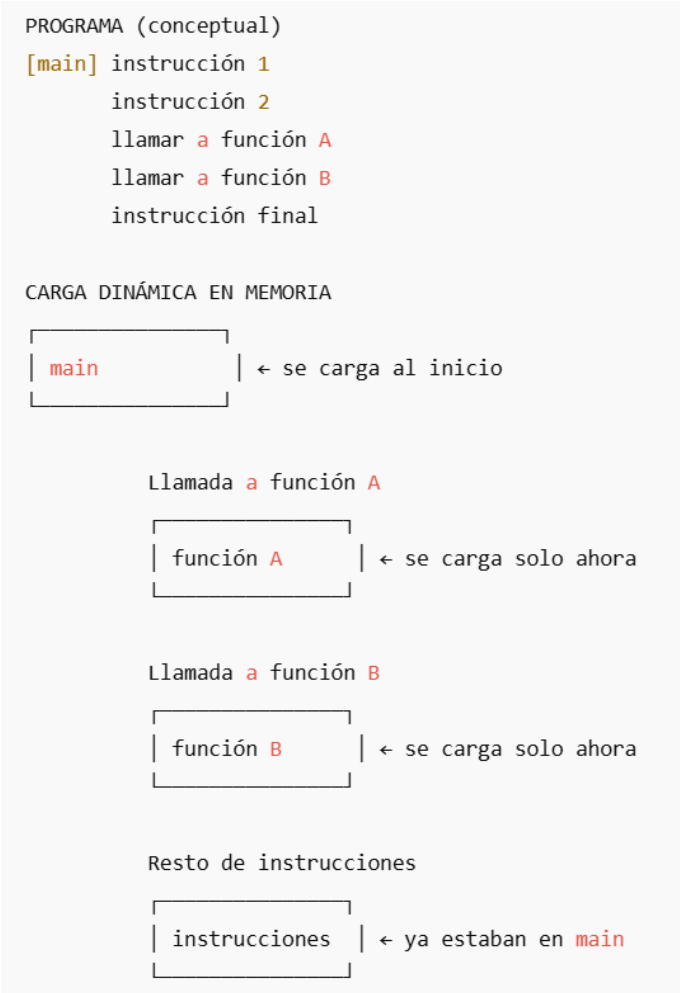
Método	Momento de Asignación	Movilidad en Memoria
Estática	Tiempo de compilación.	**Inexistente:** Dirección física invariable.
Reubicable	Tiempo de carga.	**Estática:** Una vez asignada, la ubicación es inalterable.
Dinámica	Tiempo de ejecución.	**Flexible:** El proceso es relocalizable en cualquier momento.

La ventaja del mecanismo de carga dinámica es que una rutina no utilizada no se cargará nunca en memoria. Este método resulta particularmente útil cuando se necesitan grandes cantidades de código para gestionar casos que sólo ocurren de manera infrecuente, como por ejemplo rutinas de error. En este caso, aunque el tamaño total del programa pueda ser grande, la porción que se utilice (y que por tanto se cargue) puede ser mucho más pequeña.

El mecanismo de carga dinámica no requiere de ningún soporte especial por parte del sistema operativo. ***Es responsabilidad de los programadores*** diseñar sus programas para poder aprovechar dicho método. Sin embargo, los sistemas operativos pueden ayudar al programador proporcionándole rutinas de biblioteca que implementen el mecanismo de carga dinámica.

Para asegurar que esta función no degrada el rendimiento, generalmente se realiza en el ***hardware de procesador especial*** (normalmente la MMU) en lugar de llevarse a cabo por software.

El cálculo dinámico de direcciones proporciona una flexibilidad total. Un programa se carga en cualquier región de la memoria principal. A continuación, la ejecución del programa se puede interrumpir y el programa se puede intercambiar entre disco y memoria, para posteriormente intercambiarse en una localización diferente.



Referencias y bibliografía de la sección.

- Stallings pág. 331.
- Silberschatz 8-1-4/ 8-1-5.

Direccionamiento: Direcciones físicas, lógicas y virtuales.

Una dirección generada por la CPU se denomina comúnmente ***dirección lógica***, mientras que una dirección vista por la unidad de memoria (es decir, la que se carga en el registro de direcciones de memoria de la memoria) se denomina comúnmente ***dirección física***.

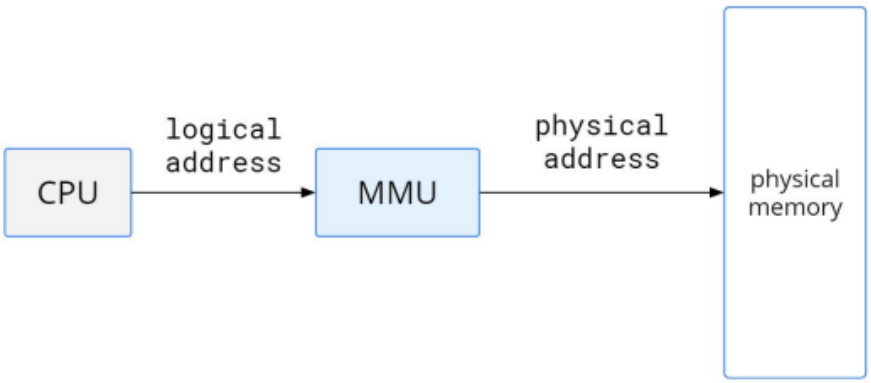
Una ***dirección absoluta*** es una dirección lógica que ya ha sido convertida a una dirección fija en memoria por el compilador o montador. Si el programa se carga en otro lugar, fallará.

Los métodos de reasignación en tiempo de compilación y en tiempo de carga generan direcciones lógicas y físicas idénticas. Sin embargo, el esquema de reasignación de direcciones en tiempo de ejecución hace que las direcciones lógica y física difieran. En este caso, usualmente decimos que la dirección lógica es una ***dirección virtual***. A veces se utilizan los términos dirección lógica y dirección virtual de manera intercambiable.

Método	Cuándo ocurre la traducción	Relación entre direcciones
Tiempo de compilación	El compilador ya sabe dónde se va a cargar el programa en RAM, así que genera direcciones físicas directamente.	Lógica = Física (son iguales)
Tiempo de carga	Se traduce cuando el programa se carga en memoria. El cargador ajusta las direcciones para que coincidan con el espacio donde se pone el programa.	Lógica = Física (ya quedan fijas al cargar)
Tiempo de ejecución	La traducción se hace mientras el programa corre, usando hardware especial (como la MMU — Memory Management Unit).	Lógica ≠ Física (el proceso "ve" unas direcciones pero en realidad está en otras)

El conjunto de todas las direcciones lógicas generadas por un programa es lo que se denomina un ***espacio de direcciones lógicas***; el conjunto de todas las direcciones físicas correspondientes a estas direcciones lógicas es un ***espacio de direcciones físicas***. Así, en el esquema de reasignación de direcciones en tiempo de ejecución, decimos que los espacios de direcciones lógicas y físicas difieren.

La correspondencia entre direcciones virtuales y físicas en tiempo de ejecución es establecida por un dispositivo hardware que se denomina ***unidad de gestión de memoria*** (***MMU***, memory management unit).



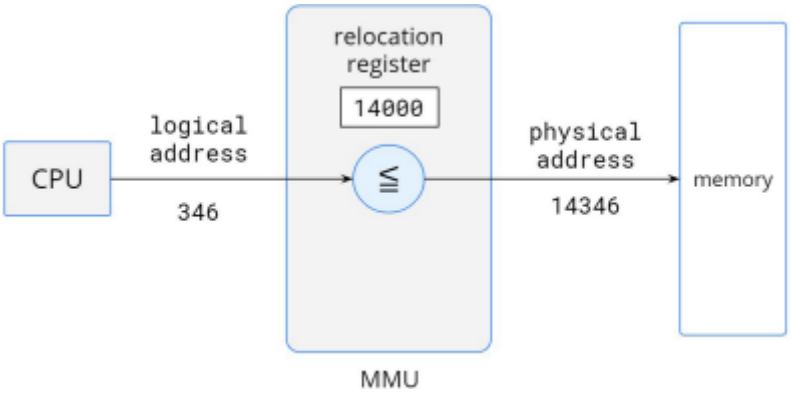
Podemos seleccionar entre varios métodos distintos para establecer esta correspondencia. Por el momento, vamos a ilustrar esta operación de asociación mediante un esquema MMU simple.

El registro base se denominará ahora ***registro de reubicación***. El valor contenido en el registro de reubicación suma a todas las direcciones generadas por un proceso de usuario en el momento de enviarlas a memoria. Por ejemplo, si la base se encuentra en la dirección 14000, cualquier intento del usuario de direccionar la posición de memoria cero se reubicará dinámicamente en la dirección 14000; un acceso a la ubicación 346 se convertirá en la ubicación 14346.

Ubicación	**Integrada en el mismo chip de la CPU.** En los procesadores actuales, ambos forman una sola pieza de silicio.
Entrada	Dirección lógica (la que genera el programa al ejecutarse).
Salida	Dirección física (la ubicación real en la **memoria principal**).
Protección	Impide que un proceso acceda a datos ajenos comparando la dirección con el registro límite.

Ilustración 7 - La Unidad de Gestión de Memoria (MMU).

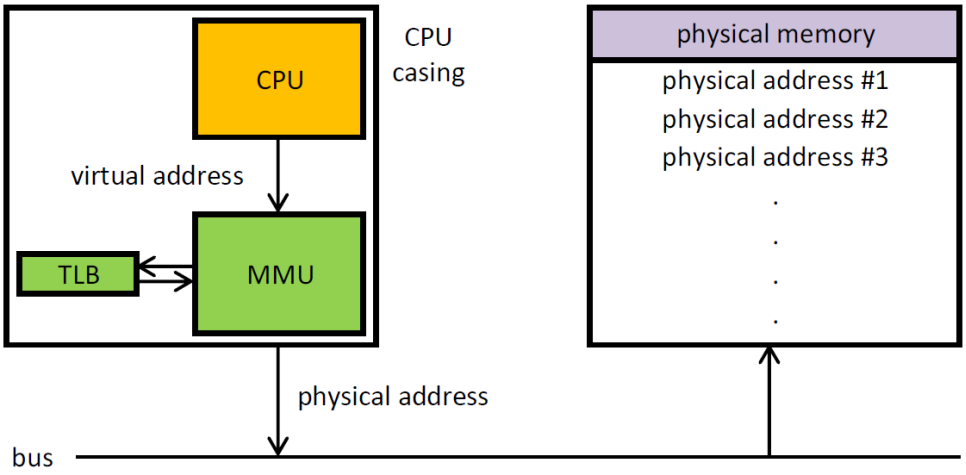
El programa de usuario nunca ve las direcciones físicas reales. El programa puede crear un puntero a la ubicación 346, almacenarlo en memoria, manipularlo y compararlo con otras direcciones, siempre como el número 346. Sólo cuando se lo utiliza como dirección de memoria (por ejemplo, en una operación de lectura o escritura indirecta) se producirá la reubicación en relación con el registro base.



El programa de usuario maneja direcciones lógicas y el hardware de conversión (mapeo) de memoria convierte esas direcciones lógicas en direcciones físicas.

La ubicación final de una dirección de memoria referenciada no se determina hasta que se realiza esa referencia.

Ahora tenemos dos tipos diferentes de direcciones: direcciones lógicas (en el rango de 0 a max) y direcciones físicas s (en el rango de R + 0 a R + max para un valor base R). El usuario sólo genera direcciones lógicas y piensa que el proceso se ejecuta en las ubicaciones comprendidas entre 0 y max. El programa de usuario suministra direcciones, lógicas y estas direcciones lógicas deben ser convertidas en direcciones físicas antes de utilizarlas.



CPU: Central Processing Unit
MMU: Memory Management Unit
TLB: Translation lookaside buffer

El concepto de un espacio de direcciones lógicas que se acopla a un espacio de direcciones físicas separado resulta crucial para una adecuada gestión de la memoria.

Concepto	Origen y Control	Naturaleza Técnica
Dirección Lógica	**Unidad de Ejecución (CPU)**	Dirección abstracta y relativa que utiliza el programa.
Mapeo	**Lógica de la MMU**	Traducción dinámica en tiempo real mediante registros base y límite.
Dirección Física	**Hardware de Memoria**	Coordenada absoluta y real para acceder a las celdas de la **memoria principal**.

Referencias y bibliografía de la sección.

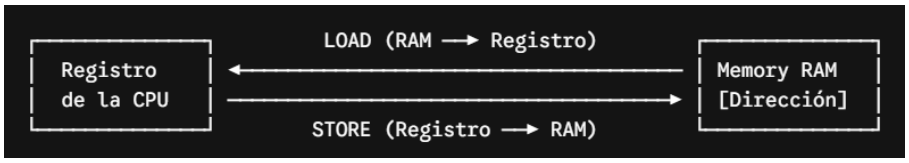
- Silberschatz 8-1-3.

CPU, memoria principal y reubicación simple.

Los programas de la computadora deben hallarse en la memoria principal (a veces también llamada memoria RAM, Random Access Memory, Memoria de Acceso Aleatorio) para ser ejecutados.

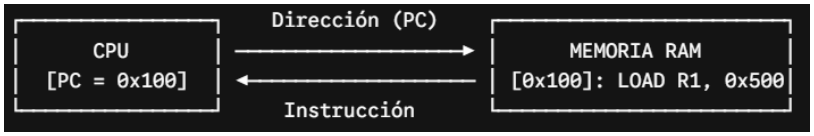
La ***memoria principal*** y los ***registros integrados*** dentro del propio procesador son las únicas áreas de almacenamiento a las que la CPU puede acceder directamente, siendo la memoria principal la única área de almacenamiento de gran tamaño (millones o miles de millones de bytes). Habitualmente, se implementa con algún tipo de tecnología de semiconductores, que forma una matriz de palabras de memoria y cada palabra tiene su propia dirección.

La interacción entre ***CPU*** (el procesador) y la ***memoria principal*** se consigue a través de una secuencia de carga (load) o almacenamiento (store) de instrucciones en direcciones específicas de memoria. La instrucción load mueve una palabra desde la memoria principal a un registro interno de la CPU, mientras que la instrucción store mueve el contenido de un registro a la memoria principal.



Aparte de las cargas y almacenamientos explícitos, la CPU puede cargar automáticamente instrucciones desde la memoria principal para su ejecución.

La CPU extrae instrucciones de la memoria de acuerdo con el valor del ***contador de programa (PC)***. Estas instrucciones pueden provocar operaciones adicionales de carga o de almacenamiento en direcciones de memoria específicas.

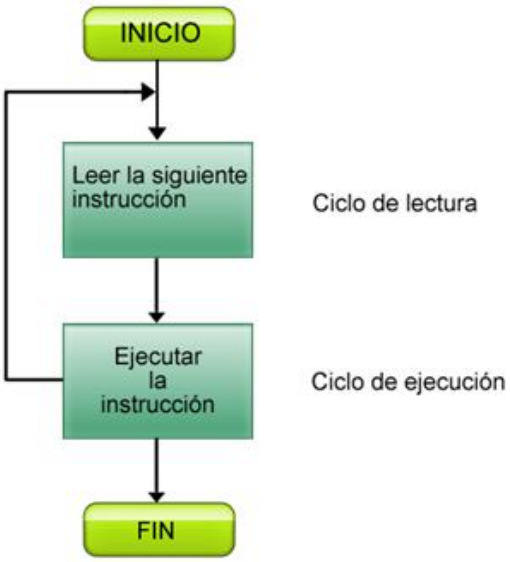


Un ciclo típico de ejecución de una instrucción procedería en primer lugar, por ejemplo, a extraer una instrucción de la memoria. Dicha instrucción se decodifica y puede hacer que se extraigan de memoria una serie de operandos.

Instrucción	Indica **qué hacer**.	Una receta de cocina.
Operando	Indica **con qué hacerlo**.	Los ingredientes (harina, huevos).

Después de haber ejecutado la instrucción con esos operandos, es posible que se almacenen los resultados de nuevo en memoria. La ***unidad de memoria***[6] tan sólo "ve" un flujo de direcciones de memoria y no sabe cómo se generan esas direcciones (mediante el contador de programa, mediante indexación, indirección, direcciones literales, etc.) ni tampoco para qué se utilizan (instrucciones o datos).

Por lo tanto, podemos ignorar el cómo genera el programa las direcciones de memoria; lo único que nos interesa es la secuencia de direcciones de memoria generadas por el programa en ejecución.



Hardware básico.

Ya dijimos que la memoria principal y los registros integrados dentro del propio procesador son las únicas áreas de almacenamiento a las que la CPU puede acceder directamente. Hay

[6] Se refiere al hardware de la memoria principal (ej.: módulo RAM).

instrucciones de máquina que toman como argumentos direcciones de memoria, pero no existe ninguna instrucción que acepte direcciones de disco.

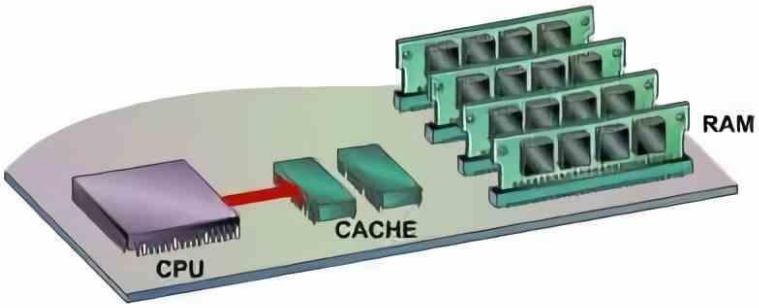
Por lo tanto, todas las instrucciones en ejecución y los datos utilizados por esas instrucciones deberán encontrarse alojados en uno de los dispositivos de almacenamiento de acceso directo.

Característica	Memoria Principal (RAM)	Almacenamiento Secundario (Disco)
Acceso	**Directo:** La CPU puede saltar a cualquier celda mediante su dirección física.	**Indirecto:** Requiere que un controlador y el SO gestionen la transferencia.

Si los datos no se encuentran en memoria, deberán llevarse hasta allí antes de que la CPU pueda operar con ellos.

Generalmente, puede accederse a los registros integrados en la CPU en un único ciclo del reloj del procesador. La mayoría de los procesadores pueden decodificar instrucciones y realizar operaciones simples con el contenido de los registros a la velocidad de una o más operaciones por cada tic[7] de reloj.

No podemos decir lo mismo de la memoria principal, a la que se accede mediante una transacción del bus de memoria. El acceso a memoria puede requerir muchos ciclos de reloj del procesador para poderse completar, en cuyo caso el procesador necesitará normalmente detenerse, ya que no dispondrá de los datos requeridos para completar la instrucción que esté ejecutando. Esta situación es intolerable, debido a la gran frecuencia con la que se accede a la memoria. El remedio consiste en añadir una memoria rápida entre la CPU y la memoria principal. Estamos hablando de la memoria caché.



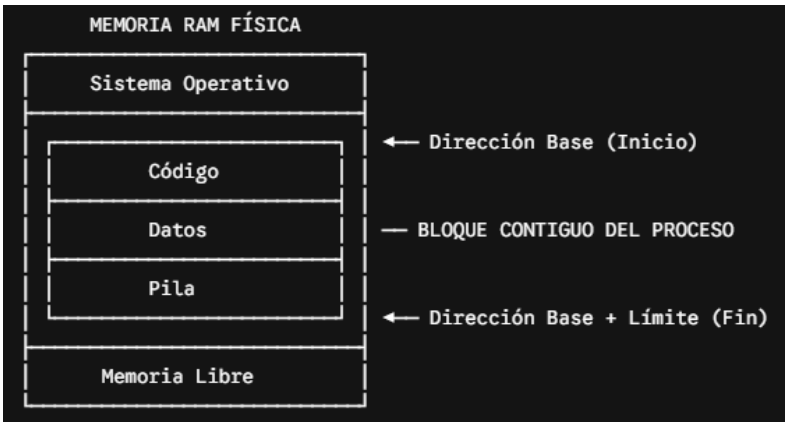
[7] El "tic de reloj" de un procesador se refiere a un ciclo de reloj, la unidad básica de tiempo que sincroniza las operaciones internas de la CPU. La velocidad del reloj, medida en Gigahertz (GHz), determina cuántos de estos ciclos (ticks) puede ejecutar la CPU por segundo. En esencia, el reloj es como un metrónomo que marca el ritmo de las operaciones del procesador.

No sólo debe preocuparnos la velocidad relativa del acceso a la memoria física, sino que también se debe garantizar una correcta operación que proteja al sistema operativo de los posibles accesos por parte de los procesos de los usuarios y que también proteja los procesos de usuario entre sí. Esta protección debe ser proporcionada por el hardware y puede implementarse de diversas formas. Por ejemplo, podemos proporcionar esta protección utilizando dos registros, usualmente una ***base*** y un ***límite***.

Primero se debe asegurar que cada proceso disponga de un espacio de memoria separado. Para hacer esto, debemos poder determinar el rango de direcciones legales a las que el proceso pueda acceder y garantizar también que el proceso sólo acceda a esas direcciones legales.

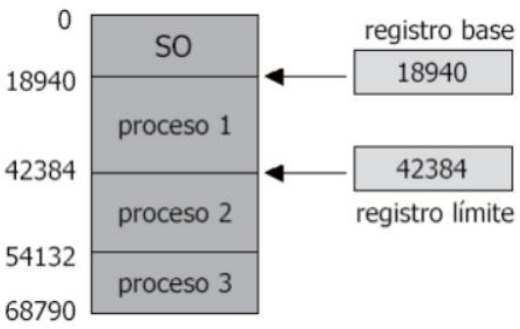
Reubicación simple con registros base y límite.

La solución sencilla utiliza una versión muy ***simple*** de la ***reubicación dinámica***[8]. Lo que hace es asociar el espacio de direcciones de cada proceso sobre una parte distinta de la memoria física, de una manera simple.



Se considera "simple" porque con este método el proceso completo se mapea como un bloque contiguo de memoria.

[8] La reubicación dinámica en memoria permite que un proceso se mueva a diferentes ubicaciones físicas en la memoria principal durante su ejecución.



La solución clásica de reubicación simple, que se utilizaba en máquinas desde la CDC 6600 (la primera supercomputadora del mundo) hasta el Intel 8088 (el corazón de la IBM PC original), es equipar cada CPU con dos registros de hardware especiales, conocidos comúnmente como los registros ***base*** y ***límite***.

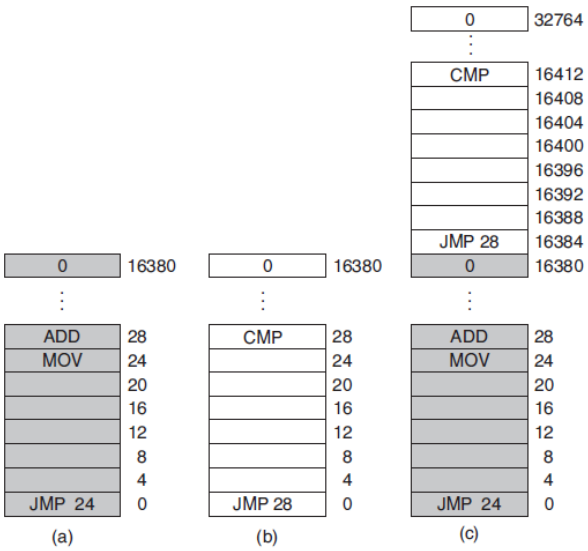
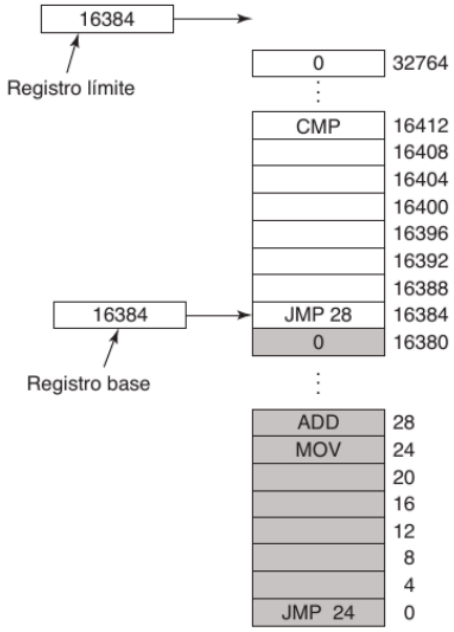


Ilustración 8 - Ilustración del problema de reubicación. (a) Un programa de 16 KB. (b) Otro programa de 16 KB. (c) Los dos programas cargados consecutivamente en la memoria.

Cuando se utilizan estos registros, los programas se cargan en ubicaciones consecutivas de memoria en donde haya espacio y sin reubicación durante la carga (parte c de la ilustración anterior).



Cuando se ejecuta un proceso, el registro base se carga con la dirección física donde empieza el programa en memoria y el registro límite se carga con la longitud del programa.

Observemos que en la figura anterior hay dos procesos en memoria, el primero se carga en la dirección 0 y con longitud (limite) de 16Kb (16384 bytes) y el segundo tiene como base la dirección 16384 y longitud 16Kb (en este caso el límite se carga en la dirección 32764).

Cada vez que un proceso hace referencia a la memoria (por ejemplo, para leer o escribir datos), el hardware de la CPU comprueba que la dirección lógica generada por el proceso sea menor que el valor del registro límite; si es igual o mayor, se genera una excepción (trap) y se aborta el acceso por violación de memoria.

Si la dirección es válida, el hardware suma de manera automática el valor del registro base a la dirección lógica para obtener la dirección física, antes de enviarla al bus de memoria.

El uso de registros base y límite es una manera fácil de proporcionar a cada proceso su propio espacio de direcciones privado, ya que a cada dirección de memoria que se genera en forma automática se le suma el contenido del registro base antes de enviarla a memoria.

En muchas implementaciones, los registros base y límite están protegidos de tal forma que sólo el sistema operativo puede modificarlos. Las instrucciones para escribir en los registros base y límite son instrucciones privilegiadas que solo pueden ejecutarse cuando la CPU está en modo kernel.

Una desventaja de la reubicación usando los registros base y límite es la necesidad de realizar una suma y una comparación en cada referencia a memoria. Las comparaciones se pueden hacer con rapidez, pero las sumas son lentas debido al tiempo de propagación del acarreo, a menos que se utilicen circuitos especiales de suma.

Vale aclarar que cuando hablamos de ***reubicación simple*** nos estamos refiriendo la forma más básica de reubicación dinámica. Sin embargo, en la amplia literatura de Sistemas Operativos,

podemos encontrar el término "reubicación dinámica" a secas, referido a sistemas más avanzados como la ***paginación***.

Reasignación de direcciones.

Usualmente, los programas residen en una unidad de almacenamiento en forma de archivos ejecutables binarios. Para poder ejecutarse, un programa deberá ser cargado en memoria y colocado dentro de un proceso.

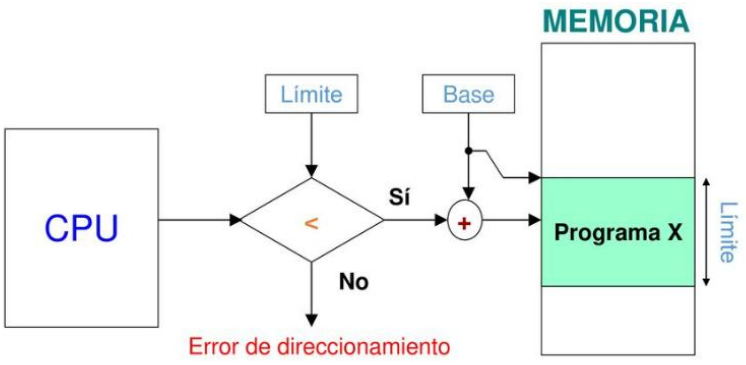
Dependiendo del mecanismo de gestión de memoria que se utilice, el proceso puede desplazarse entre unidad de almacenamiento y memoria durante su ejecución. Los programas o trabajos en el disco que están esperando ser cargados en memoria para su ejecución forman la ***cola de entrada***.

El procedimiento normal consiste en seleccionar uno de los procesos de la cola de entrada y cargar dicho proceso en memoria. Durante su ejecución, la CPU captura y procesa continuamente las instrucciones y los datos del proceso almacenados en la RAM. Cuando el proceso finaliza (o es terminado por el sistema), el sistema operativo libera su espacio de memoria y lo marca como disponible para futuros procesos.

Etapa	Ubicación Física	Estado y Función
Programa (Binario)	Disco (Almacenamiento).	Archivo pasivo. Contiene el código máquina y la estructura del programa.
Cola de Entrada	Disco / Planificador.	Conjunto de procesos residentes en disco que compiten por el acceso a la RAM.
Carga (Loading)	De Disco a RAM.	El cargador sitúa el código en memoria, resolviendo direcciones según el método (reubicable o dinámico).
Ejecución	**Memoria Principal**	El sistema operativo asigna la CPU para procesar las instrucciones y datos residentes en RAM.
Liberación	Memoria Principal.	El espacio se marca como disponible; el SO actualiza sus tablas para permitir nuevas cargas.

La mayoría de los sistemas permiten que un proceso de usuario resida en cualquier parte de la memoria física. Así, aunque el espacio de direcciones de la computadora comience en 00000, la primera dirección del proceso de usuario no tiene por qué ser 00000. Esta técnica afecta a las direcciones que el programa de usuario puede utilizar. En la mayoría de los casos, el programa de usuario tendrá que recorrer varios pasos (algunos de los cuales son opcionales)

antes de ser ejecutado. A lo largo de estos pasos, las direcciones pueden representarse de diferentes formas.



Las direcciones del programa fuente son generalmente simbólicas. Normalmente, un ***compilador*** se encargará de reasignar estas direcciones simbólicas[9] a direcciones reubicables[10] (por ejemplo, "14 bytes a partir del comienzo de este módulo").

El ***editor de montaje*** o ***cargador*** se encargará, a su vez, de reasignar las direcciones reubicables a direcciones absolutas[11] (por ejemplo, 74014). Cada operación de reasignación constituye una relación de un espacio de direcciones a otro.

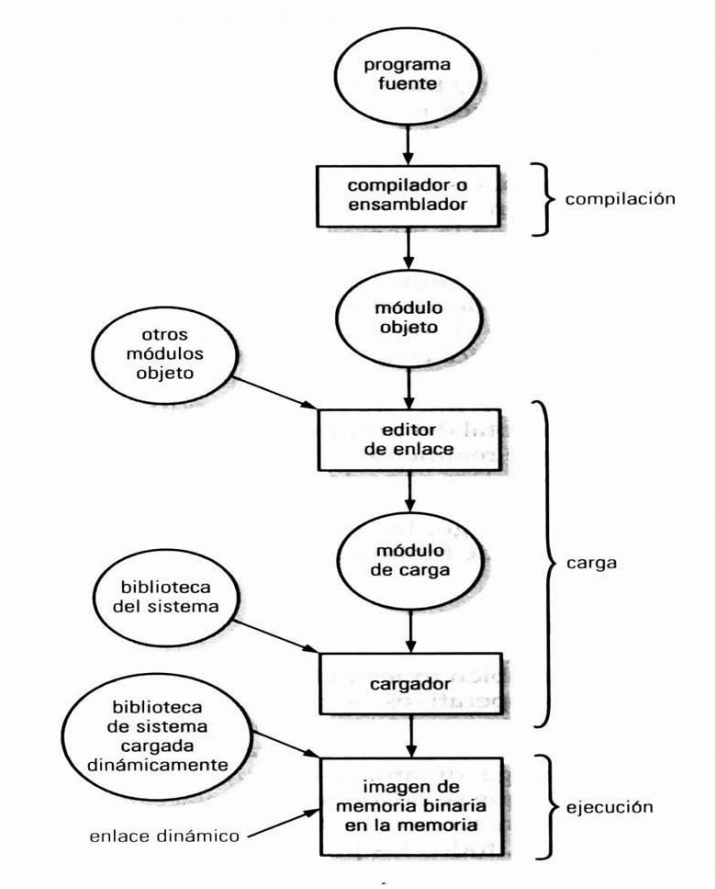
Clásicamente, la reasignación de las instrucciones y los datos a direcciones de memoria puede realizarse en cualquiera de los pasos:

- **Tiempo de compilación.** Si sabemos, en el momento de realizar la compilación, dónde va a residir el proceso en memoria, podremos generar código absoluto[12]. Por ejemplo, si sabemos que un proceso de usuario va a residir en una zona de memoria que comienza en la ubicación R, el código generado por el compilador comenzará en dicha ubicación y se extenderá a partir de ahí. Si la ubicación cambiase en algún instante posterior, entonces sería necesario recompilar ese código.
- **Tiempo de carga.** Si no conocemos, en tiempo de compilación, dónde va a residir el proceso en memoria, el compilador deberá generar código reubicable[13]. En este caso,

[9] La dirección simbólica de memoria es un nombre asignado por el programador a una ubicación específica de memoria para hacer que el programa sea más comprensible y fácil de leer. En lugar de referirse a la dirección física de la memoria, que puede cambiar, se usa un nombre (símbolo) para la variable o el dato, simplificando la programación y evitando errores.
[10] Una dirección reubicable o relativa es una dirección de memoria que se especifica como un desplazamiento a partir de un punto de referencia, en lugar de una ubicación fija y absoluta.
[11] Una dirección de memoria fija o absoluta es un identificador único y permanente para una ubicación específica dentro de la memoria de una computadora, apuntando siempre a la misma celda de memoria sin importar desde dónde se acceda.
[12] Tipo de archivo ejecutable o programa cuyo contenido ya tiene direcciones reales y definitivas en memoria. Proporciona la posición real de un elemento sin necesidad de un movimiento de referencia.
[13] Código reubicable es un tipo de código objeto o ejecutable en el que las direcciones de memoria no son fijas, sino relativas a algún punto base. Esto permite que el programa se pueda cargar en cualquier dirección de memoria y que, en el momento de la carga o del enlace, se le ajusten las direcciones finales.

se retarda la reasignación final hasta el momento de la carga. Si cambia la dirección inicial, tan sólo es necesario volver a cargar el código de usuario para incorporar el valor modificado.

- **Tiempo de ejecución.** Si el proceso puede desplazarse durante su ejecución desde un segmento de memoria a otro, entonces es necesario retardar la reasignación hasta el instante de la ejecución. Para que este esquema pueda funcionar, será preciso disponer de hardware especial (los procesadores incluyen registros y unidades especiales.). La mayoría de los sistemas operativos de propósito general utilizan este método.



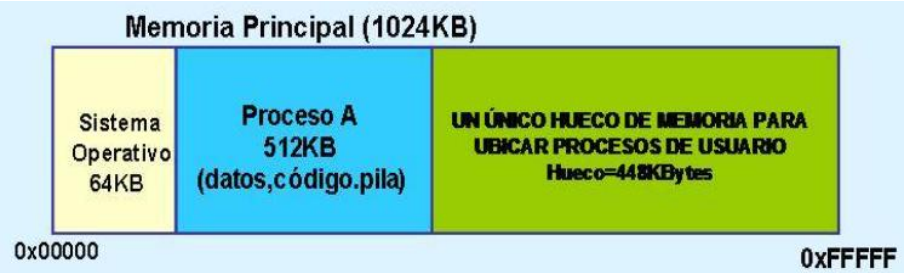
Referencias y bibliografía de la sección.

- Silberschatz 8-1-1 / 8-1-2.
- Tanenbaum 3-2.

Asignación de memoria y mapeo.

La memoria principal debe albergar tanto al sistema operativo como a los diversos procesos de usuario. Por lo tanto, necesitamos asignar las distintas partes de la memoria principal de la forma más eficiente posible.

En el pasado, era usual que la memoria esté dividida en dos particiones: una para el sistema operativo residente y otra para los procesos de usuario.



Según este esquema podemos situar al sistema operativo en la zona baja o en la zona alta de la memoria. El principal factor que afecta a esta decisión es la ubicación del ***vector de interrupciones***[14]. Puesto que el vector de interrupciones se encuentra a menudo en la parte baja de la memoria, los programadores tienden a situar también el sistema operativo en dicha zona.

1. **Evento:** Presionas una tecla.
2. **Hardware:** El controlador de teclado genera una interrupción (ej. IRQ 1).
3. **Consulta:** El procesador busca en la posición 1 del **vector de interrupciones**.
4. **Salto:** La CPU carga la dirección encontrada en el vector y comienza a ejecutar la rutina de lectura de teclado del Sistema Operativo.
5. **Retorno:** Al terminar, se restaura el proceso que estaba corriendo antes.

Normalmente, querremos tener varios procesos de usuario residentes en memoria al mismo tiempo. Por lo tanto, tenemos que considerar cómo asignar la memoria disponible a los procesos que se encuentren en la cola de entrada, esperando a ser cargados en memoria. En este esquema de ***asignación contigua de memoria***, cada proceso está contenido en una única sección contigua de memoria.

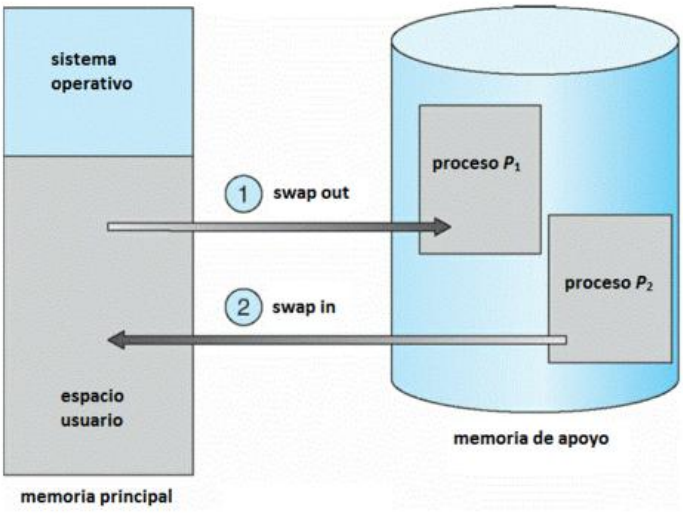
Actualmente, la memoria ya no se divide en dos grandes bloques estáticos. En los sistemas operativos modernos (como Linux, Windows o macOS), la memoria se gestiona de forma altamente dinámica y flexible.

[14] El vector de interrupciones es una tabla de direcciones que el procesador usa para saber dónde saltar cuando ocurre una interrupción. Es una estructura que asocia cada tipo de interrupción (como una del teclado, del temporizador o un error) con la rutina específica de manejo que debe ejecutarse para atenderla.

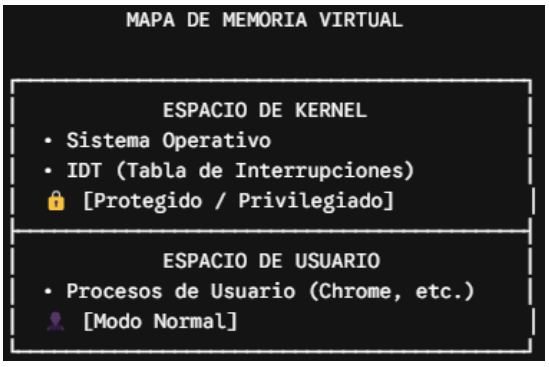
En sistemas como Windows o Red Hat (Linux), la gestión de memoria ha evolucionado hacia esquemas de ***memoria virtual***. Se mantiene una separación estricta, pero ya no se habla de "zona alta o baja" de la memoria, sino ***de espacio de kernel*** y ***espacio de usuario***.

El espacio de kernel es el área de memoria donde reside el núcleo del SO. Está protegido y los procesos de usuario no pueden acceder a él directamente.

El espacio de usuario es donde corren las aplicaciones (Chrome, Spotify, un servidor web).



Sigue existiendo el vector de interrupciones. En arquitecturas x86-64 (la que usan casi todas las PC y servidores Red Hat), se llama ***IDT*** (Interrupt Descriptor Table).



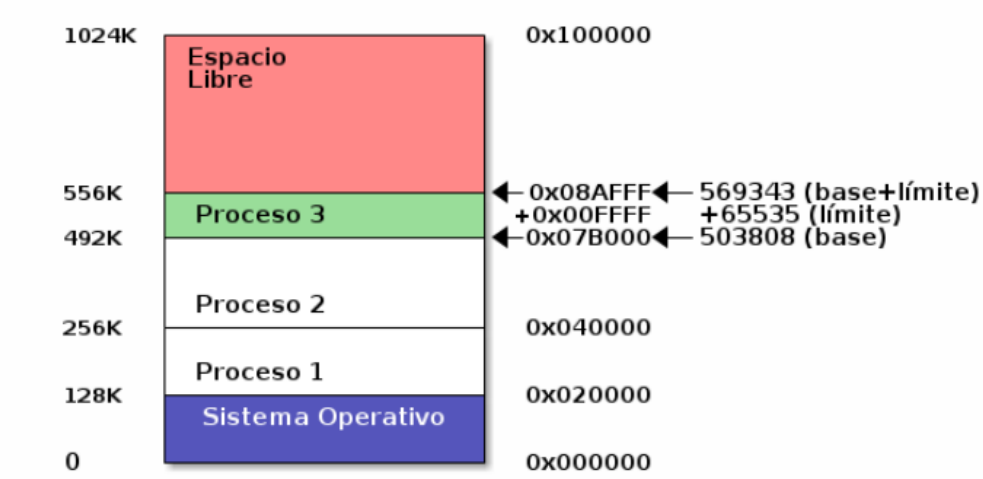
Mapeo de memoria y protección.

Como vimos anteriormente, la conversión (mapping, mapeo) de memoria y la protección se puede proporcionar utilizando un registro de reubicación (o base) y un registro límite. El

registro de reubicación contiene el valor de la dirección física más pequeña, mientras que el registro límite contiene el rango de las direcciones lógicas.

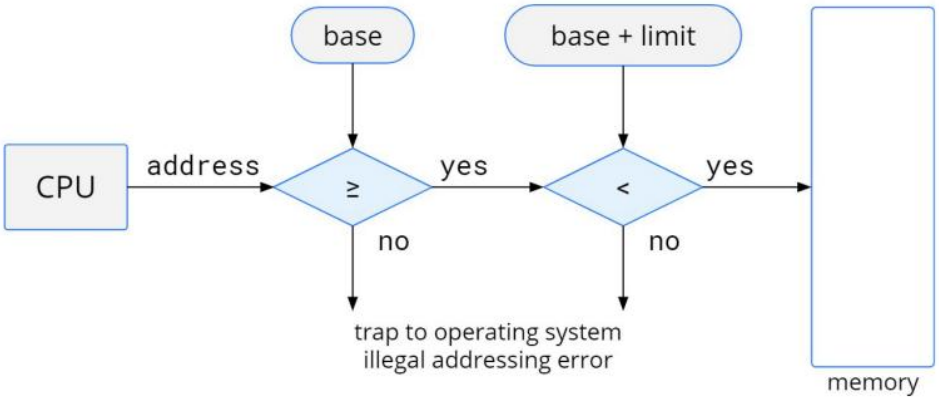
Con los registros de reubicación y de límite, cada dirección lógica debe ser inferior al valor contenido en el registro límite; la ***MMU*** convertirá la dirección lógica dinámicamente sumándole el valor contenido en el registro de reubicación. Esta dirección es la que se envía a la memoria.

La anterior, es la descripción fundamental de la ***reubicación dinámica***.



Cuando el planificador de la CPU selecciona un proceso para su ejecución, el despachador carga en los registros de reubicación y de límite los valores correctos, como parte del proceso de cambio de contexto.

Puesto que todas las direcciones generadas por la CPU se comparan con estos registros, este mecanismo nos permite proteger tanto al sistema operativo como a los programas y datos de los otros usuarios de las posibles modificaciones que pudiera realizar este proceso en ejecución.



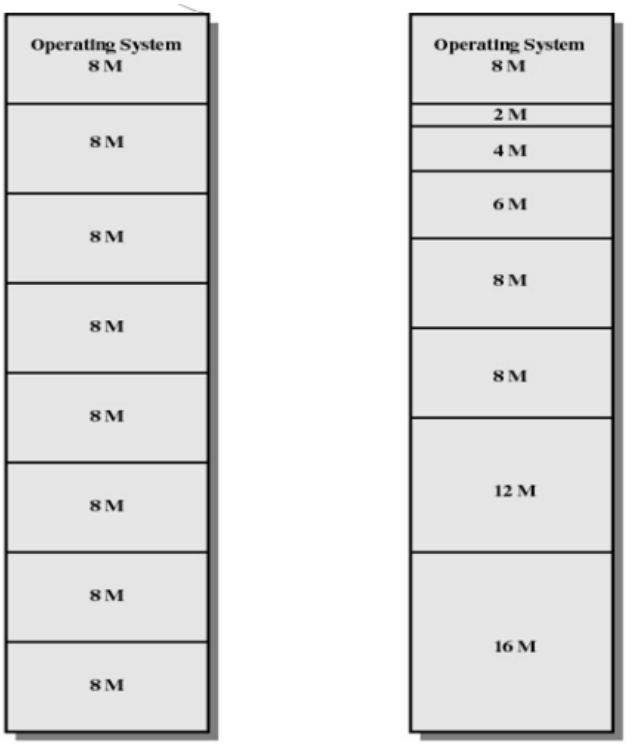
La ***reubicación dinámica*** es un esquema basado en registro de reubicación que constituye una forma efectiva de permitir que el tamaño del sistema operativo cambie dinámicamente. Esta flexibilidad resulta deseable en muchas situaciones. Por ejemplo, el sistema operativo contiene código y espacio de búfer para los controladores de dispositivo; si un controlador de dispositivo (u otro servicio del sistema operativo) no se utiliza comúnmente, no conviene mantener el código y los datos correspondientes en la memoria, ya que podríamos utilizar dicho espacio para otros propósitos. Este tipo de código se denomina en ocasiones *código transitorio del sistema operativo*, ya que se carga y descarga según sea necesario.

Por lo tanto, la utilización de este tipo de código modifica el tamaño del sistema operativo durante la ejecución del programa.

Asignación de memoria contigua.

Uno de los métodos más simples para asignar la memoria consiste en dividirla en varias particiones de tamaño fijo. Cada partición puede contener exactamente un proceso, de modo que ***el grado de multiprogramación estará limitado por el número de particiones disponibles***.

Existen dos estrategias de partición fija, utilizar particiones de igual tamaño o particiones de distinto tamaño.

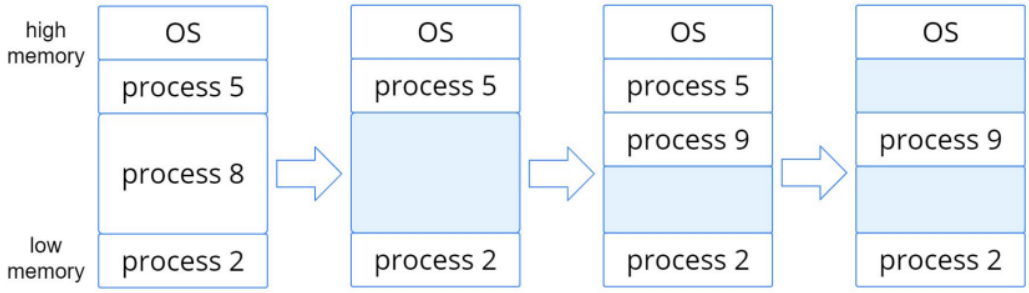


(a) Equal-size partitions (b) Unequal-size partitions

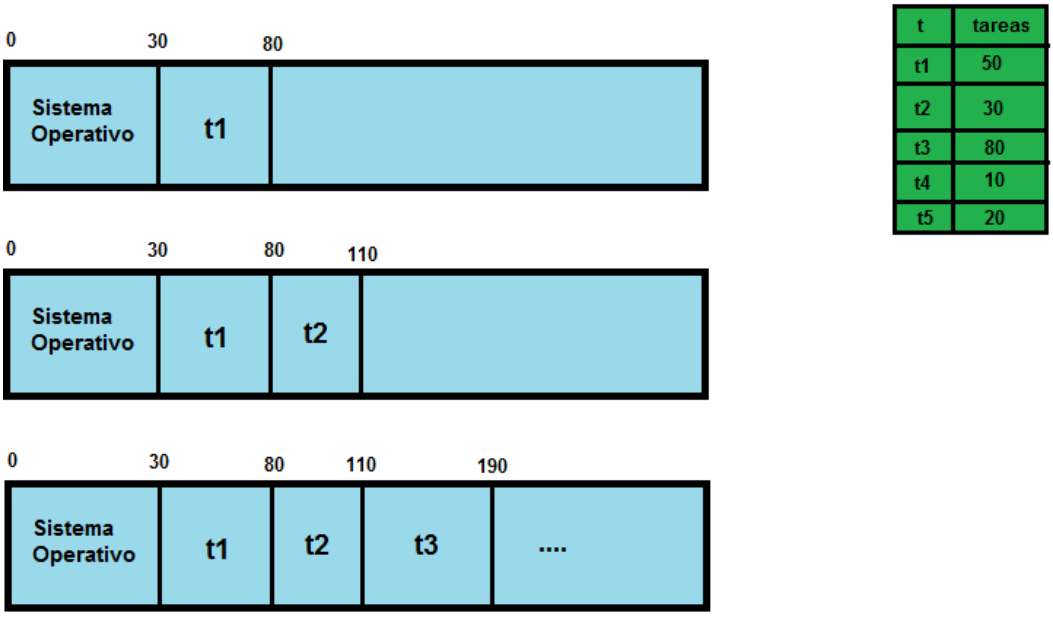
En este método de particiones múltiples, cuando una partición está libre, se selecciona un proceso de la cola de entrada y se lo carga en dicha partición. Cuando el proceso termina, la partición pasa a estar disponible para otro proceso. Este método (denominado ***MFT***) fue usado originalmente por el sistema operativo IBM 0S/360, pero ya no se lo utiliza.

En el esquema de particiones fijas, el sistema operativo mantiene una tabla que indica qué partes de la memoria están disponibles y cuáles están ocupadas. Inicialmente, toda la memoria está disponible para los procesos de usuario y se considera como un único bloque de gran tamaño de memoria disponible, al que se denomina agujero (o hueco).

Una alternativa al esquema de particiones fijas es el de ***particiones dinámicas***. En este caso, la memoria se divide en bloques a medida que los procesos llegan. El tamaño de cada partición depende del proceso que se va a cargar. A diferencia de las particiones fijas, el tamaño de las particiones se ajusta dinámicamente.



Cuando llega un proceso y necesita memoria, busquemos un agujero lo suficientemente grande como para albergar este proceso. Si lo encontramos, sólo se asigna la memoria justa necesaria, manteniendo el resto de la memoria disponible para satisfacer futuras solicitudes.

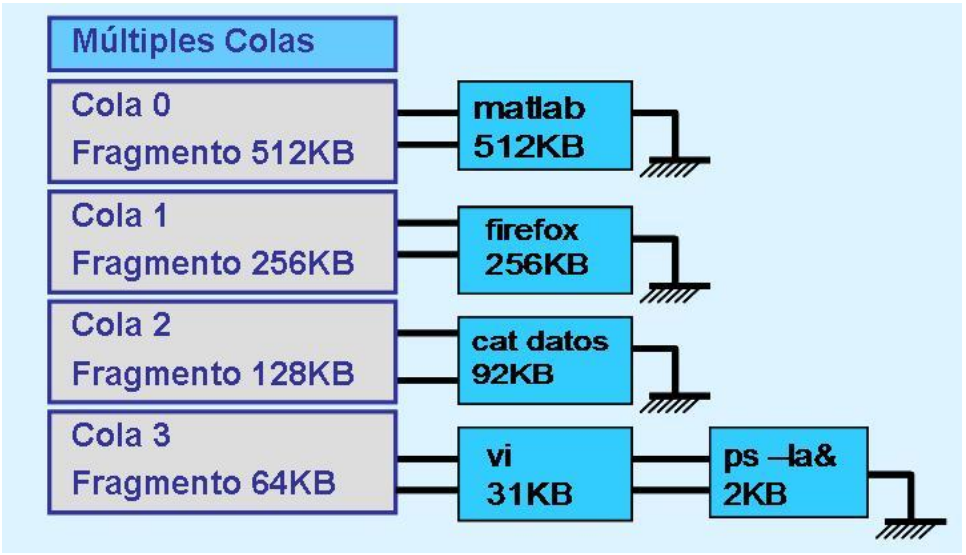


t	tareas
t1	50
t2	30
t3	80
t4	10
t5	20

A medida que los procesos entran en el sistema, se introducen en una ***cola de entrada***. El sistema operativo toma en consideración los requisitos de memoria en cada proceso y la cantidad de memoria disponible a la hora de determinar a qué procesos se le asigna la memoria.

Cuando asignamos espacio a un proceso, se carga en memoria y puede comenzar a competir por el uso de la CPU. Cuando un proceso termina, libera su memoria, que el sistema operativo podrá rellenar a continuación con otro proceso extraído de la cola de entrada.





En cualquier momento determinado, tendremos una lista de tamaños de bloque disponibles y una cola de entrada de procesos. El sistema operativo puede ordenar la cola de entrada de acuerdo con algún algoritmo de planificación, asignándose memoria a los procesos hasta que finalmente, los requisitos de memoria del siguiente proceso ya no puedan satisfacerse, es decir, hasta que no haya ningún bloque de memoria (o agujero) disponible que sea lo suficientemente grande como para albergar al siguiente proceso.

El sistema operativo puede entonces esperar hasta que haya libre un bloque de memoria lo suficientemente grande, o puede examinar el resto de la cola de entrada para ver si pueden satisfacerse los requisitos de memoria de algún otro proceso, que necesite un bloque de memoria menor.

En general, en cualquier momento determinado tendremos un conjunto de agujeros de diversos tamaños, dispersos por toda la memoria. Cuando llega un proceso y necesita memoria, el sistema explora ese conjunto en busca de un agujero que sea lo suficientemente grande como para albergar el proceso. Si el agujero es demasiado grande, se lo dividirá en dos

partes, asignándose una parte al proceso que acaba de llegar y devolviendo la otra al conjunto de agujeros.

Cuando el proceso termina, libera su bloque de memoria, que volverá a colocarse en el conjunto de agujeros.

Si el nuevo agujero es adyacente a otros agujeros, se combinan esos agujeros adyacentes para formar otros de mayor tamaño.

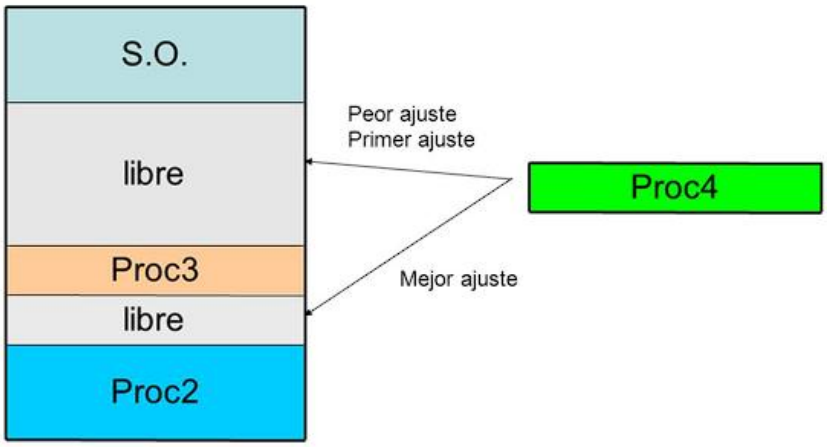
Asignación	La memoria se trata como un bloque único y contiguo. Las particiones se crean "a medida" según la solicitud.
Estructura	El sistema mantiene una tabla de **huecos** (bloques de memoria disponibles de diversos tamaños).
Cola de Entrada	El SO selecciona procesos cuyo requisito de memoria sea $\leq$ al tamaño de algún hueco disponible.
Liberación	Cuando un proceso termina, el SO intenta **coalescer** (unir) el hueco recién liberado con huecos adyacentes.

En este punto, el sistema puede tener que comprobar si hay procesos esperando a que se les asigne memoria y si esta nueva memoria liberada y recombinada permite satisfacer las demandas de algunos de los procesos en espera.

Este procedimiento constituye un caso concreto del problema general de ***asignación dinámica de espacio de memoria***, que se ocupa de cómo satisfacer una solicitud de tamaño *n* a partir de una lista de agujeros libres.

Hay muchas soluciones (algoritmos) a este problema, y las estrategias más comúnmente utilizadas para seleccionar un agujero libre entre el conjunto de agujeros disponibles son las de primer ajuste, mejor ajuste y peor ajuste.

- **Primer ajuste.** Se asigna el primer agujero que sea lo suficientemente grande. La exploración puede comenzar desde el principio del conjunto de agujeros o en el punto en que hubiera terminado la exploración anterior. Podemos detener la exploración en cuanto encontremos un agujero libre que sea lo suficientemente grande.
- **Mejor ajuste.** Se asigna el agujero más pequeño que tenga el tamaño suficiente. Debemos explorar la lista completa, a menos que ésta esté ordenada según su tamaño. Esta estrategia hace que se genere el agujero más pequeño posible con la memoria que sobre en el agujero original.
- **Peor ajuste.** Se asigna el agujero de mayor tamaño. De nuevo, debemos explorar la lista completa, a menos que ésta esté ordenada por tamaños. Esta estrategia genera el agujero más grande posible con la memoria sobrante del agujero original, lo que puede resultar más útil que el agujero más pequeño generado con la técnica de mejor ajuste.



Las simulaciones muestran que tanto la estrategia de primer ajuste como la de mejor ajuste son mejores que la de peor ajuste en términos del tiempo necesario y de la utilización del espacio de almacenamiento. No está claro cuál de las dos estrategias (la de primer ajuste o la de mejor ajuste) es mejor en términos de utilización del espacio de almacenamiento, pero la estrategia de primer ajuste es, generalmente, más rápida de implementar.

Estrategia	Selección del hueco	Ventaja	Desventaja
Primer ajuste	Primer hueco suficiente	Rápido	Fragmentación al inicio
Mejor ajuste	Hueco más pequeño posible	Aprovecha espacio	Deja muchos fragmentos chicos
Peor ajuste	Hueco más grande	Puede dejar huecos grandes	Desperdicia memoria

Referencias y bibliografía de la sección.

- Silberschatz 8-3.

Más sobre algoritmos para carga de procesos en memoria.

Cuando los procesos y huecos se mantienen en una lista ordenada por dirección, se pueden utilizar varios algoritmos para asignar memoria a un proceso creado (o a un proceso existente que se intercambie del disco).

Suponemos que el administrador de memoria sabe cuánta debe asignar. El algoritmo más simple es el de ***primer ajuste***: el administrador de memoria explora la lista de segmentos hasta

encontrar un hueco que sea lo bastante grande. Después el hueco se divide en dos partes, una para el proceso y otra para la memoria sin utilizar, excepto en el estadísticamente improbable caso de un ajuste exacto.

El algoritmo del primer ajuste es rápido debido a que busca lo menos posible.

Una pequeña variación del algoritmo del primer ajuste es el algoritmo del ***siguiente ajuste***. Funciona de la misma manera que el primer ajuste, excepto porque lleva un registro de dónde se encuentra cada vez que descubre un hueco adecuado. La siguiente vez que es llamado para buscar un hueco, empieza a buscar en la lista desde el lugar en el que se quedó la última vez, en vez de empezar siempre desde el principio, como el algoritmo del primer ajuste. Las simulaciones realizadas muestran que el algoritmo del siguiente ajuste tiene un rendimiento ligeramente peor que el del primer ajuste.

Otro algoritmo muy conocido y ampliamente utilizado es el del ***mejor ajuste***. Este algoritmo busca en toda la lista, de principio a fin y toma el hueco más pequeño que sea adecuado. En vez de dividir un gran hueco que podría necesitarse después, el algoritmo del mejor ajuste trata de buscar un hueco que esté cerca del tamaño actual necesario, que coincida mejor con la solicitud y los huecos disponibles.

Como ejemplo de los algoritmos de primer ajuste y de mejor ajuste, consideremos la figura siguiente.

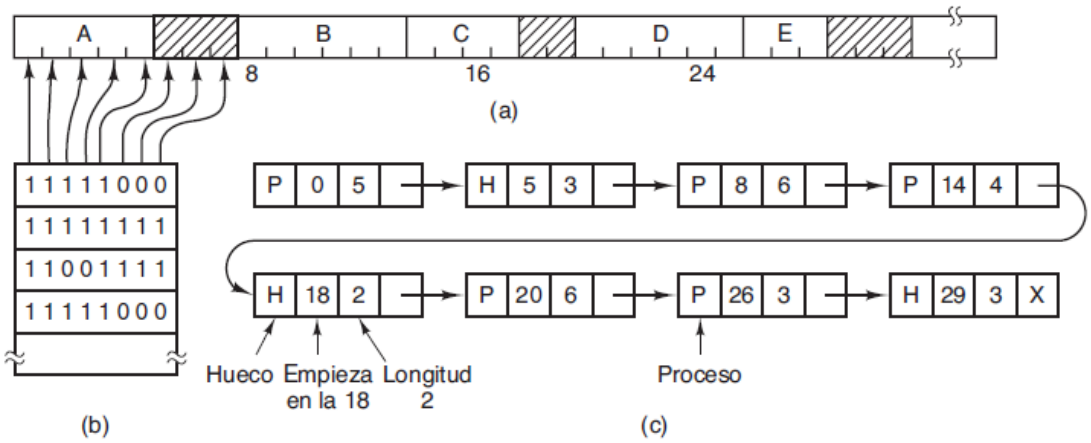


Ilustración 9 - (a) Una parte de la memoria con cinco procesos y tres huecos. Las marcas de graduación muestran las unidades de asignación de memoria. Las regiones sombreadas (0 en el mapa de bits) están libres. (b) El mapa de bits correspondiente. (c) La misma información en forma de lista.

Si se necesita un bloque de tamaño 2, el algoritmo del primer ajuste asignará el hueco en la 5, pero el del mejor ajuste asignará el hueco en la 18.

El algoritmo del mejor ajuste es más lento que el del primer ajuste, ya que debe buscar en toda la lista cada vez que se le llama. De manera sorprendente, también provoca más desperdicio de memoria que los algoritmos del primer ajuste o del siguiente ajuste, debido a que tiende a llenar la memoria con huecos pequeños e inutilizables.

El algoritmo del primer ajuste genera huecos más grandes en promedio.

Para resolver el problema de dividir las coincidencias casi exactas en un proceso y en un pequeño hueco, podríamos considerar el algoritmo del ***peor ajuste***, es decir, tomar siempre el hueco más grande disponible, de manera que el nuevo hueco sea lo bastante grande como para ser útil. La simulación ha demostrado que el algoritmo del peor ajuste no es muy buena idea tampoco.

Los cuatro algoritmos pueden ser acelerados manteniendo listas separadas para los procesos y los huecos. De esta forma, todos ellos dedican toda su energía a inspeccionar los huecos, no los procesos.

El precio inevitable que se paga por esta aceleración en la asignación es la complejidad adicional y la lentitud al desasignar la memoria, ya que un segmento liberado tiene que eliminarse de la lista de procesos e insertarse en la lista de huecos.

Si se mantienen distintas listas para los procesos y los huecos, la lista de huecos se puede mantener ordenada por el tamaño, para que el algoritmo del mejor ajuste sea más rápido.

Cuando el algoritmo del mejor ajuste busca en una lista de huecos, del más pequeño al más grande, tan pronto como encuentre un hueco que ajuste, sabrá que el hueco es el más pequeño que se puede utilizar, de aquí que se le denomine el mejor ajuste. No es necesario buscar más, como con el esquema de una sola lista. Con una lista de huecos ordenada por tamaño, los algoritmos del primer ajuste y del mejor ajuste son igual de rápidos, y hace innecesario usar el del siguiente ajuste.

Cuando los huecos se mantienen en listas separadas de los procesos, una pequeña optimización es posible. En vez de tener un conjunto separado de estructuras de datos para mantener la lista de huecos, como se hizo en la parte c de la figura anterior, la información se puede almacenar en los huecos. La primera palabra de cada hueco podría ser el tamaño del mismo y la segunda palabra un apuntador a la siguiente entrada. Los nodos de la lista de la parte c de la figura, que requieren tres palabras y un bit (P/H), ya no son necesarios.

Un algoritmo de asignación más es el denominado de ***ajuste rápido***, el cual mantiene listas separadas para algunos de los tamaños más comunes solicitados. Por ejemplo, podría tener una tabla con n entradas, en donde la primera entrada es un apuntador a la cabeza de una lista de huecos de 4 KB, la segunda entrada es un apuntador a una lista de huecos de 8 KB, la tercera entrada un apuntador a huecos de 12 KB y así sucesivamente. Por ejemplo, los huecos de 21 KB podrían colocarse en la lista de 20 KB o en una lista especial de huecos de tamaño inusual.

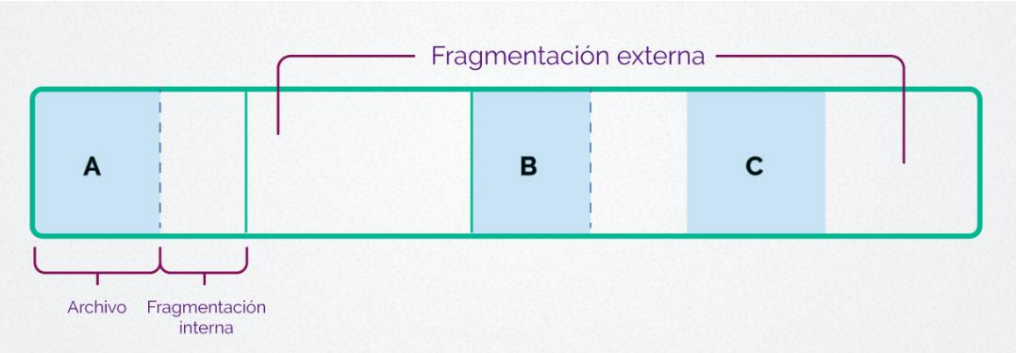
Con el algoritmo del ajuste rápido, buscar un hueco del tamaño requerido es extremadamente rápido, pero tiene la misma desventaja que todos los esquemas que se ordenan por el tamaño del hueco: cuando un proceso termina o es intercambiado, buscar en sus vecinos para ver si es posible una fusión es un proceso costoso. Si no se realiza la fusión, la memoria se fragmentará rápidamente en un gran número de pequeños huecos en los que no cabrá ningún proceso.

Referencias y bibliografía de la sección.

- Tanenbaum 3-2.

Fragmentación interna, fragmentación externa y compactación.

La fragmentación interna ocurre cuando la memoria se divide en particiones de tamaño fijo (por ejemplo, marcos de páginas en memoria principal, bloques físicos en disco). Si se asigna un bloque de datos a una o más particiones, se podría malgastar espacio en la última partición. Esto ocurrirá si la última porción de datos es más pequeña que la última partición.

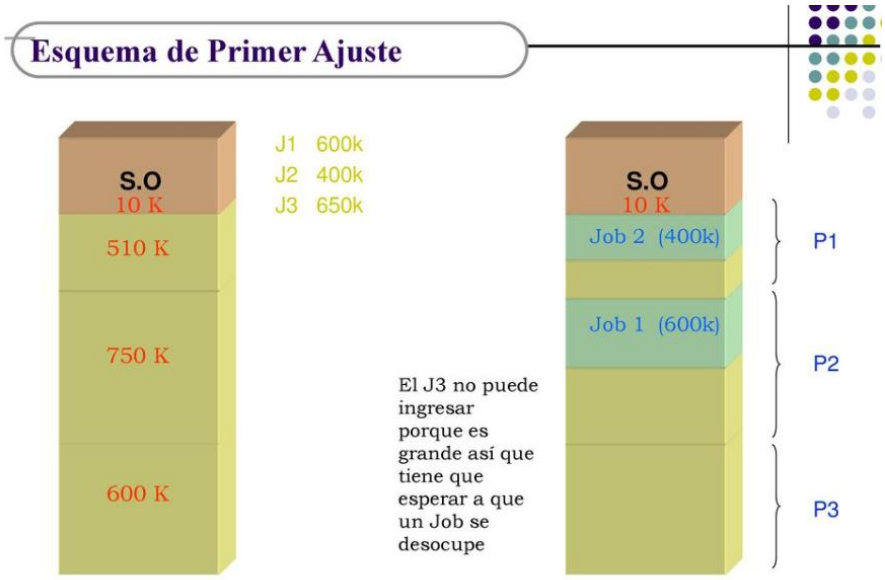


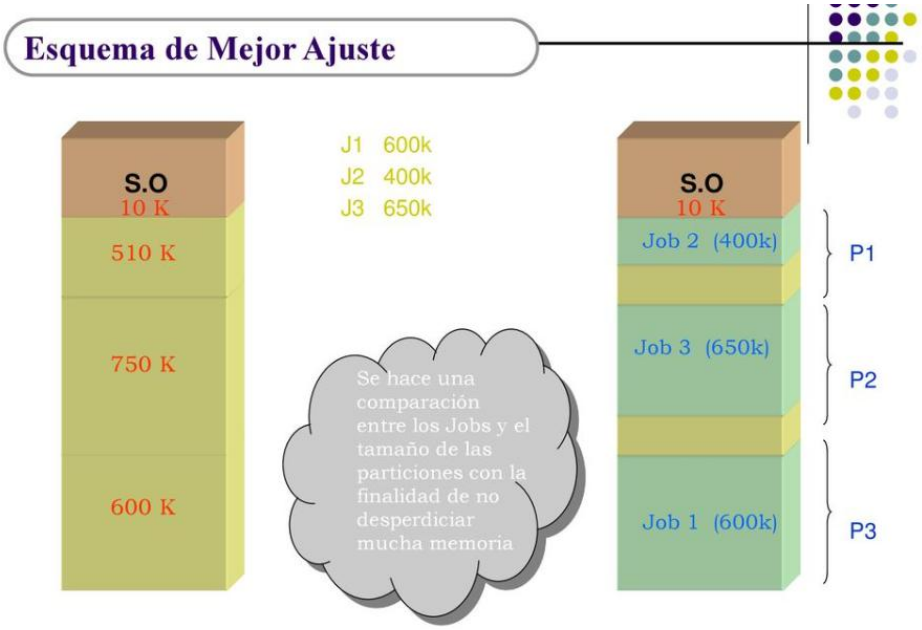
Si una partición fija mide 18.464 bytes y un proceso que solicita 18.462 bytes se aloja en ella, la partición queda ocupada por dicho proceso, generando una pérdida no utilizable de 2 bytes.

La ***fragmentación externa*** ocurre cuando la memoria se divide en particiones de tamaño variable correspondientes a los bloques de datos asignados a la memoria (por ejemplo, segmentos en memoria principal).

A medida que los segmentos o procesos se asignan y se liberan con el tiempo, se forman huecos de memoria libre entre las porciones ocupadas. Si estos huecos están dispersos y no son contiguos, el espacio libre total puede ser suficiente para un nuevo segmento, pero no es posible asignarlo por falta de contigüidad.

Tanto la estrategia de ***primer ajuste*** como la de ***mejor ajuste*** para la asignación de memoria sufren del problema de ***fragmentación externa***. A medida que se cargan procesos en memoria y se los elimina, el espacio de memoria libre se descompone en una serie de fragmentos de pequeño tamaño. El problema de la fragmentación externa aparecer cuando hay un espacio de memoria total suficiente como para satisfacer una solicitud, pero esos espacios disponibles no son contiguos; el espacio de almacenamiento está fragmentado en un gran número de pequeños agujeros.





Este problema de fragmentación puede llegar a ser muy grave. En el peor de los casos, podríamos tener un bloque de memoria libre (o desperdiciada) entre cada dos procesos; si todos estos pequeños fragmentos de memoria estuvieran en un único bloque libre de gran tamaño, podríamos ser capaces de ejecutar varios procesos más.

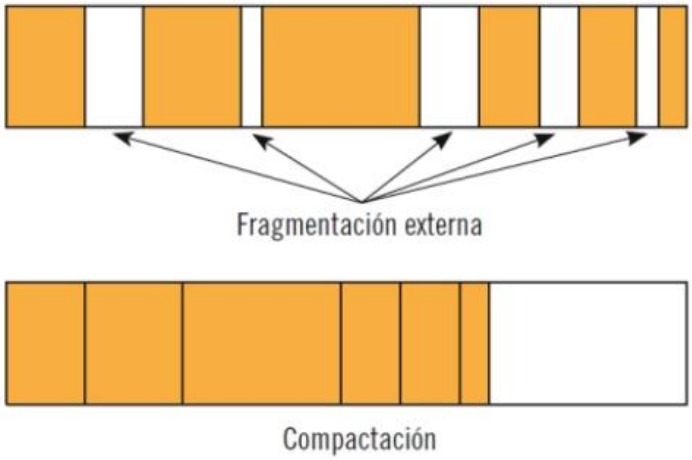
El que utilicemos el mecanismo de primer ajuste o el de mejor ajuste puede afectar al grado de fragmentación, porque la estrategia de primer ajuste es mejor para algunos sistemas, mientras que para otros resulta más adecuada la de mejor ajuste.

Otro factor diferenciador es el extremo de un bloque libre que se asigne. Es necesario determinar cuál es el fragmento que se deja como agujero restante, el situado en la parte superior o el situado en la parte inferior.

Independientemente de qué algoritmo se utilice, la fragmentación externa terminará siendo un problema.

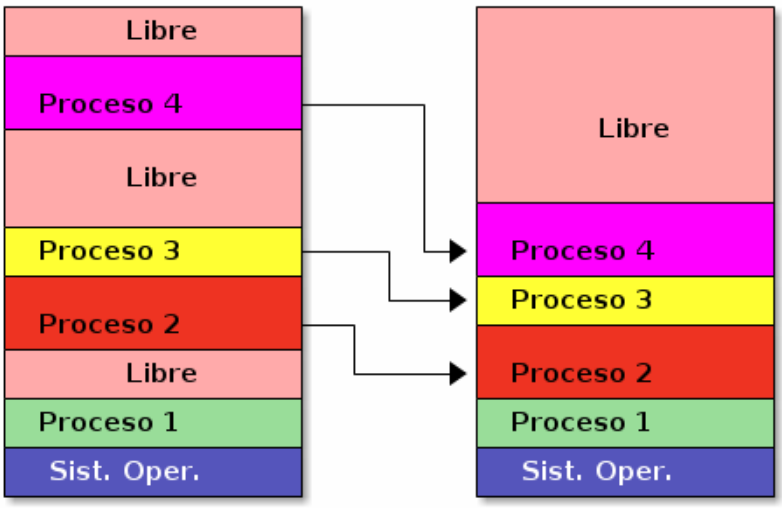
Dependiendo de la cantidad total de espacio de memoria y del tamaño medio de los procesos, esa fragmentación externa puede ser un problema grave o no.

Una técnica para eliminar la fragmentación externa es la ***compactación***: de vez en cuando, el sistema operativo desplaza los procesos en memoria, de forma que se encuentren contiguos y de este modo toda la memoria libre se encontrará unida en un bloque. Dicho de otro modo, objetivo es mover el contenido de la memoria con el fin de situar toda la memoria libre de manera contigua, para formar un único bloque de gran tamaño.



Sin embargo, la compactación no siempre es posible. Si la reubicación es estática y se lleva a cabo en tiempo de ensamblado o en tiempo de carga, no podemos utilizar el mecanismo de la compactación; la compactación sólo es posible si la reubicación es dinámica y se lleva a cabo en tiempo de ejecución. Si las direcciones se reubican dinámicamente, la reubicación sólo requerirá mover el programa y los datos y luego cambiar el registro base para reflejar la nueva dirección base utilizada.

Cuando la compactación es posible, debemos además determinar cuál es su coste. El algoritmo de compactación más simple consiste en mover todos los procesos hacia uno de los extremos de la memoria; de esta forma, todos los agujeros se moverán en la otra dirección, produciendo un único agujero de memoria disponible de gran tamaño. Sin embargo, este esquema puede ser muy caro de implementar.



La desventaja de la compactación es el hecho de que se trata de un procedimiento que consume tiempo y malgasta tiempo de procesador. Observemos que la compactación requiere la capacidad de reubicación dinámica. Es decir, debe ser posible mover un programa desde una región a otra en la memoria principal sin invalidar las referencias de la memoria de cada programa.

Otra posible solución al problema de la fragmentación externa consiste en permitir que el espacio de direcciones lógicas de los procesos no sea contiguo, lo que hace que podamos asignar memoria física a un proceso con independencia de dónde esté situada dicha memoria. Hay dos técnicas que permiten implementar esta solución y que ya hemos repasado anteriormente: la paginación y la segmentación. Asimismo, estas técnicas pueden también combinarse.

Referencias y bibliografía de la sección.

- Stallings pág. 315.
- Silberschatz 8-3-3.

Otros esquemas de administración de memoria.

Paginación.

La paginación es un esquema de gestión de memoria que permite que el espacio de direcciones físicas de un proceso no sea contiguo. La paginación evita el considerable problema de encajar fragmentos de memoria de tamaño variable en el almacén de respaldo; la mayoría de los esquemas de gestión de memoria utilizados antes de la introducción de la paginación sufrían de este problema, que surgía debido a que, cuando era necesario proceder

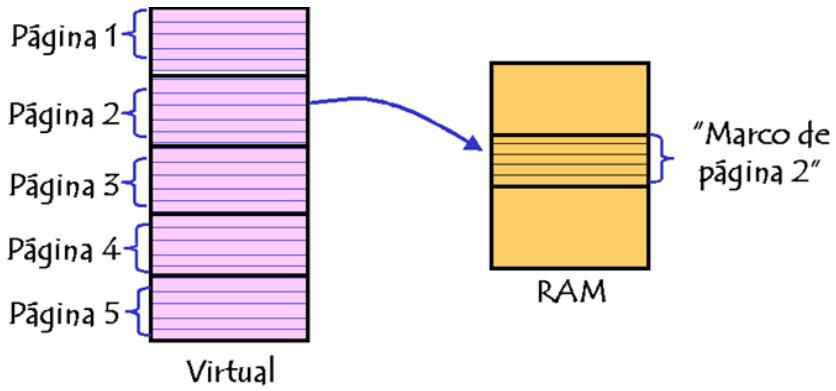
a la descarga de algunos datos o fragmentos de código que residieran en la memoria principal, tenía que encontrarse el espacio necesario en la memoria secundaria. La memoria secundaria también sufre los problemas de fragmentación, pero con la salvedad de que el acceso es mucho más lento, lo que hace que la compactación sea imposible.

Debido a sus ventajas con respecto a los métodos anteriores, la mayoría de los sistemas operativos utilizan comúnmente mecanismos de paginación de diversos tipos.

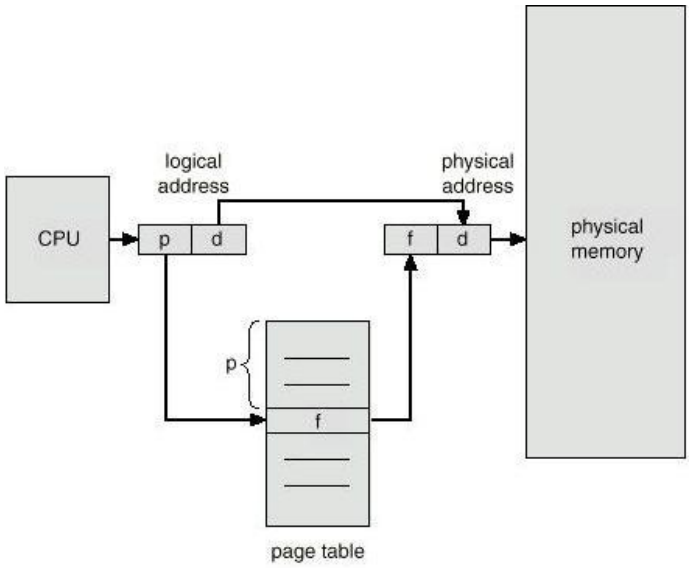
Tradicionalmente, el soporte para la paginación se gestionaba mediante hardware. Sin embargo, algunos diseños implementan los mecanismos de paginación integrando estrechamente el hardware y el sistema operativo.

Método básico.

El método básico para implementar la paginación implica descomponer la memoria física en una serie de bloques de tamaño fijo denominados ***marcos*** y descomponer la memoria lógica en bloques del mismo tamaño denominados ***páginas***.



Cuando hay que ejecutar un proceso, sus páginas se cargan desde la memoria secundaria en los marcos de memoria disponibles. La memoria secundaria está dividida en bloques de tamaño fijo que tienen el mismo tamaño que los marcos de memoria.



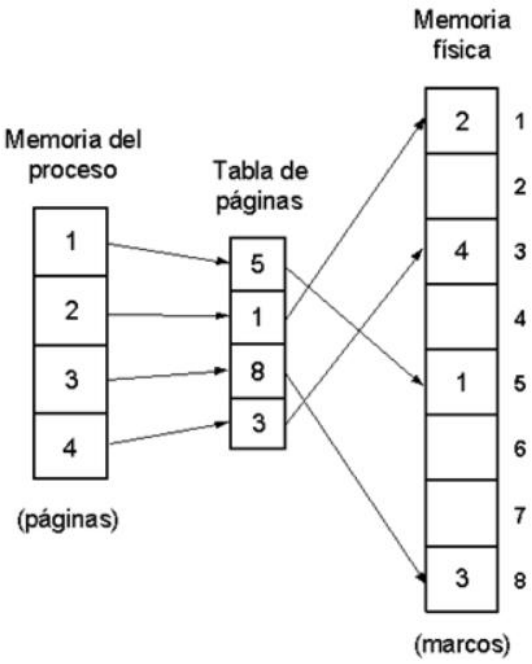
La figura anterior ilustra el soporte hardware para el mecanismo de paginación. Toda dirección generada por la CPU está dividida en dos partes: un número de página (*p*) y un desplazamiento de página (*d*).

El número de página se utiliza como índice para una ***tabla de páginas***. La tabla de páginas contiene la dirección ***base*** de cada página en memoria física; esta dirección base se combina con el ***desplazamiento*** de página para definir la dirección de memoria física que se envía a la unidad de memoria.

Podríamos decir que la paginación es una evolución directa de los registros base y límite utilizados en la ***asignación de memoria contigua***, pero con una diferencia fundamental en la granularidad (tamaño de las partes en las que dividimos la memoria) y la flexibilidad.

Característica	Asignación Contigua	Paginación
Unidad de gestión	El proceso completo (un solo bloque).	Páginas y Marcos (piezas pequeñas).
Granularidad	**Baja:** El "grano" es el proceso entero.	**Alta:** El "grano" es la página (ej. 4 KB).
Direccionamiento	Registro Base y Registro Límite.	Tabla de Páginas (múltiples bases).
Ubicación en RAM	Debe ser un bloque **seguido**.	Puede estar **disperso** por toda la RAM.

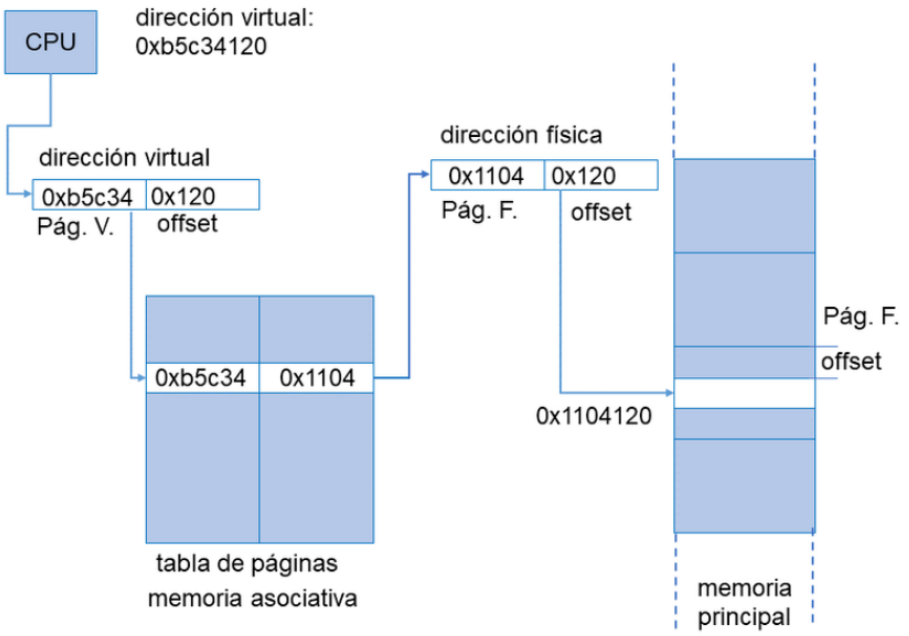
Utilizando la técnica que asignación de memoria contigua el sistema operativo ve al proceso como un solo bloque sólido. En cambio, con la técnica de la paginación el proceso se divide en cientos o miles de piezas pequeñas y uniformes (típicamente de 4 KB).



El tamaño de página (al igual que el tamaño de marco) está definido por las características del hardware (específicamente del diseño del procesador y de su MMU).El tamaño de la página es, normalmente, una potencia de 2, variando entre 512 bytes y 16 MB por página, dependiendo de la arquitectura de la computadora.

Tamaño de Página	Ventaja	Desventaja
Pequeño (ej. 512 bytes)	Reduce la **fragmentación interna** (el desperdicio al final del proceso).	La tabla de páginas se vuelve enorme y consume mucha RAM.
Grande (ej. 4 MB / 16 MB)	La tabla de páginas es pequeña y las transferencias de disco son más rápidas.	Aumenta la fragmentación interna (se desperdicia mucho espacio si el proceso no llena el último marco).

La selección de una potencia de 2 como tamaño de página hace que la traducción de una dirección lógica a un número de página y a un desplazamiento de página resulte particularmente fácil.



Resulta evidente que el propio esquema de paginación es una forma de ***reubicación dinámica***. Cada dirección lógica es asignada por el hardware de paginación a alguna dirección física. La utilización de la paginación es similar al uso de una tabla de registros base (o de reubicación), uno por cada marco de memoria.

Cuando usamos un esquema de paginación, no tenemos ***fragmentación externa***[15]. Todos los marcos libres podrán ser asignador a un proceso que los necesite. Sin embargo, lo que podemos tener es un cierto grado de ***fragmentación interna***[16], ya que lo marcos se asignan como unidad y, si los requisitos de memoria de un proceso no coinciden exactamente con las fronteras de página, el último marco asignado puede no estar completamente lleno.

Por ejemplo, si el tamaño de página es de 2.048 bytes, un proceso de 72.226 bytes necesitará 35 páginas completas más 1.086 bytes. A ese proceso se le asignarían 36 marcos, lo que daría como resultado una fragmentación interna de 2.048 – 1.086 = 962 bytes. En el peor de los casos, un proceso podría necesitar *n* páginas más 1 byte, por lo que se le asignarían n + 1 marcos, dando como resultado una fragmentación interna de tamaño prácticamente igual a un marco completo.

Si el tamaño de los procesos es independiente del tamaño de las páginas, podemos esperar que la fragmentación interna sea, como promedio, igual a media página por cada proceso. Esta consideración sugiere que conviene utilizar tamaños de página pequeños.

Se necesita dedicar recursos a la gestión de cada una de las entradas de la tabla de páginas, y estos recursos adicionales se reducen a medida que se incrementa el tamaño de las páginas (lo que reduce el tamaño de la tabla).

[15] Ocurre cuando hay espacio libre suficiente en total para satisfacer una solicitud, pero ese espacio no está contiguo, sino dividido en pequeños bloques dispersos.
[16] La fragmentación interna ocurre cuando se asigna más memoria de la necesaria a un proceso dentro de un bloque fijo, y el espacio no utilizado queda desperdiciado dentro del bloque asignado.

Asimismo, las operaciones de E/S de disco son más eficientes cuanto mayor sea el número de datos transferidos. La lectura y escritura desde el almacenamiento secundario tiene un costo fijo alto por la latencia de búsqueda y posicionamiento. Transferir bloques de datos más grandes en una sola operación amortiza ese tiempo de acceso, haciendo que las lecturas por página sean significativamente más eficientes.

Factor	Páginas Pequeñas	Páginas Grandes
Fragmentación interna	Menor (poco desperdicio)	Mayor (más desperdicio)
Tabla de páginas	Más grande (gasta RAM)	Más pequeña (ahorra RAM)
Lectura de disco	Más lenta (más viajes)	Más rápida (un solo viaje)

Generalmente, los tamaños de página utilizados en la práctica han ido creciendo con el tiempo, a medida que los procesos, los conjuntos de datos y la memoria principal han aumentado de tamaño.

Algunos procesadores y algunos kernels soportan incluso tamaños de página múltiples. Por ejemplo, Solaris utiliza tamaños de página de 8 KB y 4 MB, dependiendo de los datos almacenados por las páginas.

Usualmente, cada entrada de una tabla de páginas tiene 4 bytes de longitud, pero también ese tamaño puede variar. Una entrada de 32 bits puede apuntar a una de 2% marcos de página físicos.

Si el tamaño del marco es de 4 KB, entonces un sistema con entradas de 4 bytes podrá direccionar 2% bytes (0 16 TB) de memoria física.

Cuando llega un proceso al sistema para ejecutarlo, se examina su tamaño expresado en páginas. Cada página del proceso necesitará un marco. Por lo tanto, si el proceso requiere *n* páginas; deberá haber disponibles al menos *n* marcos en memoria. Si hay disponibles 11 marcos, se los asignará al proceso que acaba de llegar. La primera página del proceso se carga en uno de los marcos asignados y se incluye el número de marco en la tabla de páginas para este proceso. La siguiente página se carga en otro marco y su número de marco se coloca en la tabla de páginas, y así sucesivamente.

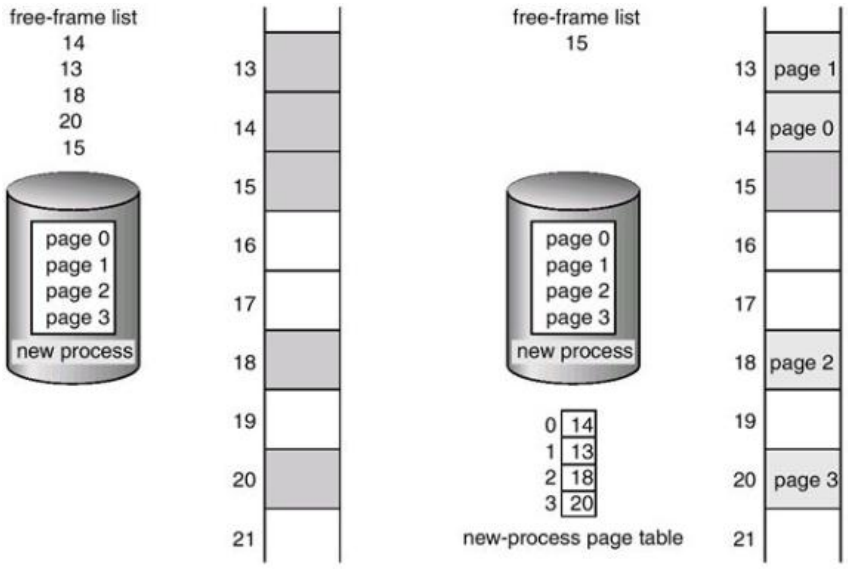
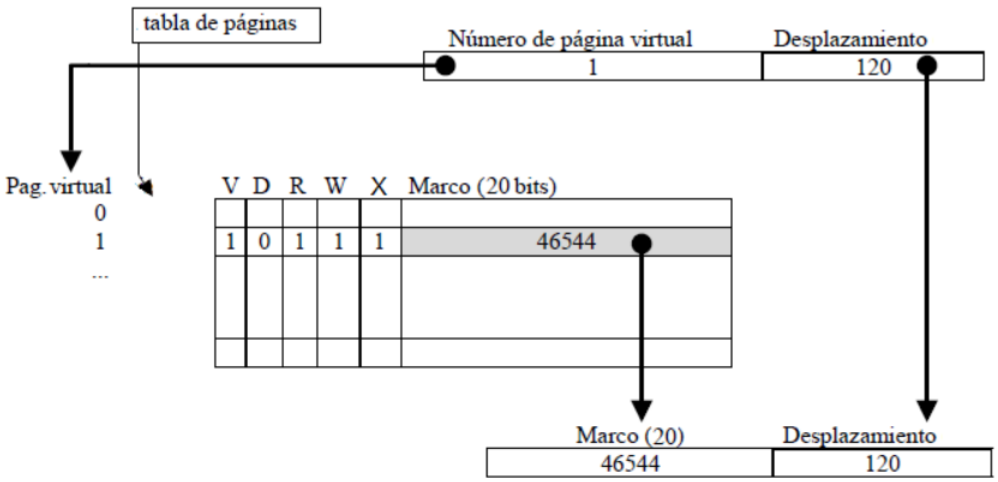


Ilustración 10 - Marcos libres antes de la asignación y después de la asignación.

Un aspecto importante de la paginación es la clara ***separación existente entre la visión de la memoria que tiene el usuario y la memoria física real***. El programa de usuario ve la memoria como un único espacio que sólo contiene ese programa.

En la práctica, el programa de usuario está disperso por toda la memoria física, lo que también sucede para los restantes programas. La diferencia entre la visión de la memoria que tiene el usuario y la memoria física real se resuelve mediante ***el hardware de traducción de direcciones***. Las direcciones lógicas se traducen a direcciones físicas y esta conversión queda oculta a ojos del usuario, siendo controlado por el sistema operativo.

Observemos que los procesos de usuario son incapaces, por definición, de acceder a la memoria que no les pertenece. No tienen forma de direccionar la memoria situada fuera de su tabla de páginas y esa tabla incluye únicamente aquellas páginas que sean propiedad del proceso.



Puesto que el sistema operativo está gestionando la memoria física, debe ser consciente de los detalles relativos a la asignación de la memoria física: qué marcos han sido asignados, qué marcos están disponibles, cuál es el número total de marcos, etc. Esta información se suele mantener en una estructura de datos denominada ***tabla de marcos***.

La tabla de marcos tiene una entrada por cada marco físico que indica si está libre o asignado y, en caso de estar asignado, a qué página de qué proceso o procesos ha sido asignado.

Además, el sistema operativo debe ser consciente de que los procesos de usuario operan en el espacio de usuario y de que todas las direcciones lógicas deben ser convertidas con el fin de generar direcciones físicas. Si un usuario hace una llamada al sistema (por ejemplo, para realizar una operación de E/S) y proporciona una dirección como parámetro (por ejemplo, un búfer), dicha dirección deberá ser convertida para obtener la dirección física correcta. El sistema operativo mantiene una copia de la tabla de páginas de cada proceso, al igual que mantiene una copia del contador de instrucciones y de los contenidos de los registros. Esta copia se utiliza para traducir las direcciones lógicas a direcciones físicas cada vez que el sistema operativo deba convertir manualmente una dirección lógica a una dirección física. El despachador de la CPU utiliza también esa copia para definir la tabla de páginas hardware en el momento de asignar la CPU a un proceso.

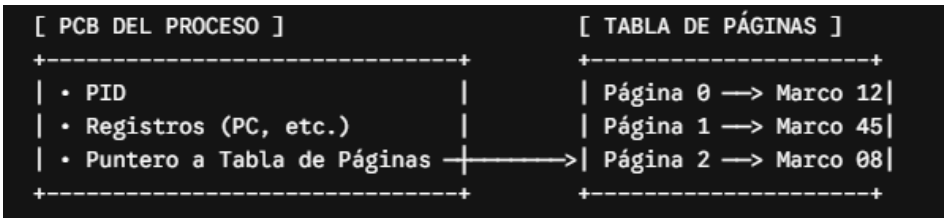
Los mecanismos de paginación incrementan, por tanto, el tiempo necesario para el cambio de contexto.

Soporte de hardware.

Cuando un programa accede a una dirección de memoria, el sistema operativo y el hardware deben traducir esa dirección virtual a una dirección física real en la RAM. Este proceso puede ser lento si se hace cada vez consultando la tabla de páginas.

Cada sistema operativo tiene sus propios métodos para almacenar tablas de páginas. La mayoría de los SO asignan una tabla de páginas para cada proceso, almacenándose un puntero esta tabla de páginas en el bloque de control del proceso.

Cuando el despachador inicia un proceso, vuelve a cargar sus registros en la CPU y configura la MMU con el puntero a la tabla de páginas guardada de ese proceso.

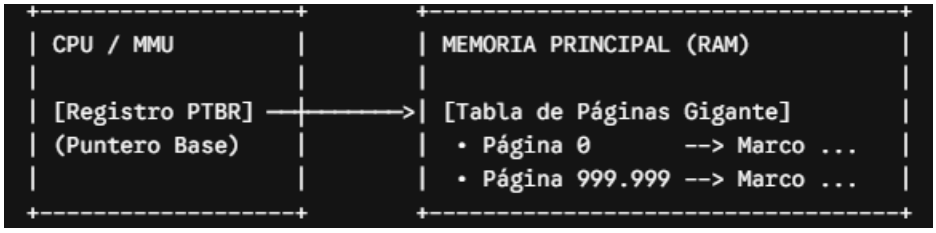


La implementación de hardware de la tabla de páginas puede hacerse de varias maneras. En el caso más simple, la tabla de páginas se implementa como un conjunto de registros dedicados (normalmente ubicados en la MMU). Estos registros deben construirse con lógica de muy alta velocidad, para hacer que el proceso de traducción de direcciones basado en la paginación sea muy eficiente.

Cada acceso a la memoria debe pasar a través del ***mapa de paginación***, por lo que la eficiencia es una de las consideraciones principales de implementación. El despachador de la CPU recarga estos registros al igual que recarga los registros restantes. Las instrucciones para cargar o modificar los registros de la tabla de páginas son, por supuesto, privilegiadas, de modo que sólo el sistema operativo puede cambiar el mapa de memoria.

El uso de registros dedicados para la tabla de páginas resulta satisfactorio si esta tabla es razonablemente pequeña (por ejemplo, 256 entradas). Sin embargo, la mayoría de las computadoras contemporáneas permiten que la tabla de páginas sea muy grande (por ejemplo, un millón de entradas); para estas máquinas, el uso de registros de alta velocidad para incrementar la tabla de páginas no resulta factible.

En lugar de utilizar registros dedicados, la tabla de páginas se mantiene en memoria principal, utilizándose un registro base de la tabla de páginas (***PTBR***, pagetable base register) para apuntar a la tabla de páginas. Es un registro dentro de la MMU o del CPU que guarda la dirección base (inicio) de la tabla de páginas del proceso que se está ejecutando.



Para cambiar las tablas de páginas, sólo hace falta cambiar este único registro, reduciéndose así sustancialmente el tiempo de cambio de contexto.

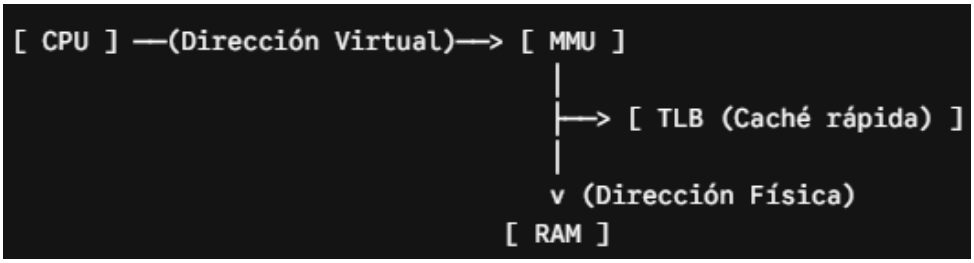
El problema con esta técnica es el tiempo requerido para acceder a una ubicación de memoria del usuario. Si queremos acceder a la ubicación 1, primero tenemos que realizar una indexación en la tabla de páginas, utilizando el valor del registro PTBR, desplazado según el número de página, para obtener el número de marco. Esta tarea requiere un acceso a memoria y nos proporciona el número de marco, que se combina con el desplazamiento de página para generar la dirección real. Sólo entonces podremos acceder a la ubicación deseada de memoria.

Con este esquema, hacen falta dos accesos de memoria para acceder a un byte (uno para obtener la entrada de la tabla de páginas y otro para obtener el byte). Por tanto, el acceso a memoria se ralentiza dividiéndose a la mitad. En la mayor parte de las circunstancias, este retardo es intolerable; para eso, es mejor utilizar un mecanismo de intercambio de memoria.

La solución estándar a este problema consiste en utilizar un caché (hardware) especial, de pequeño tamaño y con capacidad de acceso rápido, denominada ***búfer de consulta de traducción*** (***TLB***, translation lookaside buffer). El búfer o cache TLB es una memoria asociativa de alta velocidad.

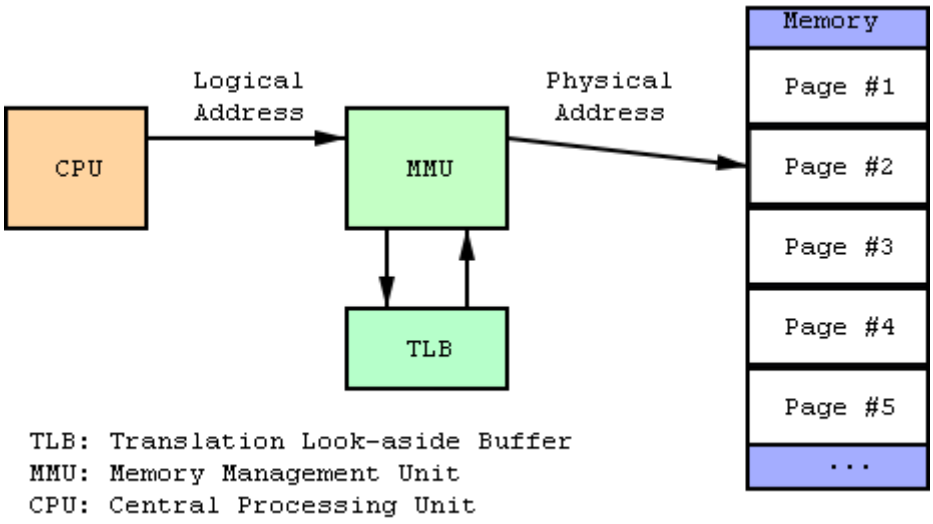
El TLB es una memoria caché especial que almacena traducciones recientes de direcciones virtuales a direcciones físicas en un sistema con paginación. Generalmente está integrado dentro de la CPU, específicamente en la MMU. Tiene una capacidad limitada (sólo guarda un número pequeño de entradas).

Funciona como una memoria intermedia muy rápida o como una caché de la tabla de páginas.



La gran ventaja del TLB es que mejora enormemente el rendimiento del sistema de memoria. Reduce el tiempo de acceso a memoria en sistemas con paginación, ya que evita consultar la tabla de páginas completa cada vez.

En la gran mayoría de las arquitecturas modernas (x86, ARM, RISC-V, etc.),el Translation Lookaside Buffer (TLB) está integrado físicamente dentro de la MMU o muy cerca de ella en el hardware.



Protección.

La protección de memoria en un entorno paginado se consigue mediante una serie de bits de protección asociados con cada marco. Normalmente, estos bits se mantienen en la tabla de páginas.

Uno de los bits puede definir una página como de lectura/escritura o de sólo lectura. Toda referencia a memoria pasa a través de la tabla de páginas con el fin de encontrar el número de marco correcto.

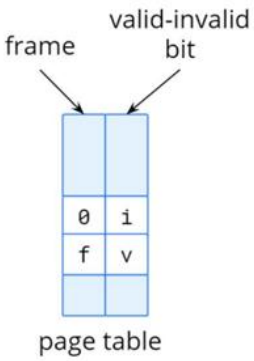
Al mismo tiempo que se calcula la dirección física, pueden comprobarse los bits de protección para verificar que no se esté haciendo ninguna escritura en una página de sólo lectura. Todo intento de escribir una página de sólo lectura provocará una interrupción hardware al sistema operativo (o una violación de protección de memoria).

Podemos ampliar fácilmente este enfoque con el fin de proporcionar un nivel más fino de protección. Podemos diseñar un hardware que proporcione protección de sólo lectura, de lectura/escritura o de sólo ejecución, o podemos permitir cualquier combinación de estos accesos, proporcionando bits de protección separados para cada tipo de acceso. Los intentos ilegales provocarán una interrupción hardware hacia el sistema operativo.

Generalmente, se suele asociar un bit adicional con cada entrada de la tabla de páginas: un bit válido/inválido.

Cuando se configura este bit como “válido”, la página asociada se encontrará en el espacio de direcciones lógicas del proceso y será, por tanto, una página legal (o válida).

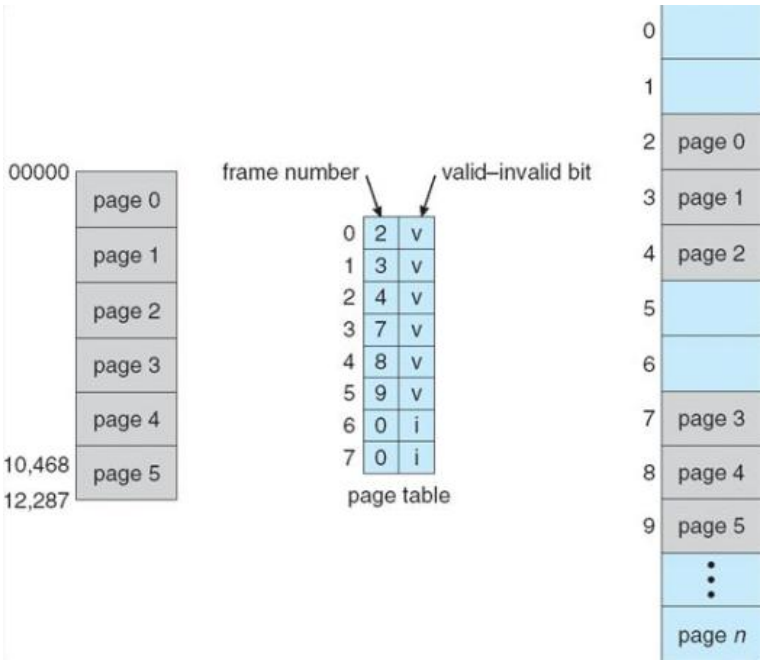
Cuando se configura el bit como “inválido”, la página no se encuentra en el espacio de direcciones lógicas del proceso.



Las direcciones ilegales se capturan utilizando el bit válido/inválido.

El sistema operativo configura este bit para cada página, con el fin de permitir o prohibir el acceso a dicha página.

Supongamos, por ejemplo, que en un sistema con un espacio de direcciones de 14 bits (0 a 16383), tenemos un programa que sólo debe utilizar las direcciones comprendidas entre 0 y 10468. Dado un tamaño de página de 2 KB, tendremos la situación mostrada en siguiente figura.



La dirección de las páginas 0, 1, 2, 3, 4 y 5 se convierten normalmente mediante la tabla de páginas, pero cualquier intento de generar una dirección en las páginas 6 o 7, sin embargo, se encontrará con que el bit válido/inválido está configurado como inválido, por lo que la computadora generará una interrupción hacia el sistema operativo (referencia de página inválida).

Observemos que este esquema genera un problema: puesto que el programa sólo se extiende hasta la dirección 10468, todas las referencias que se hagan más allá de dicha dirección serán ilegales. Sin embargo, las referencias a la página 5 están clasificadas como válidas, por lo que los accesos a las direcciones que van hasta la posición 12287 son válidas. Sólo las direcciones comprendidas entre 12288 y 16383 están marcadas como inválidas.

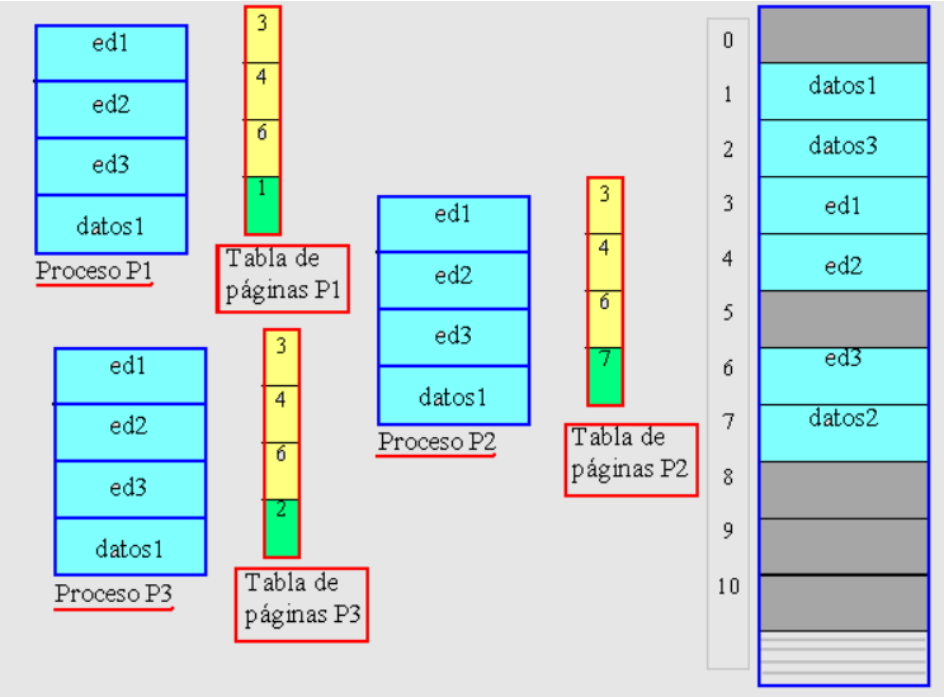
Este problema surge como resultado del tamaño de página de 2 KB y refleja el problema de la fragmentación interna que sufren los mecanismos de paginación.

Los procesos raramente utilizan todo su rango de direcciones. De hecho, muchos procesos sólo emplean una pequeña fracción del espacio de direcciones que tiene disponible. Sería un desperdicio, en estos casos, crear una tabla de páginas con entradas para todas las páginas del rango de direcciones. La mayor parte de esta tabla permanecería sin utilizar, pero ocupando un valioso espacio de memoria. Algunos sistemas proporcionan mecanismos hardware especiales, en la forma de un ***registro de longitud de la tabla de páginas*** (***PTLR***, pagetable length register), para indicar el tamaño de la tabla de páginas. Este valor se compara con cada dirección lógica para verificar que esa dirección se encuentre dentro del rango de direcciones válidas definidas para el proceso.

Si esta comprobación fallara, se produciría una interrupción de error dirigida al sistema operativo.

Páginas compartidas.

Una ventaja de la paginación es la posibilidad de compartir código común. Esta consideración es particularmente importante en un entorno de tiempo compartido. Consideremos un sistema que de soporte a 40 usuarios, cada uno de los cuales esté ejecutando un editor de texto. Si el editor de texto está compuesto por 150 KB de código y 50 KB de espacio de datos, necesitaremos 8000 KB para dar soporte a los 40 usuarios. Sin embargo, si se trata de código reentrante[17] (o código puro), podrá ser compartido, como se muestra en la siguiente figura.



Podemos ver un editor de tres páginas (cada página tiene 50 KB de tamaño, utilizándose un tamaño de página tan grande para simplificar la figura) compartido entre los tres procesos. Cada proceso tiene su propia página de datos.

El código reentrante es código que no se automodifica, nunca cambia durante la ejecución. De esta forma, dos o más procesos pueden ejecutar el mismo código al mismo tiempo. Cada proceso tendrá su propia copia de los registros y del almacenamiento de datos, para albergar los datos requeridos para la ejecución del proceso. Por supuesto, los datos correspondientes a dos procesos distintos serán diferentes.

Sólo es necesario mantener en la memoria física una copia del editor. La tabla de páginas de cada usuario se corresponderá con la misma copia física del editor, pero las páginas de datos se harán corresponder con marcos diferentes. Así, para dar soporte a 40 usuarios, sólo necesitaremos una copia del editor (150 KB), más 40 copias del espacio de datos de 50 KB de

[17] El código reentrante, también conocido como código puro, es código que puede ser interrumpido y reanudado sin afectar su integridad o la de otros procesos que también puedan estar usando ese mismo código. Esto significa que múltiples procesos pueden ejecutar la misma función simultáneamente sin problemas de sincronización o datos corruptos.

cada usuario. El espacio total requerido será ahora de 2.150 KB, en lugar de 8.000 KB, lo que representa un ahorro considerable.

También pueden compartirse otros programas que se utilicen a menudo, como, por ejemplo, compiladores, sistemas de ventanas, bibliotecas de tiempo de ejecución, sistemas de base de datos, etc. Para poder compartirlo, el código debe ser reentrante. El carácter de sólo lectura del código compartido no debe dejarse a expensas de la corrección de ese código, sino que el propio sistema operativo debe imponer esta propiedad.

Segmentación.

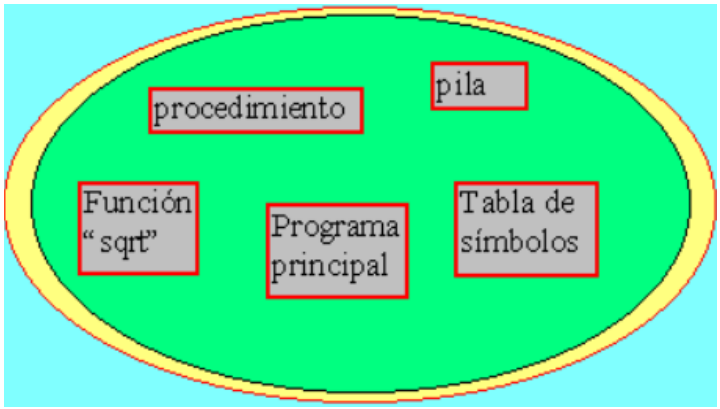
Un aspecto importante de la gestión de memoria que se volvió inevitable con los mecanismos de paginación es la separación existente entre la vista que el usuario tiene de la memoria y la memoria física real. Como ya comentamos, la vista que el usuario tiene de la memoria no es la misma que la memoria física real, sino que esa vista del usuario se mapea sobre la memoria física.

Este mapeo permite la diferenciación entre memoria lógica y memoria física.

Método básico.

Los usuario y programadores prefieren ver la memoria como una colección de segmentos de tamaño variable, sin que exista necesariamente ninguna ordenación entre dichos segmentos.

Revisemos la forma en que pensamos en un programa en el momento de escribirlo. Tendemos a considerarlo como un programa principal con un conjunto de métodos, procedimientos o funciones y que también puede incluir diversas estructuras de datos (objetos, matrices, pilas, variables, etc.).



Para referirnos a cada uno de estos módulos o elementos de datos, utilizamos su nombre. Hablamos acerca de "la pila", "la biblioteca matemática", "el programa principal", etc., sin preocuparnos de las direcciones de memoria que estos elementos puedan apuntar.

Segmento (Nombre)	¿Qué guarda?
Segmento de Código	Las instrucciones de qué debe hacer el programa.
Segmento de Datos	Las variables globales que usa el programa.
Segmento de Pila (Stack)	Las notas temporales de las funciones que está haciendo.

No nos importa si la pila está almacenada antes o después de la función *sgrt ().* Cada uno de estos segmentos tiene una longitud variable, que está definida intrínsecamente por el segmento que cumpla ese segmento del programa. Los elementos dentro de un segmento están identificados por su desplazamiento con respecto al inicio del segmento: la primera instrucción del programa, la séptima entrada de la pila, la quinta instrucción de la función sgrt (), etc.

La ***segmentación*** es un esquema de gestión de memoria que soporta esta visión de la memoria que tienen los usuarios. El espacio lógico de direcciones es una colección de segmentos y cada segmento tiene un nombre y una longitud. Las direcciones especifican tanto el nombre del segmento como el desplazamiento dentro de ese segmento. El usuario especifica, por lo tanto, cada dirección proporcionando dos valores: un nombre de segmento y un desplazamiento.

Atributo	¿Qué es?	Función Principal
Nombre (Número)	El **ID** del segmento.	Identifica qué bloque usar (ej: Código o Pila).
Longitud (Límite)	El **tamaño** del segmento.	Indica dónde termina el bloque para dar seguridad.

Comparemos este esquema con el de paginación, en el que el usuario especificaba una única dirección, que hardware particionaba en un número de páginas y en un desplazamiento, de forma invisible para el programador.

Con la segmentación se divide el espacio lógico del programa en segmentos de tamaño variable, cada uno con un significado lógico completo.

A diferencia de la ***paginación***, que generalmente usa bloques de tamaño fijo, la segmentación divide la memoria en bloques lógicos coherentes según la función que cumplen.

Por simplicidad de implementación, los segmentos están numerados y se hace referencia ellos mediante un número de segmento, en lugar de utilizar un nombre de segmento. Así, una dirección lógica estará compuesta por una pareja del tipo:

<número-segmento, desplazamiento>

Normalmente, el programa del usuario se compila y el compilador construye automáticamente los segmentos para reflejar el programa de entrada.

Un compilador C, podría crear segmentos separados para los siguientes elementos:

1. El código.
2. Las variables globales.
3. El cúmulo de memoria a partir del cual se asigna la memoria.
4. Las pilas utilizadas por cada hilo de ejecución.
5. La biblioteca C estándar.

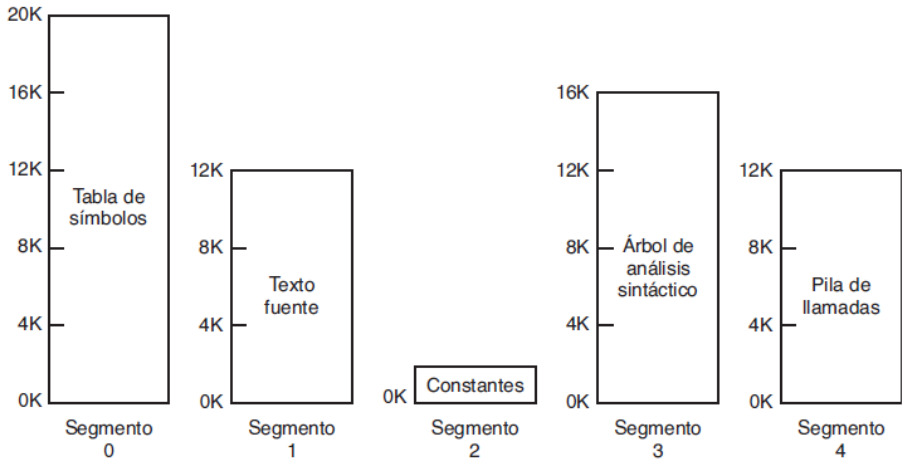


Ilustración 11 - Una memoria segmentada permite que cada tabla crezca o se reduzca de manera independiente a las otras tablas.

Soporte de hardware.

Aunque el usuario puede ahora hacer referencia a los objetos del programa utilizando una dirección bidimensional, la memoria física real continúa siendo, por supuesto, una secuencia unidimensional de bytes. Por lo tanto, deberemos definir una implementación para mapear las direcciones bidimensionales definidas por el usuario sobre las direcciones físicas unidimensionales. Este mapeo se lleva a cabo mediante una ***tabla de segmentos***. Cada entrada de la tabla de segmentos tiene una dirección base del segmento y un límite del segmento. La dirección base del segmento contiene la dirección física inicial del lugar donde el segmento reside dentro de la memoria, mientras que el límite del segmento especifica la longitud de éste.

Tabla de segmentos

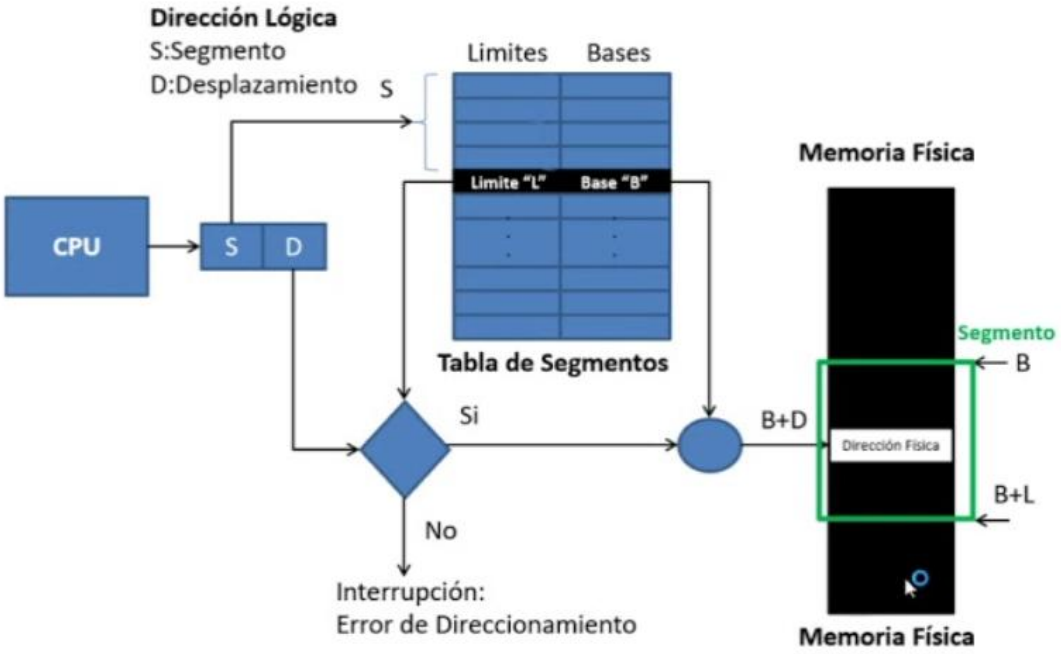
	Límite (longitud del segmento)	Base (dirección inicial del segmento)
0	1000	1400
1	400	6300
2	400	4300
3	1100	3200
4	1000	4700

Una dirección lógica estará compuesta de dos partes: un número de segmento, *s*, y un desplazamiento dentro de ese segmento, *d*.

El número de segmento se utiliza como índice para la tabla de segmentos. El desplazamiento *d* de la dirección lógica debe estar comprendido entre 0 y el límite del segmento; si no lo está, se producirá una interrupción hacia el sistema operativo (intento de direccionamiento lógico más allá del final del segmento).

```
MMU busca:
   Base del segmento "Datos" = 2000
   Desplazamiento = 50
                          ↓
Dirección física = 2000 + 50 = 2050
```

Cuando un desplazamiento es legal, se lo suma a la dirección base del segmento para generar la dirección de memoria física del byte deseado. La tabla de segmentos es, por lo tanto, es esencialmente una matriz de parejas de registros base-límite.



Ejemplo: Intel Pentium.

Tanto la paginación como la segmentación tienen ventajas y desventajas. De hecho, algunas arquitecturas proporcionan ambos mecanismos.

Analicemos la arquitectura Intel Pentium[18], que soporta tanto una segmentación pura como una segmentación con paginación.

En los sistemas Pentium, la CPU genera direcciones lógicas que se entregan a ***la unidad de segmentación***. Ésta produce una dirección lineal para cada dirección lógica y esa dirección lineal se entrega a continuación a la ***unidad de paginación***, que a su vez genera la dirección física de la memoria principal. Así, las unidades de segmentación y de paginación forman el equivalente de una unidad de gestión de memoria (MMU, memory management unit).

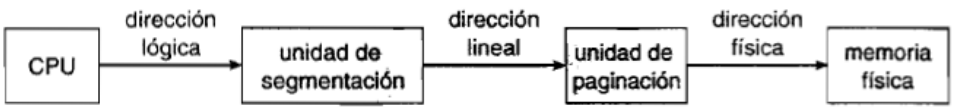


Ilustración 12 - Traducción de direcciones lógicas a direcciones físicas en el Pentium.

[18] La arquitectura Pentium se refiere a una serie de microprocesadores de Intel basados en la arquitectura x86. El Pentium original, lanzado en 1993, fue la quinta generación de esta arquitectura y la sucesora del chip 486.

Mecanismo de segmentación en el Pentium.

La arquitectura Pentium permite que un segmento tenga un tamaño de hasta 4 GB y el número máximo de segmentos por cada proceso es de 16 KB. El espacio lógico de direcciones en un proceso está divido en dos particiones.

La primera partición está compuesta de hasta 8 KB segmentos que son privados de ese proceso; la segunda partición está compuesta de hasta 8 KB segmentos, compartidos entre todos los procesos.

La información acerca de la primera partición se almacena en la tabla local de descriptores (LDT, local descriptor table); la información acerca de la segunda partición se almacena en la tabla global de descriptores (GDT, global descript table). Cada entrada de la LDT y de la GDT está compuesta por un descriptor de segmento de 8 bytes que incluye información detallada acerca de ese segmento concreto, incluyendo la posición base y el límite del segmento.

La dirección lógica es una pareja (selector, desplazamiento), donde el selector es un número de 16 bits, en el que *s* designa el número de segmento, *g* indica si el segmento está en la GDT o en la LDT y *p* son dos bits de protección. El desplazamiento es un número de 32 bits que especifica la ubicación del byte (o palabra) dentro del segmento en cuestión.

La máquina tiene seis registros de segmento, lo que permite que un proceso pueda direccionar en cualquier momento concreto hasta seis segmentos distintos. También tiene seis registros de microprograma[19] de 8 bytes para almacenar los correspondientes descriptores de la LDT o de la GDT.

Esta caché permite al Pentium tener que leer el descriptor desde la memoria para cada referencia de memoria.

La dirección lineal en el Pentium tiene 32 bits de longitud y está formada de la manera siguiente: el registro de segmento apunta a la entrada apropiada de la LDT o de la GDT; la información de base y de límite acerca del segmento en cuestión se utiliza para generar una dirección lineal.

En primer lugar, se utiliza el límite para comprobar la validez de la dirección. Si la dirección no es válida, se genera un fallo de memoria, lo que provoca una interrupción dirigida al sistema operativo. Si es válida, entonces se suma el valor del desplazamiento al valor de la base, lo que da como resultado una dirección lineal de 32 bits.

[19] Los registros de microprograma (también llamados registros de control) son pequeños almacenes de datos de alta velocidad dentro de la CPU que contienen las microinstrucciones que controlan el funcionamiento interno de la CPU.

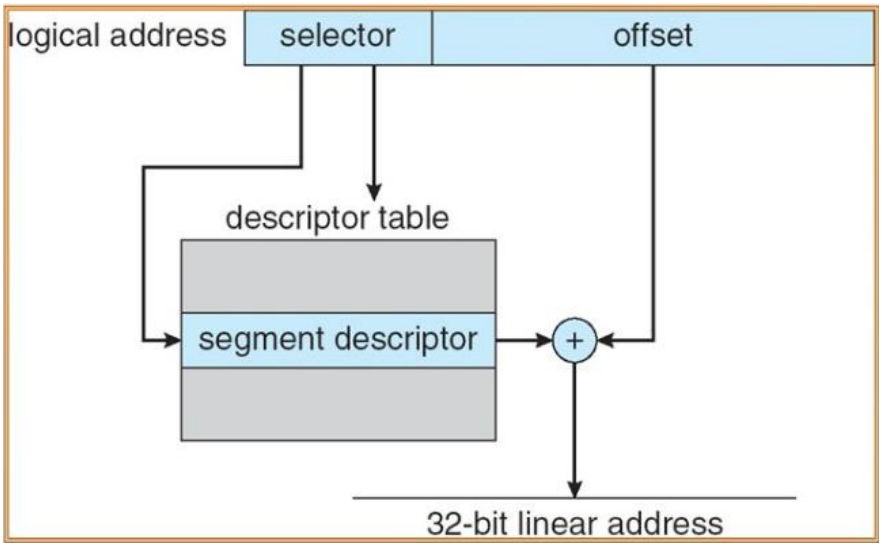


Ilustración 13 - Segmentación en el Intel Pentium.

Mecanismo de paginación en el Pentium.

La arquitectura Pentium permite utilizar un tamaño de página de 4 KB o de 4 MB. Para las páginas de 4 KB, el Pentium utiliza un esquema de paginación en dos niveles.

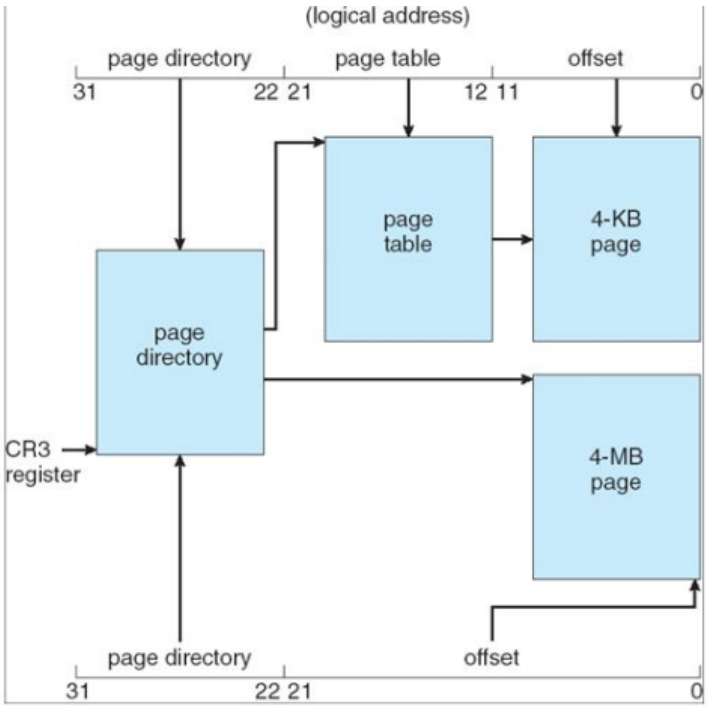


Ilustración 14 - Mecanismo de paginación en la arquitectura Pentium.

En la figura anterior se muestra el mecanismo de traducción de direcciones del Intel Pentium con mayor detalle. Los diez bits de mayor peso[20] hacen referencia a una entrada en la tabla de páginas externa, que en el Pentium se denomina ***directorio de páginas*** (el registro CR3 apunta al directorio de páginas para el proceso actual).

La entrada en el directorio de páginas apunta a una tabla de paginación interna que está indexada según el contenido de los diez bits más internos de la dirección lineal. Finalmente, los bits 0-11 de menor peso[21] indican el desplazamiento dentro de la página de 4 KB a la que se apunta desde la tabla de páginas.

Una de las entradas del directorio de páginas es el indicador *Page Size* que (si está activado) indica que el tamaño del marco de página es de 4 MB y no de 4 KB, que es el valor normal. Si este indicador está activado, el directorio de páginas apunta directamente al marco de página de 4 MB, puenteando la tabla de páginas interna, en cuyo caso los 22 bits de menor peso de la dirección lineal constituyen el desplazamiento dentro del marco de página de 4 MB.

Para mejorar la eficiencia de utilización de la memoria física, las tablas de páginas del Intel Pentium pueden cargarse y descargarse de disco. En este caso, se utiliza un bit válido/inválido dentro de la entrada del directorio de páginas para indicar si la tabla a la que apunta la entrada se encuentra en memoria o en disco. Si la tabla se encuentra en el disco, el sistema operativo puede usar los otros 31 bits para especificar la ubicación de la tabla dentro del disco; como consecuencia, la tabla puede cargarse en memoria cuando sea necesario.

Referencias y bibliografía de la sección.

- Tanenbaum 3-2 / 3-7.
- Silberschatz 8-4 / 8-6.

Tablas de páginas y segmentos.

Tablas de páginas.

Tanto las particiones de tamaño fijo como variable son ineficientes en el uso de la memoria; las primeras provocan ***fragmentación interna***, las últimas ***fragmentaciones externa***.

Supóngase, sin embargo, que la memoria principal se divide en porciones de tamaño fijo relativamente pequeños, y que cada proceso también se divide en porciones pequeñas del

[20] En computación, los bits de mayor peso (Most Significant Bit, MSB) son los bits que representan el valor más alto en una representación binaria de un número. Son los bits más a la izquierda en la representación. En el número binario 10110011, el bit de mayor peso es el 1 en la posición 7 (2^7), mientras que el bit menos significativo es el 1 en la posición 0 (2^0).

[21] El "bit de menor peso" (Least Significant Bit o LSB) es el bit que tiene el menor valor posicional en un número binario. En otras palabras, es el bit que está más a la derecha en la representación binaria de un número. En el número binario 1011, el LSB es el '1' en la posición más a la derecha.

mismo tamaño fijo. Como ya vimos, a dichas porciones del proceso, conocidas como páginas, se les asigna porciones disponibles de memoria, conocidas como marcos, o marcos de páginas.

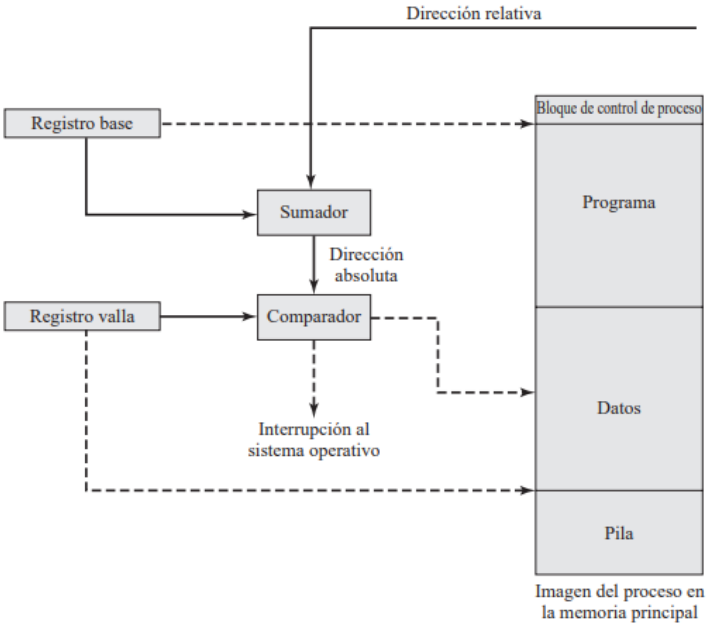


Ilustración 15 - Soporte hardware para la reubicación.

La Figura siguiente ilustra el uso de páginas y marcos.

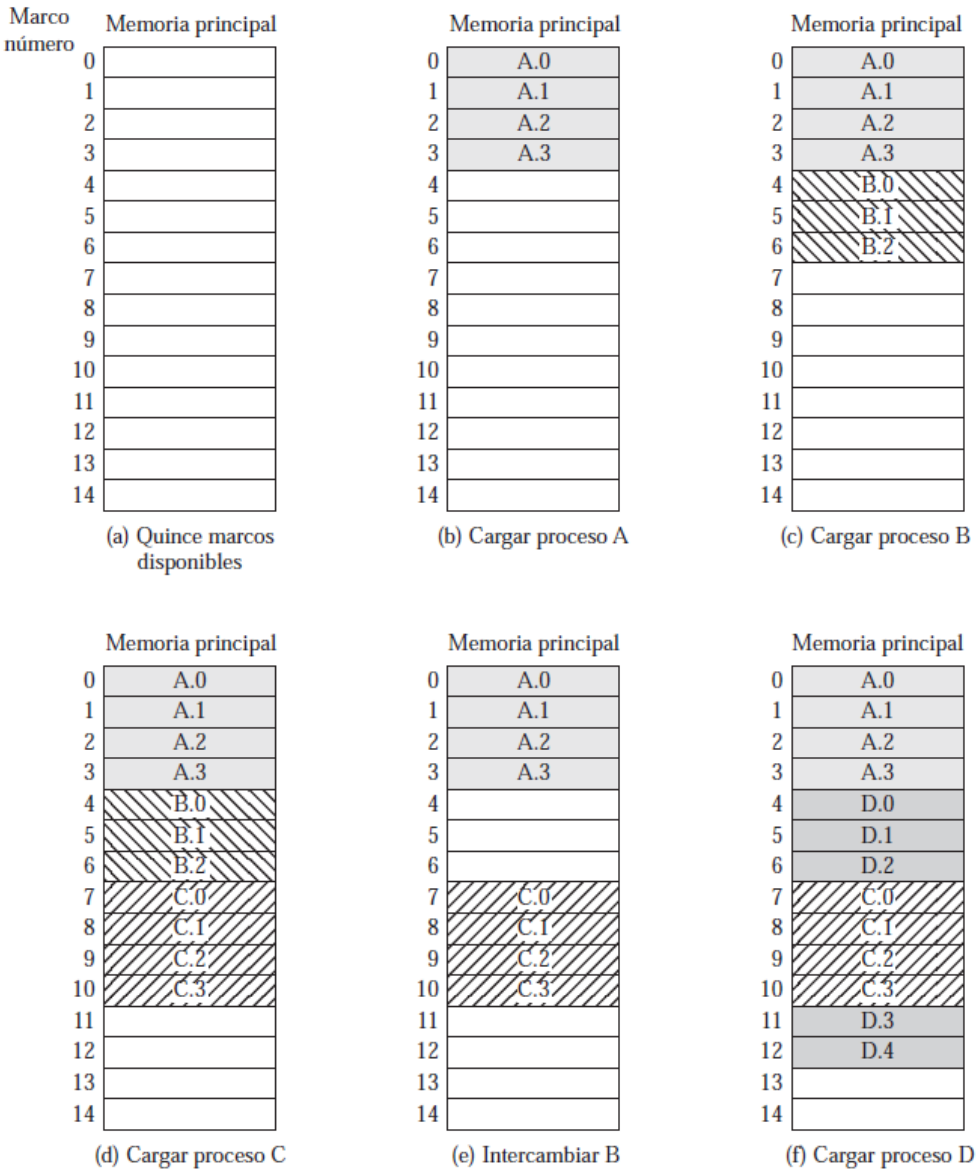


Ilustración 16 - Asignación de páginas de proceso a marcos libres.

En un momento dado, algunos de los marcos de la memoria están en uso y algunos están libres. El sistema operativo mantiene una lista de marcos libres.

El proceso A, almacenado en disco, está formado por cuatro páginas. En el momento de cargar este proceso, el sistema operativo encuentra cuatro marcos libres y carga las cuatro páginas del proceso A en dichos marcos (parte b de la figura).

El proceso B, formado por tres páginas, y el proceso C, formado por cuatro páginas, se cargan a continuación. En ese momento, el proceso B se suspende y deja la memoria principal.

Posteriormente, todos los procesos de la memoria principal se bloquean, y el sistema operativo necesita traer un nuevo proceso, el proceso D, que está formado por cinco páginas.

Ahora supongamos, como en este ejemplo, que no hay suficientes marcos contiguos sin utilizar para ubicar un proceso. En necesario considera si esto evitaría que el sistema operativo cargara el proceso D. La respuesta es no, porque una vez más se puede utilizar el concepto de dirección lógica.

Un registro base sencillo de direcciones no basta en esta ocasión. En su lugar, el sistema operativo mantiene una ***tabla de páginas*** por cada proceso. La tabla de páginas muestra la ubicación del marco por cada página del proceso.

Dentro del programa, cada dirección lógica está formada por un número de página y un desplazamiento dentro de la página. Es importante recordar que, en el caso de una partición simple, una dirección lógica es la ubicación de una palabra relativa al comienzo del programa; el procesador la traduce en una dirección física.

Con paginación, la traducción de direcciones lógicas a físicas las realiza también el hardware del procesador. Ahora el procesador debe conocer cómo acceder a la tabla de páginas del proceso actual.

Presentado como una dirección lógica (número de página, desplazamiento), el procesador utiliza la tabla de páginas para producir una dirección física (número de marco, desplazamiento).

Continuando con nuestro ejemplo, las cinco páginas del proceso D se cargan en los marcos 4, 5, 6, 11 y 12. La Figura siguiente muestra las diferentes tablas de páginas en este momento.

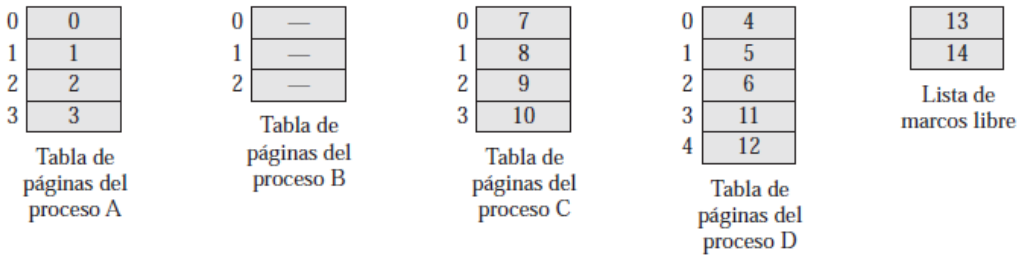


Ilustración 17 - Estructuras de datos para el ejemplo de la Figura 7.9 en el instante (f).

Una tabla de páginas contiene una entrada por cada página del proceso, de forma que la tabla se indexe fácilmente por el número de página (iniciando en la página 0). Cada entrada de la tabla de páginas contiene el número del marco en la memoria principal, si existe, que contiene la página correspondiente.

Adicionalmente, el sistema operativo mantiene una única lista de marcos libres de todos los marcos de la memoria que se encuentran actualmente no ocupados y disponibles para las páginas.

Tablas de segmentos.

Un programa de usuario se puede subdividir utilizando segmentación, con lo cual el programa y sus datos asociados se dividen en un número de segmentos.

No se requiere que todos los programas sean de la misma longitud, aunque hay una longitud máxima de segmento.

Como en el caso de la paginación, una dirección lógica utilizando segmentación está compuesta por dos partes, en este caso un número de segmento y un desplazamiento. Debido al uso de segmentos de distinto tamaño, la segmentación es similar al particionamiento dinámico. En la ausencia de un esquema de overlays[22] o el uso de la memoria virtual, se necesitaría que todos los segmentos de un programa se cargaran en la memoria para su ejecución.

La diferencia, comparada con el particionamiento dinámico, es que con la segmentación un programa podría ocupar más de una partición, y estas particiones no necesitan ser contiguas. La segmentación elimina la fragmentación interna, pero, al igual que el particionamiento dinámico, sufre de fragmentación externa. Sin embargo, debido a que el proceso se divide en varias piezas más pequeñas, la fragmentación externa debería ser menor.

Mientras que la paginación es invisible al programador, la segmentación es normalmente visible y se proporciona como una utilidad para organizar programas y datos. Normalmente, el programador o compilador asignará programas y datos a diferentes segmentos. Para los propósitos de la programación modular, los programas o datos se pueden dividir posteriormente en múltiples segmentos.

El inconveniente principal de este servicio es que el programador debe ser consciente de la limitación de tamaño de segmento máximo.

Otra consecuencia de utilizar segmentos de distinto tamaño es que no hay una relación simple entre direcciones lógicas y direcciones físicas.

De forma análoga a la paginación, un esquema de segmentación sencillo haría uso de una ***tabla de segmentos*** por cada proceso y una lista de bloques libre de memoria principal. Cada entrada de la tabla de segmentos tendría que proporcionar la dirección inicial de la memoria principal del correspondiente segmento. La entrada también debería proporcionar la longitud del segmento, para asegurar que no se utilizan direcciones no válidas. Cuando un proceso entra en el estado Ejecutando, la dirección de su tabla de segmentos se carga en un registro especial utilizado por el hardware de gestión de la memoria.

Consideremos una dirección de *n+m* bits, donde los *n* bits de la izquierda corresponden al número de segmento y los *m* bits de la derecha corresponden al desplazamiento. En el ejemplo de la figura siguiente (parte c), $n = 4$ y $m = 12$.

[22] Overlay es una técnica para ejecutar programas más grandes que la memoria física disponible.
En vez de cargar todo el programa a la RAM al mismo tiempo, se divide en partes (módulos) y se cargan solo las secciones que se necesitan en ese momento, superponiéndolas (overlay) en la misma región de memoria.

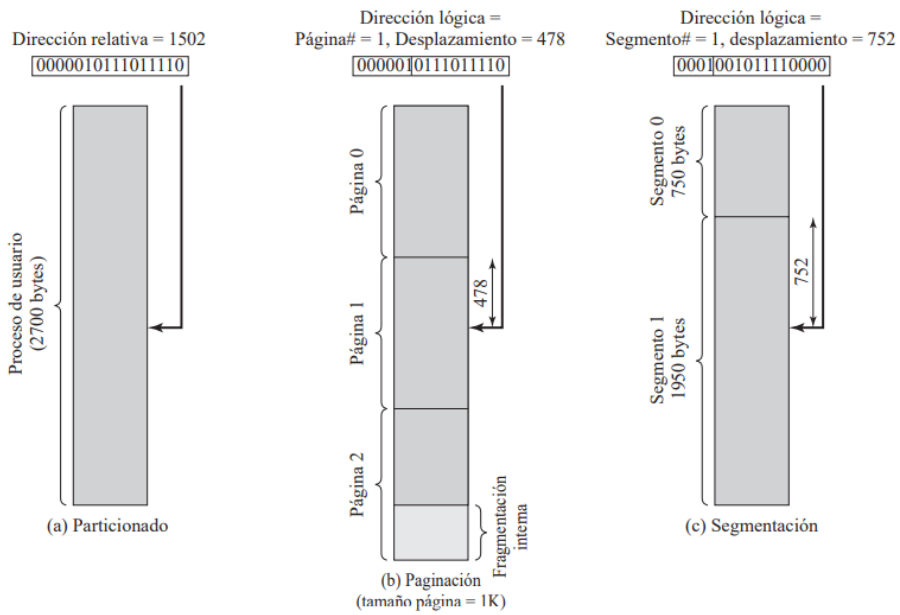


Ilustración 18 - Direcciones lógicas.

Por tanto, el tamaño de segmento máximo es 212 = 4096. Se necesita llevar a cabo los siguientes pasos para la traducción de direcciones:

- Extraer el número de segmento como los *n* bits de la izquierda de la dirección lógica.
- Utilizar el número de segmento como un índice a la tabla de segmentos del proceso para encontrar la dirección física inicial del segmento.
- Comparar el desplazamiento, expresado como los *m* bits de la derecha, y la longitud del segmento. Si el desplazamiento es mayor o igual que la longitud, la dirección no es válida.
- La dirección física deseada es la suma de la dirección física inicial y el desplazamiento.

En el ejemplo, se parte de la dirección lógica 0001001011110000, que corresponde al segmento número 1, desplazamiento 752. Supongamos que este segmento reside en memoria principal, comenzando en la dirección física inicial 0010000000100000. Por tanto, la dirección física es 0010000000100000 + 001011110000 = 0010001100010000 (parte b de la siguiente figura).

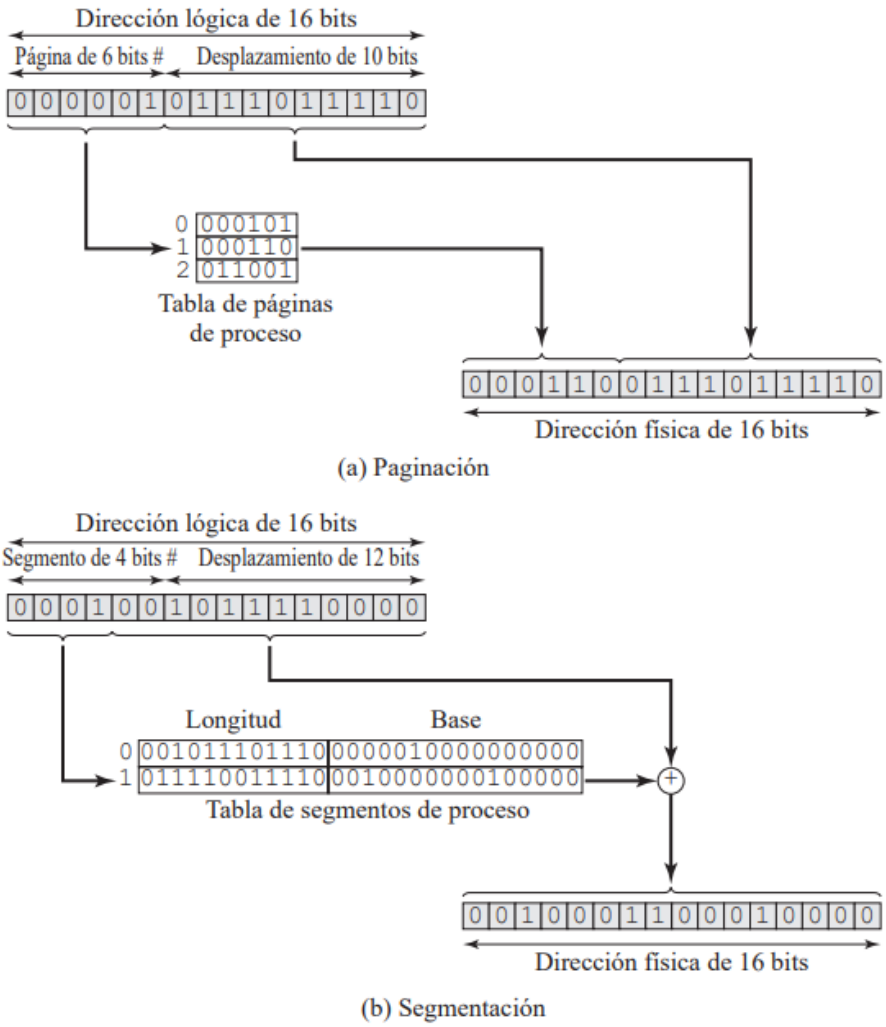


Ilustración 19 - Ejemplos de traducción de direcciones lógicas a físicas.

Resumiendo, con la segmentación simple, un proceso se divide en un conjunto de segmentos que no tienen que ser del mismo tamaño. Cuando un proceso se trae a memoria, todos sus segmentos se cargan en regiones de memoria disponibles, y se crea ***la tabla de segmentos***.

Referencias y bibliografía de la sección.

- Stallings pág. 321.

Estructura de las tablas de páginas.

Vamos a explorar algunas de las técnicas más comunes para estructurar la tabla de páginas.

Paginación jerárquica.

La mayoría de los sistemas informáticos modernos soportan un gran espacio de direcciones lógicas. En este tipo de entorno, la propia tabla de páginas llega a ser excesivamente grande. Por ejemplo, consideremos un sistema con un espacio lógico de direcciones de 32 bits. Si el tamaño de página en dicho sistema es de 4 KB, entonces la tabla de páginas puede estar compuesta de hasta un millón de entradas.

Suponiendo que cada entrada esté compuesta por 4 bytes, cada proceso puede necesitar hasta 4 MB de espacio físico de direcciones sólo para la tabla de páginas. Obviamente, no conviene asignar la tabla de páginas de forma contigua en memoria principal. Una solución simple a este problema consiste en dividir la tabla de páginas en fragmentos más pequeños y podemos llevar a cabo esta división de varias formas distintas.

Una de esas formas consiste en utilizar un ***algoritmo de paginación de dos niveles***, en el que la propia tabla de páginas está también paginada.

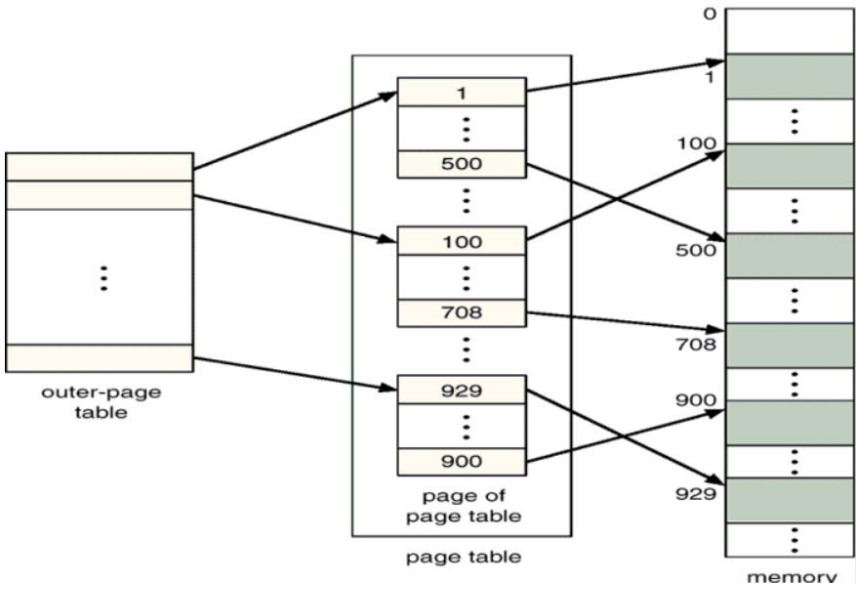
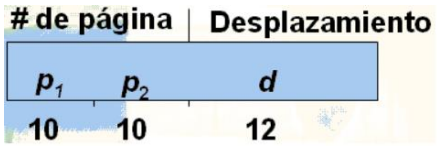


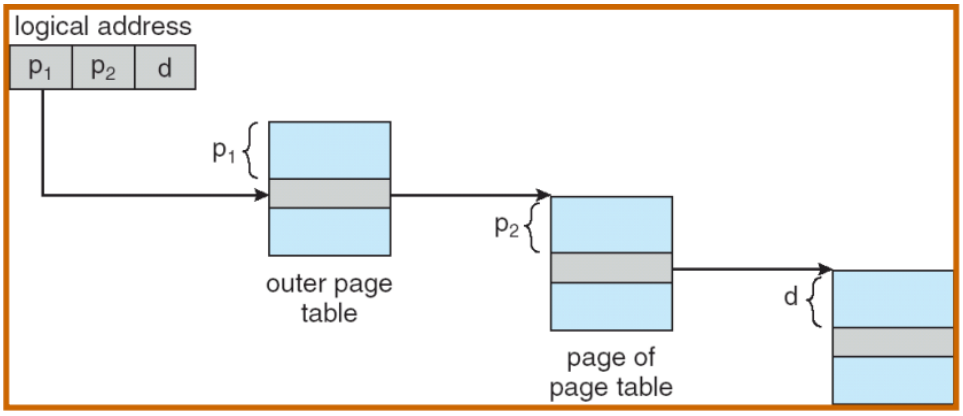
Ilustración 20 - Un esquema de tabla de páginas en dos niveles.

En nuestro ejemplo de una máquina de 32 bits con un tamaño de página de 4 KB; una dirección lógica estará dividida en un número de página de 20 bits y un desplazamiento de página de 12 bits.

Puesto que la tabla de páginas está paginada, el número de páginas se divide a su vez en un número de página de 10 bits y un desplazamiento de página de 10 bits. De este modo, una dirección lógica tendría la estructura siguiente:



Donde p1 es un índice a la tabla de páginas externa y p2 es el desplazamiento dentro de la página de la tabla de páginas externa. El método de traducción de direcciones para esta arquitectura se muestra en la siguiente figura.



Puesto que la traducción de las direcciones se realiza comenzando por la tabla de páginas externas y continuando luego por la interna, este esquema también se conoce con el nombre de tabla de ***páginas con correspondencia (mapeo) directa***.

Tablas de páginas hash.

Una técnica común para gestionar los espacios de direcciones superiores a 32 bits consiste en utilizar una tabla hash de páginas, donde el valor ***hash***[23] es el número de página virtual. Cada entrada de la tabla hash contiene una lista enlazada de elementos que tienen como valor hash una misma ubicación (con el fin de tratar las colisiones).

Cada elemento está compuesto de tres campos: (1) el número de página virtual, (2) el valor del marco de página mapeado y (3) un punto del siguiente elemento de la lista enlazada.

El algoritmo funciona de la forma siguiente: al número de página virtual de la dirección virtual se le aplica una función hash para obtener un valor que se utiliza como índice para la tabla hash. El número de página virtual se compara con el campo 1 del primer elemento de la lista

[23] Un "hash" (o función hash) es un algoritmo que convierte una entrada de datos de cualquier tamaño en una cadena de caracteres de longitud fija.

enlazada. Si hay una correspondencia, se utiliza el marco de página correspondiente (campo 2) para formar la dirección física deseada. Si no se produce una correspondencia, se exploran las subsiguientes entradas de la lista enlazada hasta encontrar el correspondiente número de página virtual.

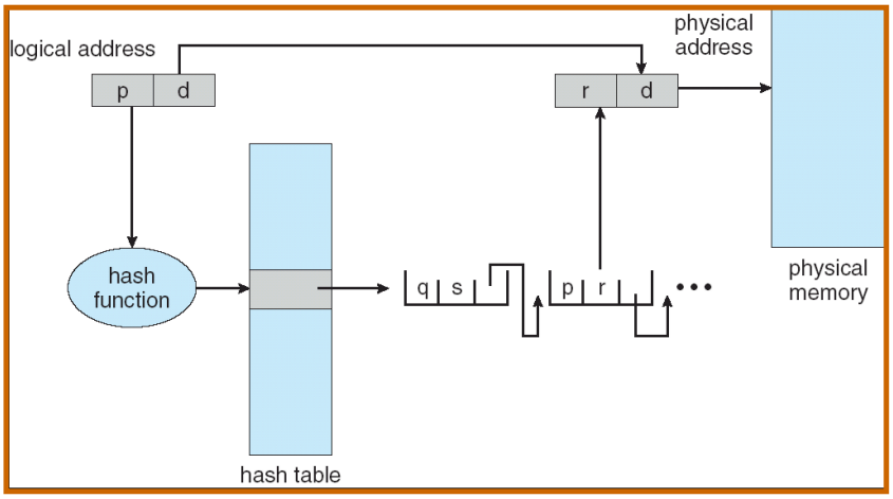


Ilustración 21 - Tabla de páginas hash.

También se ha propuesto una variante de este esquema que resulta más adecuada para los espacios de direcciones de 64 bits. Esta variante utiliza tablas de páginas en ***clúster***[24], que son similares a las tablas de páginas hash, salvo porque cada entrada de la tabla hash apunta a varias páginas (como por ejemplo 16) en lugar de a una sola página. De este modo, una única entrada de la tabla de páginas puede almacenar las correspondencias (mapeos) para múltiples marcos de página física. Las tablas de páginas en cluster son particularmente útiles para espacios de direcciones dispersos en los que las referencias a memoria son no continuas y están dispersas por todo el espacio de direcciones.

Tablas de páginas invertidas.

Usualmente, cada proceso tiene una tabla de páginas asociada. La tabla de páginas incluye una entrada por cada página que el proceso esté utilizando (o un elemento por cada dirección virtual, independientemente de la validez de ésta).

Esta representación de la tabla resulta natural, ya que los procesos hacen referencia a las páginas mediante las direcciones virtuales de éstas. El sistema operativo debe entonces traducir cada referencia a una dirección física de memoria.

Puesto que la tabla está ordenada según la dirección virtual, el sistema operativo podrá calcular dónde se encuentra en la tabla la entrada de dirección física asociada y podrá utilizar dicho valor directamente.

[24] En términos generales, un "cluster" (o clúster) se refiere a una agrupación o conjunto de elementos relacionados entre sí.

Una de las desventajas de este método es que cada tabla de página puede estar compuesta por millones de entradas. Estas tablas pueden ocupar una gran cantidad de memoria física, simplemente para controlar el modo en que se están utilizando otras partes de la memoria física.

Para resolver este problema, podemos utilizar una ***tabla de páginas invertida***. Las tablas de páginas invertidas tienen una entrada por cada página real (o marco) de la memoria. Cada entrada está compuesta de la dirección virtual de la página almacenada en dicha ubicación de memoria real, e incluye información acerca del proceso que posee dicha página. Por lo tanto, en el sistema sólo habrá una única tabla de páginas y esa tabla sólo tendrá una entrada por cada página de memoria física.

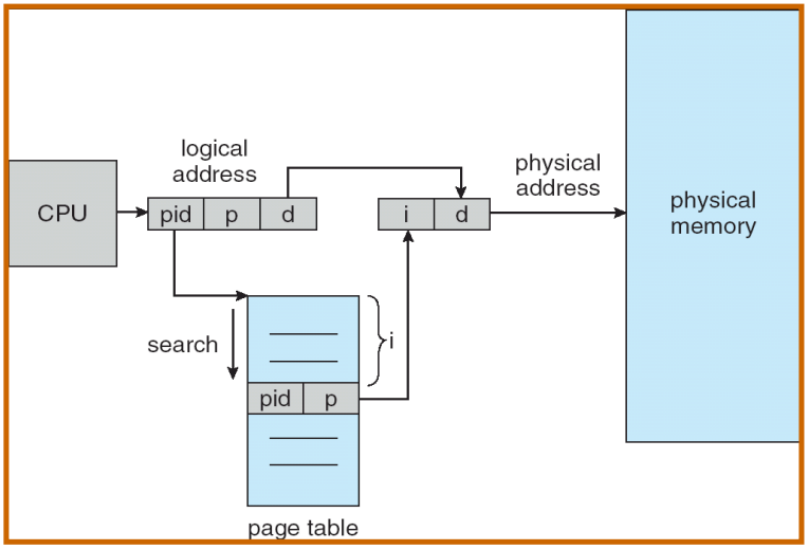


Ilustración 22 - Tabla de páginas invertida.

Si se compara con una tabla de páginas estándar. Las tablas de páginas invertidas requieren a menudo que se almacene un identificador del espacio de direcciones[25] en cada entrada de la tabla de páginas, ya que la tabla contiene usualmente varios espacios de direcciones distintos que se hacen corresponder con la memoria física.

[25] Un identificador de espacio de direcciones (ASID) es un número único que el sistema operativo asigna a cada proceso o tarea en ejecución. Este ASID, junto con la dirección virtual, ayuda a la Unidad de Gestión de Memoria (MMU) a identificar la memoria utilizada por cada proceso, incluso si diferentes procesos utilizan la misma dirección virtual.

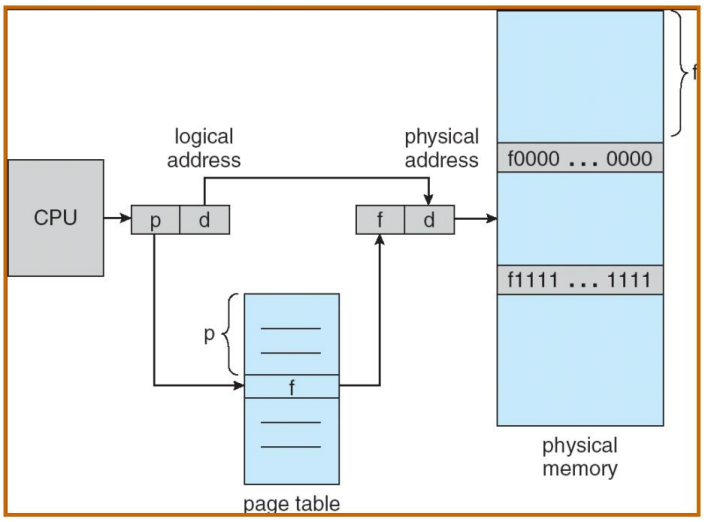


Ilustración 23 - Tabla de páginas estándar.

Almacenar el identificador del espacio de direcciones garantiza que cada página lógica de un proceso concreto se relacione con el correspondiente marco de página física.

Como ejemplos de sistemas que utilizan las tablas de páginas invertidas podemos citar el PowerPC[26] y la arquitectura UltrasPARC[27] de 64 bits.

Los sistemas que utilizan tablas de páginas invertidas tienen dificultades para implementar el concepto de memoria compartida. La memoria compartida se implementa usualmente en forma de múltiples direcciones virtuales (una para cada proceso que comparte la memoria), todas las cuales se hacen corresponder con una misma dirección física. Este método estándar no puede utilizarse con las tablas de páginas invertidas, porque sólo hay una única entrada de página virtual para cada página física y, como consecuencia, una misma página física no puede tener dos (o más) direcciones virtuales compartidas. Una técnica simple para resolver este problema consiste en permitir que la tabla de páginas sólo contenga una única correspondencia de una dirección virtual con la dirección física compartida. Esto significa que las referencias a direcciones virtuales que no estén asociadas darán como resultado un fallo de página.

Referencias y bibliografía de la sección.

- Silberschatz 8-5.
- Stallings pág. 344.

[26] PowerPC (Performance Optimization With Enhanced RISC – Performance Computing) es una arquitectura de procesador basada en el diseño RISC (Reduced Instruction Set Computer), desarrollada originalmente en 1991 por IBM, Apple y Motorola bajo la alianza llamada AIM.
[27] UltraSPARC es una familia de microprocesadores RISC desarrollada por Sun Microsystems a partir de mediados de los años 90. Fue diseñada para usarse principalmente en servidores y estaciones de trabajo que ejecutaban el sistema operativo Solaris, una versión de Unix.

Algoritmos de reemplazo de páginas.

Cuando ocurre un fallo de página, el sistema operativo tiene que elegir una página para desalojarla (eliminarla de memoria) y hacer espacio para la página entrante. Si la página a eliminar se modificó mientras estaba en memoria, debe volver a escribirse en el disco para actualizar la copia del mismo.

No obstante, si la página no se ha modificado (por ejemplo, si contiene el texto del programa), la copia ya está actualizada y no se necesita rescribir. La página que se va a leer sólo sobrescribe en la página que se va a desalojar.

Aunque sería posible elegir una página al azar para desalojarla en cada fallo de página, el rendimiento del sistema es mucho mayor si se selecciona una página que no sea de uso frecuente. Si se elimina una página de uso frecuente, tal vez tenga que traerse de vuelta rápidamente, lo cual produce una sobrecarga adicional. Se ha realizado mucho trabajo, tanto teórico como experimental en el tema de los algoritmos de reemplazo de páginas.

El algoritmo de reemplazo de páginas óptimo.

El mejor algoritmo de reemplazo de páginas posible es fácil de describir, pero imposible de implementar: al momento en que ocurre un fallo de página, hay cierto conjunto de páginas en memoria y una de éstas se referenciará en la siguiente instrucción (la página que contiene la instrucción); otras páginas tal vez no se referencien sino hasta 10, 100 o tal vez 1000 instrucciones después. Cada página se puede etiquetar con el número de instrucciones que se ejecutarán antes de que se haga referencia por primera vez a esa página.

El algoritmo óptimo de reemplazo de páginas establece que la página con la etiqueta más alta debe eliminarse. Si una página no se va a utilizar durante 8 millones de instrucciones y otra no se va a utilizar durante 6 millones de instrucciones, al eliminar la primera se enviará el fallo de página que la obtendrá de vuelta lo más lejos posible en el futuro. Al igual que las personas, las computadoras tratan de posponer los eventos indeseables el mayor tiempo posible.

El único problema con este algoritmo es que no se puede realizar. Al momento del fallo de página, el sistema operativo no tiene forma de saber cuándo será la próxima referencia a cada una de las páginas. Aun así, al ejecutar un programa en un simulador y llevar la cuenta de todas las referencias a páginas, es posible implementar un algoritmo óptimo de reemplazo de página en la *segunda* corrida, al utilizar la información de referencia de páginas recolectada durante la *primera* corrida.

De esta manera, se puede comparar el rendimiento de los algoritmos realizables con el mejor posible. Si un sistema operativo logra un rendimiento de, por ejemplo, sólo 1 por ciento peor que el algoritmo óptimo, el esfuerzo invertido en buscar un mejor algoritmo producirá cuando mucho una mejora del 1 por ciento.

Para evitar cualquier posible confusión, hay que aclarar que este registro de referencias de páginas se refiere sólo al programa que se acaba de medir y sólo con una entrada específica. Por lo tanto, el algoritmo de reemplazo de páginas que se derive de esto es específico para ese programa y esos datos de entrada. Aunque este método es útil para evaluar los algoritmos de reemplazo de páginas, no es útil en sistemas prácticos.

El algoritmo de reemplazo de páginas no usadas recientemente.

Para permitir que el sistema operativo recolecte estadísticas útiles sobre el uso de páginas, la mayor parte de las computadoras con memoria virtual tienen dos bits de estado asociados a cada página. ***R*** se establece cada vez que se hace referencia a la página (lectura o escritura); ***M*** se establece cuando se escribe en la página (es decir, se modifica). Los bits están contenidos en cada entrada de la tabla de páginas, como se muestra en la siguiente figura.

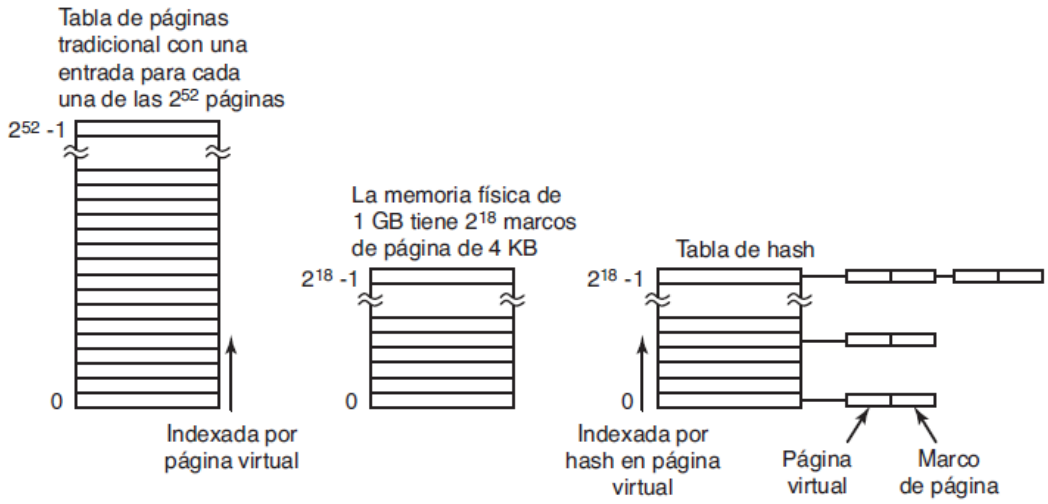


Ilustración 24 - Comparación de una tabla de páginas tradicional con una tabla de páginas invertida.

Es importante tener en cuenta que estos bits se deben actualizar en cada referencia a la memoria, por lo que es imprescindible que se establezcan mediante el hardware. Una vez que se establece un bit en 1, permanece así hasta que el sistema operativo lo restablece.

Si el hardware no tiene estos bits, pueden simularse de la siguiente forma. Cuando se inicia un proceso, todas sus entradas en la tabla de páginas se marcan como que no están en memoria. Tan pronto como se haga referencia a una página, ocurrirá un fallo de página. Entonces, el sistema operativo establece el bit *R* (en sus tablas internas), cambia la entrada en la tabla de páginas para que apunte a la página correcta, con modo de SÓLO LECTURA, y reinicia la instrucción. Si la página se modifica después, ocurrirá otro fallo de página que permite al sistema operativo establecer el bit *M* y cambiar el modo de la página a LECTURA/ESCRITURA.

Los bits *R* y *M* se pueden utilizar para construir un algoritmo simple de paginación de la siguiente manera. Cuando se inicia un proceso, ambos bits de página para todas sus páginas se establecen en 0 mediante el sistema operativo. El bit *R* se borra en forma periódica (en cada interrupción de reloj) para diferenciar las páginas a las que no se ha hecho referencia recientemente de las que si se han referenciado.

Cuando ocurre un fallo de página, el sistema operativo inspecciona todas las páginas y las divide en 4 categorías con base en los valores actuales de sus bits *R* y *M*:

- Clase 0: no ha sido referenciada, no ha sido modificada.

- Clase 1: no ha sido referenciada, ha sido modificada.
- Clase 2: ha sido referenciada, no ha sido modificada.
- Clase 3: ha sido referenciada, ha sido modificada.

Aunque las páginas de la clase 1 parecen a primera instancia imposibles, ocurren cuando una interrupción de reloj borra el bit *R* de una página de la clase 3. Las interrupciones de reloj no borran el bit *M* debido a que esta información se necesita para saber si la página se ha vuelto a escribir en el disco o no. Al borrar *R,* pero no *M* se obtiene una página de clase 1.

El algoritmo ***NRU*** (*Not Recently Used*, No usada recientemente) elimina una página al azar de la clase de menor numeración que no esté vacía. En este algoritmo está implícita la idea de que es mejor eliminar una página modificada a la que no se haya hecho referencia en al menos un pulso de reloj que una página limpia de uso frecuente.

La principal atracción del NRU es que es fácil de comprender, moderadamente eficiente de implementar y proporciona un rendimiento que, aunque no es óptimo, puede ser adecuado.

El algoritmo de reemplazo de páginas primera en entrar, primera en salir (FIFO).

Otro algoritmo de paginación con baja sobrecarga es el de ***primera en entrar, primera en salir*** (*First-In, First-Out,* ***FIFO***). Para ilustrar cómo funciona, consideremos un supermercado con suficientes repisas como para mostrar exactamente *k* productos distintos. Un día, cierta empresa introduce un nuevo alimento de conveniencia: yogurt orgánico instantáneo liofilizado que se puede reconstituir en un horno de microondas. Es un éxito inmediato, por lo que nuestro supermercado finito se tiene que deshacer de un producto antiguo para tenerlo en existencia.

Una posibilidad es buscar el producto que el supermercado haya tenido en existencia por más tiempo (por ejemplo, algo que se empezó a vender hace 120 años) y deshacerse de él por la razón de que nadie está interesado ya. En efecto, el supermercado mantiene una lista ligada de todos los productos que vende actualmente en el orden en el que se introdujeron. El nuevo pasa a la parte final de la lista; el que está al frente de la lista se elimina.

Como un algoritmo de reemplazo de páginas, se aplica la misma idea. El sistema operativo mantiene una lista de todas las páginas actualmente en memoria, en donde la llegada más reciente está en la parte final y la menos reciente en la parte frontal. En un fallo de página, se elimina la página que está en la parte frontal y la nueva página se agrega a la parte final de la lista.

Cuando se aplica en las tiendas, FIFO podría eliminar la gomina para el bigote, pero también podría eliminar harina, sal o mantequilla.

Cuando se aplica a las computadoras, surge el mismo problema. Por esta razón es raro que se utilice FIFO en su forma pura.

El algoritmo de reemplazo de páginas segunda oportunidad.

Una modificación simple al algoritmo FIFO que evita el problema de descartar una página de uso frecuente es inspeccionar el bit *R* de la página más antigua. Si es 0, la página es antigua y no se ha utilizado, por lo que se sustituye de inmediato. Si el bit *R* es 1, el bit se borra, la página se pone al final de la lista de páginas y su tiempo de carga se actualiza, como si acabara de llegar a la memoria. Después la búsqueda continúa.

La operación de este algoritmo, conocido como ***segunda oportunidad***, se muestra en la figura siguiente.

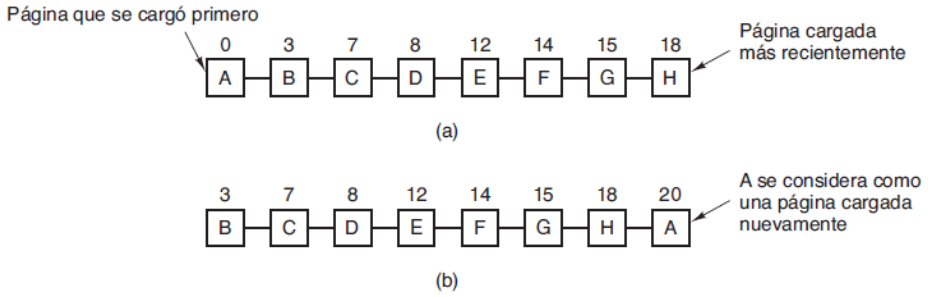


Ilustración 25 - Operación del algoritmo de la segunda oportunidad. (a) Páginas ordenadas con base en FIFO. (b) Lista de las páginas si ocurre un fallo de página en el tiempo 20 y A tiene su bit R activado. Los números encima de las páginas son sus tiempos de carga.

En la parte *a* de la figura vemos que las páginas de la *A* a la *H* se mantienen en una lista ligada y se ordenan con base en el tiempo en que llegaron a la memoria.

Suponga que ocurre un fallo de página en el tiempo 20. La página más antigua es *A*, que llegó en el tiempo 0, cuando se inició el proceso. Si el bit *R* de *A* esta desactivado, se desaloja de la memoria, ya sea escribiéndola en el disco (si está sucia) o sólo se abandona (si está limpia). Por otro lado, si el bit *R* está activado, *A* se coloca al final de la lista y su "tiempo de carga" se restablece al tiempo actual (20). El bit *R* también esta desactivado. La búsqueda de una página adecuada continúa con *B*.

Lo que el algoritmo de segunda oportunidad está buscando es una página antigua a la que no se haya hecho referencia en el intervalo de reloj más reciente. Si se ha hecho referencia a todas las páginas, el algoritmo segunda oportunidad se degenera y se convierte en FIFO puro.

De manera específica, imaginemos que todas las páginas en la parte *a* de la figura tienen sus bits *R* activados. Una por una, el sistema operativo mueve las páginas al final de la lista, desactivando el bit *R* cada vez que adjunta una página al final de la lista. En un momento dado regresará a la página *A*, que ahora tiene su bit *R* desactivado. En este punto, *A* se desaloja. Por ende, el algoritmo siempre termina.

El algoritmo de reemplazo de páginas reloj.

Aunque el algoritmo segunda oportunidad es razonable, también es innecesariamente ineficiente debido a que está moviendo constantemente páginas en su lista. Un mejor método

sería mantener todos los marcos de página en una lista circular en forma de reloj, como se muestra en la figura siguiente.

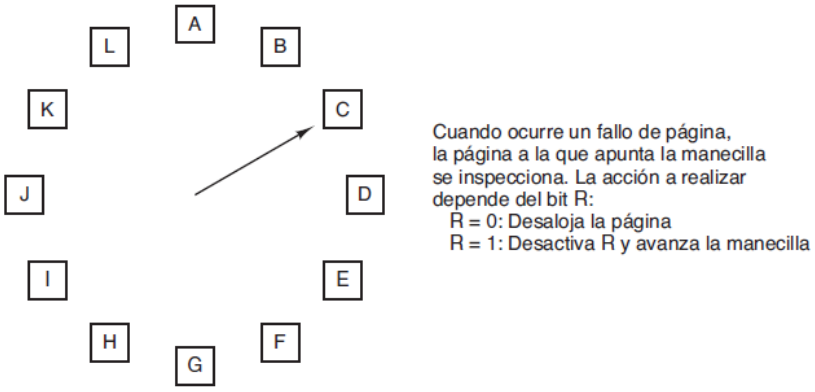


Ilustración 26 - El algoritmo de reemplazo de páginas en reloj.

La manecilla apunta a la página más antigua. Cuando ocurre un fallo de página, la página a la que apunta la manecilla se inspecciona. Si el bit *R* es 0, la página se desaloja, se inserta la nueva página en el reloj en su lugar y la manecilla se avanza una posición. Si *R* es 1, se borra y la manecilla se avanza a la siguiente página. Este proceso se repite hasta encontrar una página con *R* = 0. No es de sorprender que a este algoritmo se le llame en ***reloj***.

El algoritmo de reemplazo de páginas menos usadas recientemente (LRU).

Una buena aproximación al algoritmo óptimo se basa en la observación de que las páginas que se hayan utilizado con frecuencia en las últimas instrucciones, tal vez se volverán a usar con frecuencia en las siguientes. Por el contrario, las páginas que no se hayan utilizado por mucho tiempo probablemente seguirán sin utilizarse por mucho tiempo más. Esta idea sugiere un algoritmo factible: cuando ocurra un fallo de página, hay que descartar la página que no se haya utilizado durante la mayor longitud de tiempo. A esta estrategia se le conoce como paginación ***LRU*** (*Least Recently Used*, Menos usadas recientemente).

Aunque en teoría el algoritmo LRU es realizable, no es barato. Para implementar el LRU por completo, es necesario mantener una lista enlazada de todas las páginas en memoria, con la página de uso más reciente en la parte frontal y la de uso menos reciente en la parte final. La dificultad es que la lista debe actualizarse en cada referencia a memoria. Buscar una página en la lista, eliminarla y después pasarla al frente es una operación que consume mucho tiempo, incluso mediante hardware (suponiendo que pudiera construirse dicho hardware).

Sin embargo, hay otras formas de implementar el algoritmo LRU con hardware especial. Consideremos primero la forma más simple. Este método requiere equipar el hardware con un contador de 64 bits, llamado *C*, el cual se incrementa de manera automática después de cada instrucción.

Además, cada entrada en la tabla de páginas debe tener también un campo lo bastante grande como para poder guardar el contador. Después de cada referencia a memoria, el valor actual

de *C* se almacena en la entrada en la tabla de páginas para la página que se acaba de referenciar. Cuando ocurre un fallo de página, el sistema operativo examina todos los contadores en la tabla de páginas para encontrar el menor. Esa página es la de uso menos reciente.

Ahora veamos un segundo algoritmo LRU mediante hardware. Para una máquina con *n* marcos de página, el hardware del LRU puede mantener una matriz de *n* x *n* bits (inicialmente, todos son 0).

Cada vez que se hace referencia a la página *k*, el hardware primero establece todos los bits de la fila *k* en 1 y después todos los bits de la columna k en 0. En cualquier instante, la fila cuyo valor binario sea menor es la de uso menos reciente, la fila cuyo valor sea el siguiente más bajo es la del siguiente uso menos reciente, y así en lo sucesivo. El funcionamiento de este algoritmo se muestra en la figura siguiente para cuatro marcos de página y referencias a páginas en el siguiente orden: 0 1 2 3 2 1 0 3 2 3.

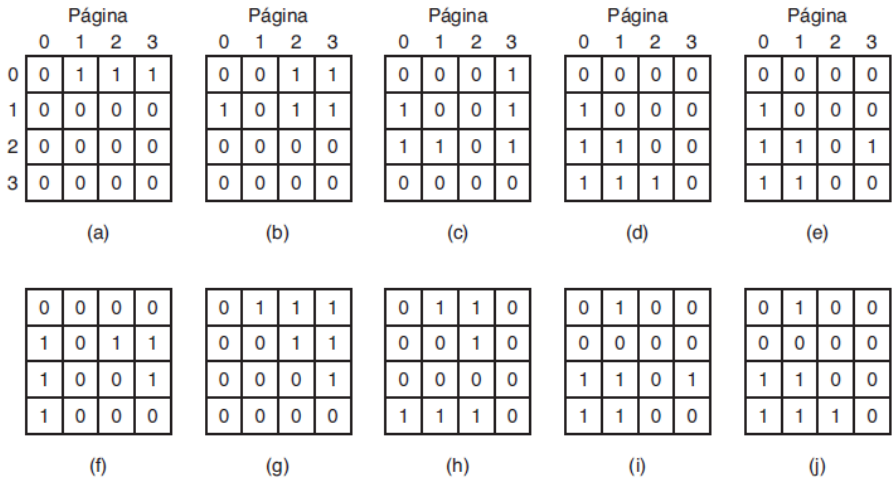


Ilustración 27 - LRU usando una matriz cuando se hace referencia a las páginas en el orden 0, 1, 2, 3, 2, 1, 0, 3, 2, 3.

Después de hacer referencia a la página 0, tenemos la situación de la parte *a* de la figura. Después de hacer referencia a la página 1, tenemos la situación de la parte *b* de la figura , y así en lo sucesivo.

El algoritmo de reemplazo de páginas de conjunto de trabajo.

En la forma más pura de paginación, los procesos inician sin ninguna de sus páginas en la memoria. Tan pronto como la CPU trata de obtener la primera instrucción, recibe un fallo de página, causando que el sistema operativo tenga que traer la página que contiene la primera instrucción. Por lo general a este fallo le siguen otros fallos de página por las variables globales y la pila. Después de cierto tiempo, el proceso tiene la mayoría de las páginas que necesita y se establece para ejecutarse con relativamente pocos fallos de página. A esta estrategia se le conoce como ***paginación bajo demanda***, debido a que las páginas se cargan sólo según la demanda, no por adelantado.

Desde luego que es fácil escribir un programa de prueba que lea de manera sistemática todas las páginas en un espacio de direcciones extenso, produciendo tantos fallos de página que no haya suficiente memoria como para contenerlos todos. Por fortuna, la mayoría de los procesos no trabajan de esta manera. Exhiben una ***localidad de referencia***, lo cual significa que durante cualquier fase de ejecución el proceso hace referencia sólo a una fracción relativamente pequeña de sus páginas.

Por ejemplo, cada pasada de un compilador con varias pasadas hace referencia sólo a una fracción de todas las páginas y cada vez es una fracción distinta.

El conjunto de páginas que utiliza un proceso en un momento dado se conoce como su ***conjunto de trabajo*.** Si todo el conjunto de trabajo está en memoria, el proceso se ejecutará sin producir muchos fallos hasta que avance a otra fase de ejecución (por ejemplo, la siguiente pasada del compilador). Si la memoria disponible es demasiado pequeña como para contener todo el conjunto de trabajo completo, el proceso producirá muchos fallos de página y se ejecutará lentamente, ya que para ejecutar una instrucción se requieren unos cuantos nanosegundos y para leer una página del disco se requiere mucho más tiempo. Con estas proporciones, se requeriría una eternidad para terminar. Se dice que un programa que produce fallos de página cada pocas instrucciones está ***sobrepaginando*** (***thrashing***).

En un sistema de multiprogramación, los procesos se mueven con frecuencia al disco (es decir, todas sus páginas se eliminan de la memoria) para dejar que otros procesos tengan oportunidad de usar la CPU. Acá surge la pregunta de qué hacer cuando un proceso se regresa una y otra vez. Técnicamente, no hay que hacer nada. El proceso producirá fallos de página sólo hasta que se haya cargado su conjunto de trabajo.

El problema es que tener 20, 100 o incluso 1000 fallos de página cada vez que se carga un proceso es algo lento, además de que se desperdicia un tiempo considerable de la CPU, ya que el sistema operativo tarda unos cuantos milisegundos de tiempo de la CPU en procesar un fallo de página.

Por lo tanto, muchos sistemas de paginación tratan de llevar la cuenta del conjunto de trabajo de cada proceso y se aseguran que esté en memoria antes de permitir que el proceso se ejecute. Este método se conoce como ***modelo del conjunto de trabajo***. Está diseñado para reducir en gran parte la proporción de fallos de página. Al proceso de cargar las páginas *antes* de permitir que se ejecuten los procesos también se le conoce como **prepaginación**. Tenga en cuenta que el conjunto de trabajo cambia con el tiempo.

Debido a que el conjunto de trabajo varía lentamente con el tiempo, es posible realizar una predicción razonable en cuanto a qué páginas se necesitarán cuando el programa se reinicie, con base en su conjunto de trabajo la última vez que se detuvo. La prepaginación consiste en cargar estas páginas antes de reanudar el proceso.

Para implementar el modelo del conjunto de trabajo, es necesario que el sistema operativo lleve la cuenta de cuáles páginas están en el conjunto de trabajo. Tener esta información también nos conduce de inmediato a un posible algoritmo de reemplazo de páginas: cuando ocurra un fallo de página, hay que buscar una página que no se encuentre en el conjunto de trabajo y desalojarla.

Para implementar dicho algoritmo se requiere una manera precisa de determinar cuáles páginas están en el conjunto de trabajo.

Desde luego que tener una definición operacional del conjunto de trabajo no significa que haya una forma eficiente de calcularlo durante la ejecución de un programa.

Sin embargo, el proceso de mantener el registro de desplazamiento y procesarlo en un fallo de página sería en extremo costoso, por lo que esta técnica nunca se utiliza.

En vez de ello, se utilizan varias aproximaciones. Por ejemplo, en vez de definir el conjunto de trabajo como las páginas utilizadas durante los 10 millones de referencias a memoria anteriores, podemos definirlo como el conjunto de páginas utilizadas durante los últimos 100 milisegundos de tiempo de ejecución. En la práctica, dicha definición es igual de conveniente y es mucho más fácil trabajar con ella. Observemos que para cada proceso sólo cuenta su propio tiempo de ejecución. Así, si un proceso empieza su ejecución en el tiempo *T* y ha tenido 40 milisegundos de tiempo de la CPU a un tiempo real de *T* + 100 milisegundos, para los fines del conjunto de trabajo su tiempo es de 40 mseg. La cantidad de tiempo de la CPU que ha utilizado en realidad un proceso desde que empezó se conoce comúnmente como su ***tiempo virtual actual***.

Con esta aproximación, el conjunto de trabajo de un proceso es el conjunto de páginas a las que se ha hecho referencia durante los últimos τ segundos de tiempo virtual.

Ahora veamos un algoritmo de reemplazo de páginas basado en el conjunto de trabajo. La idea básica es buscar una página que no esté en el conjunto de trabajo y desalojarla. En la figura siguiente vemos una porción de una tabla de páginas para cierta máquina.

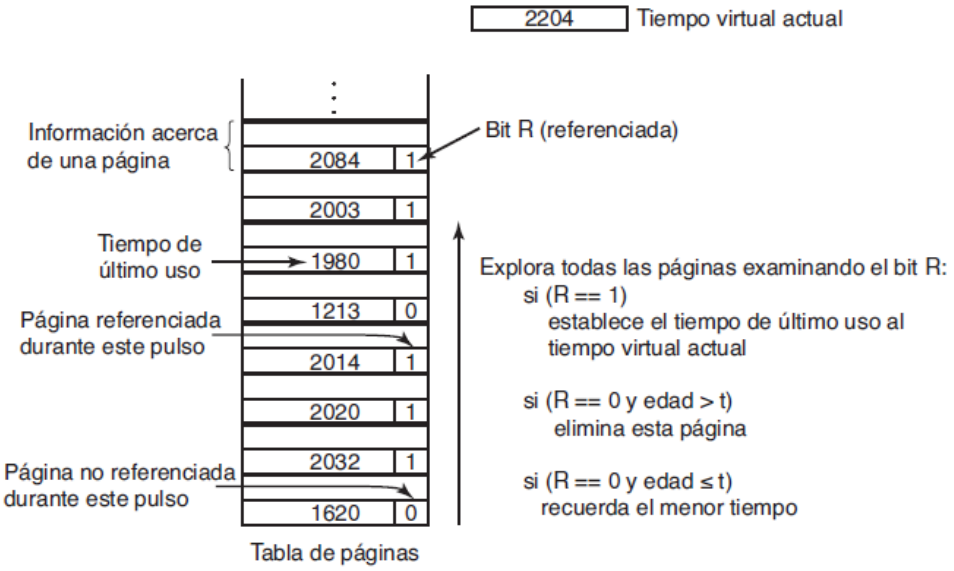


Ilustración 28 - El algoritmo del conjunto de trabajo.

Ya que sólo las páginas que están en memoria se consideran como candidatos para el desalojo, este algoritmo ignora las páginas que no están en la memoria. Cada entrada contiene (al menos) dos elementos clave de información: el tiempo (aproximado) que se utilizó la página por última vez y el bit *R* (referenciada). El rectángulo vacío de color blanco simboliza los demás campos que no necesita este algoritmo, como el número de marco de página, los bits de protección y el bit *M* (modificado).

El algoritmo funciona de la siguiente manera. Se supone que el hardware debe establecer los bits *R* y *M*, como vimos antes. De manera similar, se supone que una interrupción periódica de reloj hace que se ejecute software para borrar el bit *Referenciada* en cada pulso de reloj. En cada fallo de página se explora la tabla de páginas en busca de una página adecuada para desalojarla.

A medida que se procesa cada entrada, se examina el bit *R*. Si es 1, el tiempo virtual actual se escribe en el campo *tiempo de último uso* en la tabla de páginas, indicando que la página estaba en uso al momento en que ocurrió el fallo de página. Como se hizo referencia a la página durante el pulso de reloj actual, es evidente que está en el conjunto de trabajo y no es candidata para la eliminación (se supone que τ abarca varios pulsos de reloj).

Si *R* es 0, no se ha hecho referencia a la página durante el pulso de reloj actual y puede ser candidata para la eliminación. Para ver si debe o no eliminarse, se calcula su edad (el tiempo virtual actual menos su *tiempo de último uso*) y se compara con τ. Si la edad es mayor que τ, la página ya no se encuentra en el conjunto de trabajo y la nueva página la sustituye. La exploración continúa actualizando las entradas restantes.

No obstante, si *R* es 0 pero la edad es menor o igual que τ, la página está todavía en el conjunto de trabajo. La página se reserva temporalmente, pero se apunta la página con la mayor edad (el menor valor de *tiempo de último uso*). Si toda la tabla completa se explora sin encontrar un candidato para desalojar, eso significa que todas las páginas están en el conjunto de trabajo. En ese caso, si se encontraron una o más páginas con *R = 0,* se desaloja la más antigua. En el peor caso, se ha hecho referencia a todas las páginas durante el pulso de reloj actual (y, por ende, todas tienen *R* = 1), por lo que se selecciona una al azar para eliminarla, de preferencia una página limpia, si es que existe.

El algoritmo de reemplazo de páginas WSClock.

Al algoritmo básico del conjunto de trabajo es incómodo ya que exige explorar toda la tabla de páginas en cada fallo de página hasta localizar un candidato adecuado. Un algoritmo mejorado, basado en el algoritmo de reloj pero que también utiliza la información del conjunto de trabajo, se conoce como ***WSClock***. Debido a su simplicidad de implementación y buen rendimiento, es muy utilizado en la práctica.

La estructura de datos necesaria es una lista circular de marcos de página, como en el algoritmo de reloj, mostrada en la parte a de la figura siguiente.

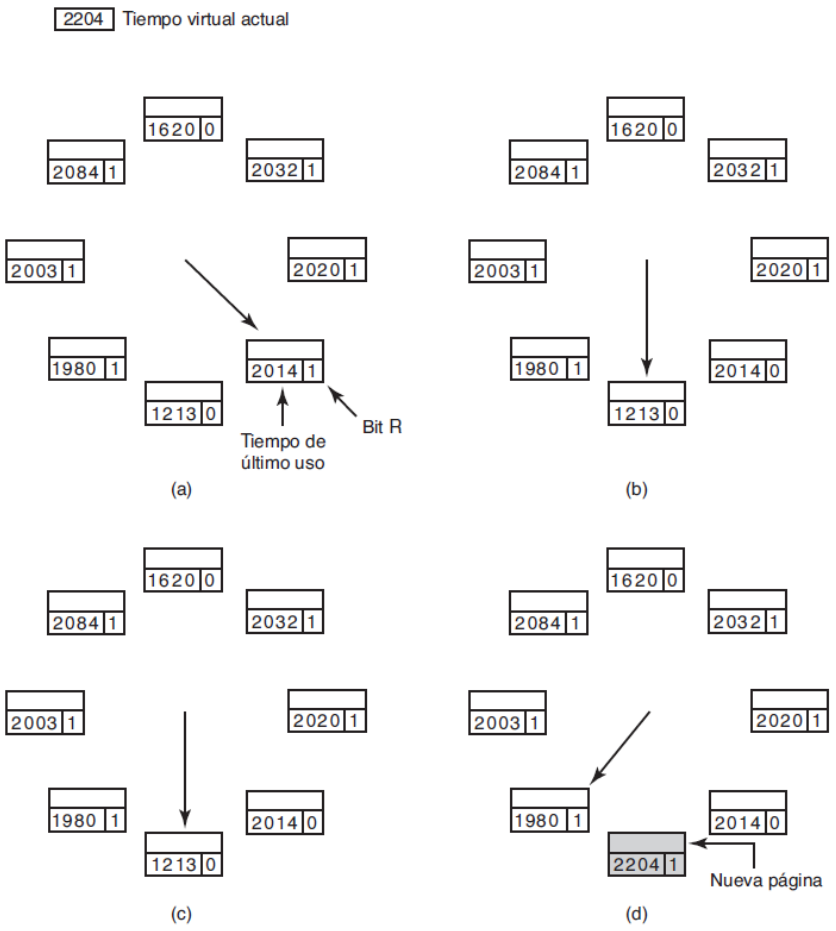


Ilustración 29 - Operación del algoritmo WSClock. (a) y (b) dan un ejemplo de lo que ocurre cuando R _ 1. (c) y (d) dan un ejemplo de cuando R _ 0.

Al principio, esta lista está vacía. Cuando se carga la primera página, se agrega a la lista. A medida que se agregan más páginas, pasan a la lista para formar un anillo. Cada entrada contiene el campo *Tiempo de último uso* del algoritmo básico del conjunto de trabajo, así como el bit *R* (mostrado) y el bit *M* (no mostrado).

Al igual que con el algoritmo de reloj, en cada fallo de página se examina primero la página a la que apunta la manecilla. Si el bit *R* es 1, la página se ha utilizado durante el pulso actual, por lo que no es candidata ideal para la eliminación. Después el bit *R* se establece en 0, la manecilla se avanza a la siguiente página y se repite el algoritmo para esa página. El estado después de esta secuencia de eventos se muestra en la parte *b* de la figura.

Ahora consideremos lo que ocurre si la página a la que apunta la manecilla tiene *R* = 0, como se muestra en la parte *c* figura. Si la edad es mayor que τ y la página está limpia, significa que no se encuentra en el conjunto de trabajo y existe una copia válida en el disco. El marco de página simplemente se reclama y la nueva página se coloca ahí, como se muestra en la parte *d* de la figura.

Por otro lado, si la página está sucia no se puede reclamar de inmediato, ya que no hay una copia válida presente en el disco. Para evitar una conmutación de procesos, la escritura al disco se planifica, pero la manecilla avanza y el algoritmo continúa con la siguiente página.

Después de todo, podría haber una página antigua y limpia más allá de la línea que se pueda utilizar de inmediato.

En principio, todas las páginas se podrían planificar para la E/S de disco en un ciclo alrededor del reloj. Para reducir el tráfico de disco se podría establecer un límite para permitir que se escriban de vuelta un máximo de *n* páginas. Una vez que se llega a este límite, no se planifican nuevas escrituras.

Es necesario preguntarse qué ocurre si la manecilla llega otra vez a su punto inicial. Hay dos casos a considerar:

1. Se ha planificado por lo menos una escritura.
2. No se han planificado escrituras.

En el primer caso, la manecilla sólo sigue moviéndose, buscando una página limpia. Como se han planificado una o más escrituras, en algún momento se completará alguna escritura y su página se marcará como limpia. La primera página limpia que se encuentre se desaloja. Esta página no es necesariamente la primera escritura planificada, ya que el controlador de disco puede reordenar escrituras para poder optimizar el rendimiento del disco.

En el segundo caso, todas las páginas están en el conjunto de trabajo, de otra manera se hubiera planificado por lo menos una escritura. Sin información adicional, lo más simple por hacer es reclamar cualquier página limpia y usarla. La ubicación de una página limpia podría rastrearse durante el barrido. Si no existen páginas limpias, entonces se selecciona la página actual como la víctima y se escribe de vuelta al disco.

Algoritmo	Comentario
Óptimo	No se puede implementar, pero es útil como punto de comparación
NRU (No usadas recientemente)	Una aproximación muy burda del LRU
FIFO (primera en entrar, primera en salir)	Podría descartar páginas importantes
Segunda oportunidad	Gran mejora sobre FIFO
Reloj	Realista
LRU (menos usadas recientemente)	Excelente, pero difícil de implementar con exactitud
NFU (no utilizadas frecuentemente)	Aproximación a LRU bastante burda
Envejecimiento	Algoritmo eficiente que se aproxima bien a LRU
Conjunto de trabajo	Muy costoso de implementar
WSClock	Algoritmo eficientemente bueno

Referencias y bibliografía de la sección.

- Tanenbaum 3-4.

Administración de memoria libre.

Cuando la memoria se asigna en forma ***dinámica***, el sistema operativo debe administrarla. En términos generales, hay dos formas de llevar el registro del uso de la memoria: ***mapas de bits*** y ***listas libres (listas ligadas)***.

Administración de memoria con mapas de bits.

Con un mapa de bits, la memoria se divide en unidades de asignación tan pequeñas como unas cuantas palabras y tan grandes como varios kilobytes. Para cada unidad de asignación hay un bit correspondiente en el mapa de bits, que es 0 si la unidad está libre y 1 si está ocupada (o viceversa).

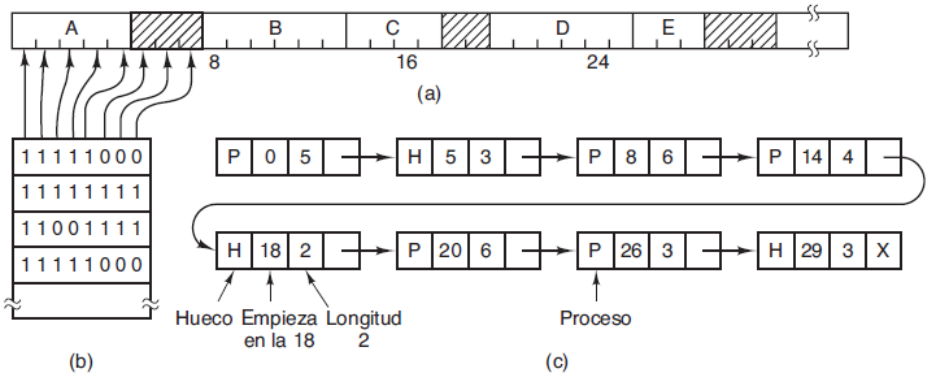


Ilustración 30 - (a) Una parte de la memoria con cinco procesos y tres huecos. Las marcas de graduación muestran las unidades de asignación de memoria. Las regiones sombreadas (0 en el mapa de bits) están libres. (b) El mapa de bits correspondiente. (c) La misma información en forma de lista.

La figura anterior muestra parte de la memoria y el mapa de bits correspondiente. El tamaño de la unidad de asignación es una importante cuestión de diseño. Entre más pequeña sea la unidad de asignación, mayor será el mapa de bits. Sin embargo, aun con una unidad de asignación tan pequeña como 4 bytes, 32 bits de memoria sólo requerirán 1 bit del mapa.

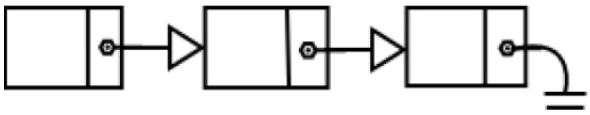
Si la unidad de asignación se elige de manera que sea grande, el mapa de bits será más pequeño, pero se puede desperdiciar una cantidad considerable de memoria en la última unidad del proceso si su tamaño no es un múltiplo exacto de la unidad de asignación.

Un mapa de bits proporciona una manera simple de llevar el registro de las palabras de memoria en una cantidad fija de memoria, debido a que el tamaño del mapa de bits sólo depende del tamaño de la memoria y el tamaño de la unidad de asignación.

El problema principal es que, cuando se ha decidido llevar un proceso de *k* unidades a la memoria, el administrador de memoria debe buscar en el mapa para encontrar una serie de *k* bits consecutivos con el valor 0 en el mapa de bits. El proceso de buscar en un mapa de bits una serie de cierta longitud es una operación lenta (debido a que la serie puede cruzar los límites entre las palabras en el mapa); éste es un argumento contra los mapas de bits.

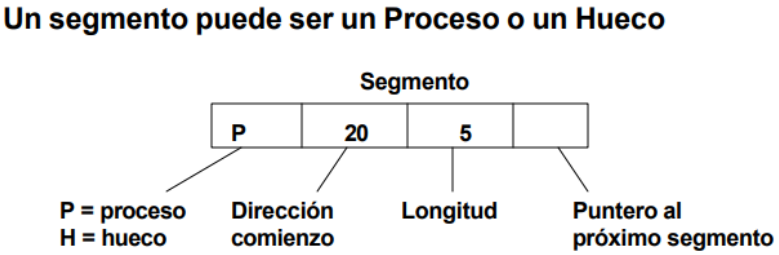
Administración de memoria con listas ligadas.

Otra manera de llevar el registro de la memoria es mantener una ***lista ligada*** (o enlazada) de segmentos de memoria asignados y libres, en donde un segmento contiene un proceso o es un hueco vacío entre dos procesos.



Lista enlazada

Volviendo al ejemplo de la última figura, la memoria de (a) se representa en (c) como una lista ligada de segmentos. Cada entrada en la lista especifica un hueco (H) o un proceso (P), la dirección en la que inicia, la longitud y un apuntador a la siguiente entrada.



En este ejemplo, la lista de segmentos se mantiene ordenada por dirección. Al ordenarla de esta manera, tenemos la ventaja de que cuando termina un proceso o se intercambia, el proceso de actualizar la lista es simple. Un proceso en terminación por lo general tiene dos vecinos (excepto cuando se encuentra en la parte superior o inferior de la memoria). Éstos pueden ser procesos o huecos, lo cual conduce a las cuatro combinaciones de la figura siguiente.

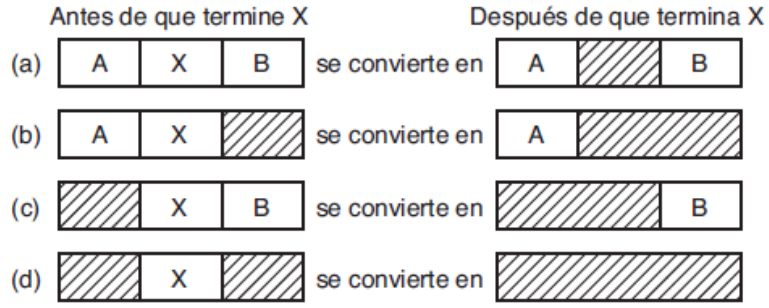
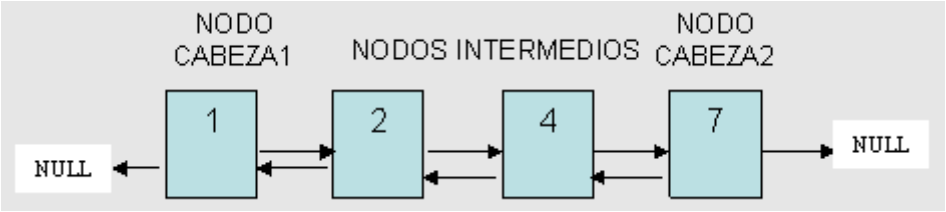


Ilustración 31 - Cuatro combinaciones de los vecinos para el proceso en terminación, X.

Para actualizar la lista se requiere reemplazar una P por una H. En (b) y (c) dos entradas se fusionan en una sola y la lista se reduce en una entrada. En (d), se fusionan las tres entradas y dos elementos son eliminados de la lista.

Como la ranura de la tabla de procesos para el proceso en terminación en general apuntará a la entrada en la lista para el mismo proceso, puede ser conveniente tener la lista como una lista doblemente ligada (o una lista para la memoria usada y otra para los huecos), en vez de la lista simplemente ligada. Esta estructura facilita encontrar la entrada anterior y ver si es posible una combinación.



Referencias y bibliografía de la sección.

- Tanenbaum 3-2-3.

Intercambio.

En la práctica, la cantidad total de memoria principal que requieren todos los procesos es, a menudo, mucho mayor de lo que puede acomodarse en la memoria disponible.

En un sistema Windows o Linux común, se pueden iniciar entre 40 y 60 procesos o más cada vez que se inicia la computadora. Por ejemplo, cuando se instala una aplicación Windows, a menudo emite comandos de manera que en los inicios subsiguientes del sistema se inicie un proceso que no haga nada, excepto buscar actualizaciones para la aplicación. Dicho proceso puede ocupar fácilmente de 5 a 10 MB de memoria.

Otros procesos en segundo plano comprueban el correo entrante, las conexiones de red entrantes y muchas otras cosas. Y todo esto se hace antes de que se inicie el primer programa de usuario.

Hoy en día, los programas de aplicaciones de usuario serios pueden ejecutarse ocupando fácilmente desde unos pocos megabytes hasta varios gigabytes. En consecuencia, para mantener todos los procesos en memoria todo el tiempo se requiere una gran cantidad de memoria.

A través de los años se han desarrollado dos esquemas generales para lidiar con la sobrecarga de memoria. La estrategia más simple, conocida como ***intercambio***, consiste en llevar cada proceso completo a memoria, ejecutarlo durante cierto tiempo y después regresarlo al disco.

Los procesos inactivos mayormente son almacenados en disco, de tal manera que no ocupan memoria cuando no se están ejecutando (aunque algunos de ellos se despiertan periódicamente para realizar su trabajo y después vuelven a quedar inactivos).

La otra estrategia, conocida como ***memoria virtual***, permite que los programas se ejecuten incluso cuando sólo se encuentran en forma parcial en la memoria.

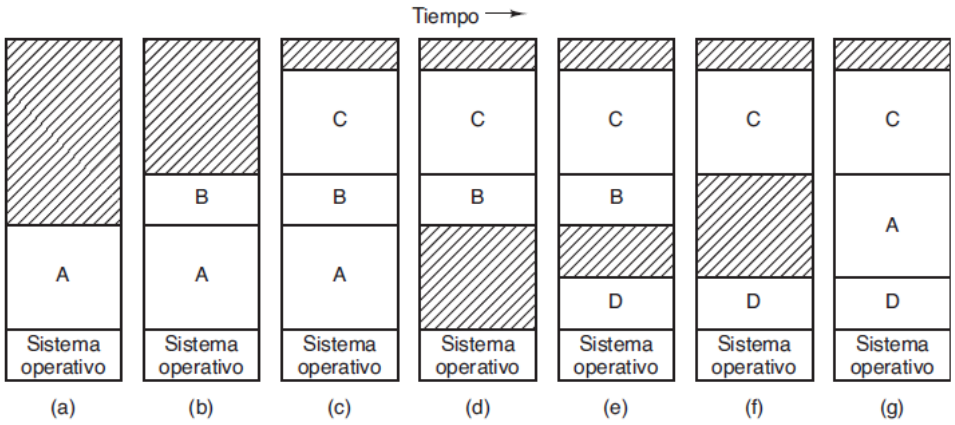


Ilustración 32 - La asignación de la memoria cambia a medida que llegan procesos a la memoria y salen de ésta. Las regiones sombreadas son la memoria sin usar.

Como vemos en la figura anterior, al principio, sólo el proceso *A* está en la memoria. Después los procesos *B* y *C* se crean o se intercambian desde disco. En otra parte de la figura (d), *A* se intercambia al disco. Después llega *D* y sale *B*. Por último, *A* entra de nuevo.

Como *A* está ahora en una ubicación distinta, las direcciones que contiene se deben reubicar, ya sea mediante software cuando se intercambia o (más probablemente) mediante hardware durante la ejecución del programa. Por ejemplo, los registros base y límite funcionarían bien en este caso.

Cuando el intercambio crea varios huecos en la memoria, es posible combinarlos todos en uno grande desplazando los procesos lo más hacia abajo que sea posible. Esta técnica se conoce como ***compactación de memoria***. Por lo general no se realiza debido a que requiere mucho tiempo de la CPU. Por ejemplo, en una máquina con 1 GB que pueda copiar 4 bytes en 20 nseg, se requerirían aproximadamente 5 segundos para compactar toda la memoria.

Un aspecto que vale la pena mencionar es la cantidad de memoria que debe asignarse a un proceso cuando éste se crea o se intercambia. Si los procesos se crean con un tamaño fijo que nunca cambia, entonces la asignación es sencilla: el sistema operativo asigna exactamente lo necesario, ni más ni menos.

No obstante, si los segmentos de datos de los procesos pueden crecer, por ejemplo, mediante la asignación dinámica de memoria proveniente de un heap[28], como en muchos lenguajes de programación, ocurre un problema cuando un proceso trata de crecer.

Si hay un hueco adyacente al proceso, puede asignarse y se permite al proceso crecer en el hueco; si el proceso está adyacente a otro proceso, el proceso en crecimiento tendrá que moverse a un hueco en memoria que sea lo bastante grande como para alojarlo, o habrá que intercambiar uno o más procesos para crear un hueco con el tamaño suficiente.

Si un proceso no puede crecer en memoria y el área de intercambio en el disco está llena, el proceso tendrá que suspenderse hasta que se libere algo de espacio (o se puede eliminar).

Si se espera que la mayoría de los procesos crezcan a medida que se ejecuten, probablemente sea conveniente asignar un poco de memoria adicional cada vez que se intercambia o se mueve un proceso, para reducir la sobrecarga asociada con la acción de mover o intercambiar procesos que ya no caben en su memoria asignada.

No obstante, al intercambiar procesos al disco, se debe intercambiar sólo la memoria que se encuentre en uso; es un desperdicio intercambiar también la memoria adicional.

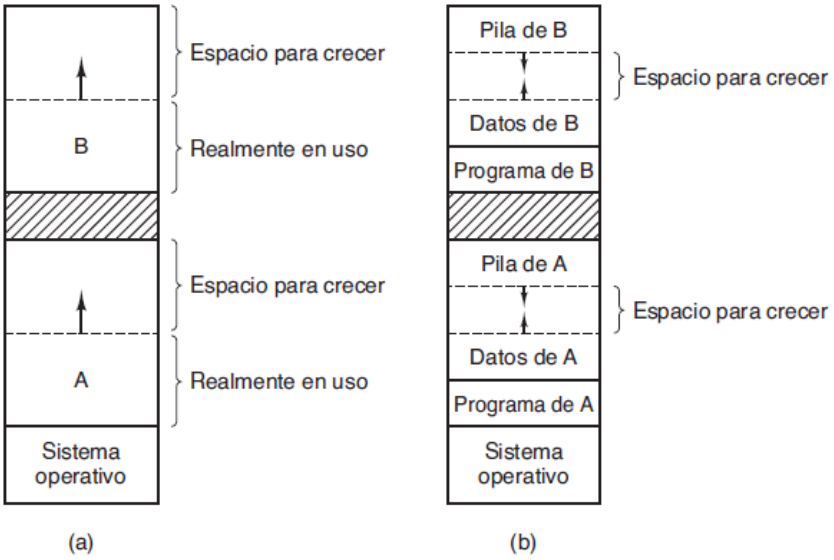


Ilustración 33 - (a) Asignación de espacio para un segmento de datos en crecimiento. (b) Asignación de espacio para una pila en crecimiento y un segmento de datos en crecimiento.

En la figura anterior podemos ver una configuración de memoria en la cual se ha asignado espacio para que crezcan dos procesos.

[28] Es un área de memoria dinámica donde se almacenan objetos y datos durante la ejecución de un programa. A diferencia de la pila (stack), que tiene un tamaño fijo y utiliza una gestión de memoria de tipo "LIFO" (último en entrar, primero en salir), el heap permite una asignación y liberación más flexible de memoria.

Referencias y bibliografía de la sección.

- Tanenbaum 3-2.

Memoria virtual.

Mientras que los registros base y límite se pueden utilizar para crear la abstracción de los espacios de direcciones, hay otro problema que se tiene que resolver: la administración del agrandamiento del software (*bloatware*).

Aunque el tamaño de las memorias se incrementa con cierta rapidez, el del software aumenta con una mucha mayor. En la década de 1980, muchas universidades operaban un sistema de tiempo compartido con docenas de usuarios (más o menos satisfechos) trabajando simultáneamente en una VAX[29] de 4 MB. Ahora Microsoft recomienda tener por lo menos 4 GB para que un sistema Windows 11 de un solo usuario ejecute aplicaciones simples.



Ilustración 34 - 4 MB VAX 8600 memory board.

Como consecuencia de estos desarrollos, existe la necesidad de ejecutar programas que son demasiado grandes como para caber en la memoria y sin duda existe también la necesidad de tener sistemas que puedan soportar varios programas ejecutándose al mismo tiempo, cada uno de los cuales cabe en memoria, pero que en forma colectiva exceden el tamaño de la misma.

[29] VAX (acrónimo de extensión de dirección virtual) es una serie de computadoras con una arquitectura de conjunto de instrucciones (ISA) de 32 bits y memoria virtual, desarrollada y comercializada por Digital Equipment Corporation (DEC) a finales del siglo XX.

El intercambio no es una opción atractiva, ya que una unidad de almacenamiento tiene una velocidad de transferencia muy baja en relación a la RAM y se hace la carga completa del proceso en memoria.

El problema de que los programas sean más grandes que la memoria ha estado presente desde los inicios de la computación, en áreas limitadas como la ciencia y la ingeniería (para simular la creación del universo o, incluso, para simular una nueva aeronave, se requiere mucha memoria).

Una solución que se adoptó en la década de 1960 fue dividir los programas en pequeñas partes, conocidas como ***sobrepuestos*** (***overlays***). Cuando empezaba un programa, todo lo que se cargaba en memoria era el administrador de sobrepuestos, que de inmediato cargaba y ejecutaba el sobrepuesto 0; cuando éste terminaba, indicaba al administrador de sobrepuestos que cargara el 1 encima del sobrepuesto 0 en la memoria (si había espacio) o encima del mismo (si no había espacio).

Algunos sistemas de sobrepuestos eran muy complejos, ya que permitían varios sobrepuestos en memoria a la vez. Los sobrepuestos se mantenían en el disco, intercambiándolos primero hacia adentro de la memoria y después hacia afuera de la memoria mediante el administrador de sobrepuestos.

Aunque el trabajo real de intercambiar sobrepuestos hacia adentro y hacia afuera de la memoria lo realizaba el sistema operativo, el de dividir el programa en partes tenía que realizarlo el programador en forma manual.

El proceso de dividir programas grandes en partes modulares más pequeñas consumía mucho tiempo, y era aburrido y propenso a errores. Pocos programadores eran buenos para esto. No pasó mucho tiempo antes de que alguien ideara una forma de pasar todo el trabajo a la computadora.

El método ideado en la década de 1960 se conoce actualmente como ***memoria virtual***. La idea básica detrás de la memoria virtual es que cada proceso tiene su propio espacio de direcciones, el cual se divide en trozos llamados ***páginas***.

Cada página es un rango contiguo de direcciones. Estas páginas se asocian a la memoria física, pero no todas tienen que estar en la memoria física para poder ejecutar el programa. Cuando el programa hace referencia a una parte de su espacio de direcciones que está en la memoria física, el hardware realiza la asociación necesaria al instante.

Cuando el programa hace referencia a una parte de su espacio de direcciones que ***no*** está en la memoria física, el sistema operativo recibe una alerta para buscar la parte faltante y volver a ejecutar la instrucción que falló.

En cierto sentido, la memoria virtual es una generalización de la idea de los registros base y límite.

Con la memoria virtual, en vez de tener una reubicación por separado para los segmentos de texto y de datos, todo el espacio de direcciones se puede asociar a la memoria física en unidades pequeñas equitativas.

Podríamos decir entonces, que la memoria virtual es un mecanismo que permite a un programa usar más memoria de la que físicamente tiene el sistema.

Es importante entender la sutil diferencia entre la ***memoria virtual*** y la ***paginación bajo demanda***. Mientras la primera es el objetivo o el concepto general, la ***paginación bajo demanda*** es la técnica o el mecanismo para lograr ese objetivo.

- **Objetivo:** Ejecutar programas más grandes que la memoria principal → **Memoria Virtual**.
- **Técnica:** Cargar solo las páginas que se van a usar → **Paginación bajo demanda**.
- **Resultado:** Uso eficiente de la **memoria principal**, inicio rápido de aplicaciones y mayor capacidad de tener muchos procesos abiertos.

La memoria virtual funciona muy bien en un sistema de multiprogramación, con bits y partes de muchos programas en memoria a la vez. Mientras un programa espera a que una parte del mismo se lea y coloque en la memoria, la CPU puede otorgarse a otro proceso.

La posibilidad de ejecutar un programa que sólo se encuentra parcialmente en la memoria proporciona muchas ventajas:

- Los programas ya no estarían restringidos por la cantidad de memoria física disponible, usuarios podrían escribir programas para un espacio de direcciones virtual extremadamente grande, simplificándose así la tarea de programación.
- Puesto que cada programa de usuario podría ocupar menos memoria física, se podrían ejecutar más programas al mismo tiempo, con el correspondiente incremento en la tasa de utilización del procesador y en la tasa de procesamiento, y sin incrementar el tiempo de puesta ni el tiempo de ejecución.
- Se necesitarían menos operaciones de E/S para cargar o intercambiar cada programa de usuario con el fin de almacenarlo en memoria, de manera que cada programa de usuario ejecutaría más rápido.

Por tanto, ejecutar programas que no se encuentren completamente en memoria proporciona ventajas tanto para el sistema como para el usuario.

La memoria virtual incluye la separación de la memoria lógica, tal como la percibe el usuario, con respecto a la memoria física. Esta separación permite proporcionar a los programadores una memoria virtual extremadamente grande, cuando sólo está disponible una memoria física de menor tamaño.

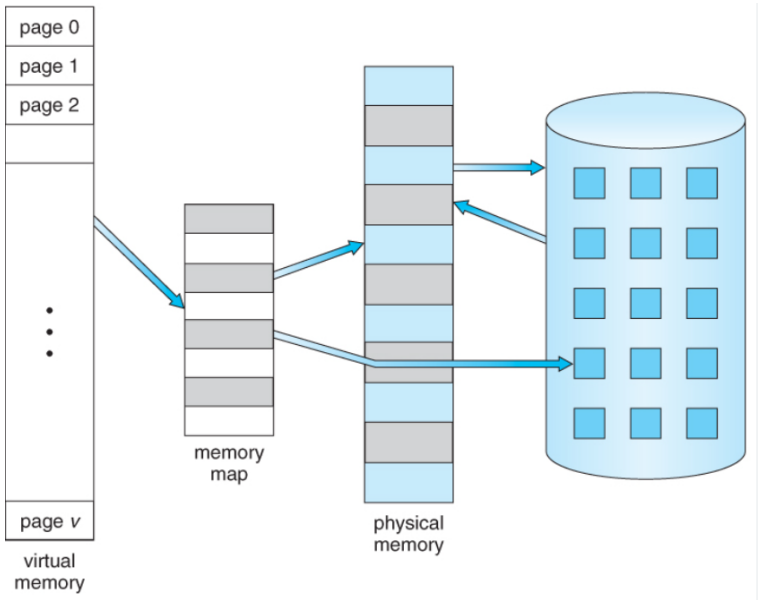


Ilustración 35 - Diagrama que muestra una memoria virtual de tamaño mayor que la memoria física.

La memoria virtual facilita enormemente la tarea de programación, porque el programador ya no tiene por qué preocuparse de la cantidad de memoria física disponible; en lugar de ello, puede concentrarse en el problema que deba resolver mediante su programa.

El espacio de direcciones virtuales de un proceso hace referencia a la forma lógica (o virtual) de almacenar un proceso en la memoria. Típicamente, esta forma consiste en que el proceso comienza en una cierta dirección lógica (por ejemplo, la dirección 0) y está almacenado de forma contigua en la memoria, como se muestra en la siguiente figura.

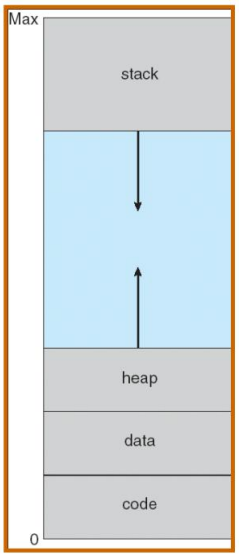


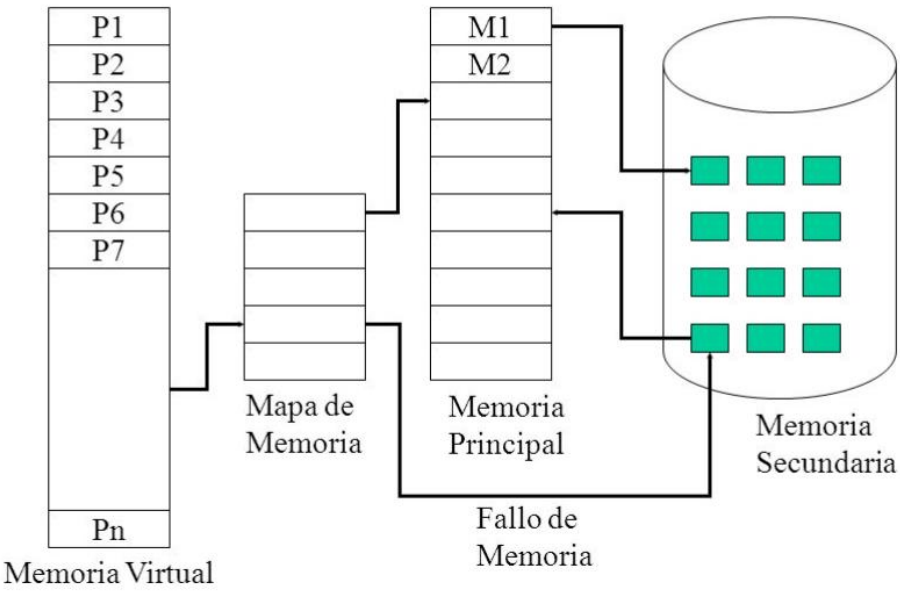
Ilustración 36 - Espacio de direcciones virtual.

Recordemos que la memoria física puede estar organizada en marcos de página y que los marcos de página física asignados a un proceso pueden no ser contiguos. Es responsabilidad de la unidad de gestión de memoria (MMU, memory management unit) establecer la correspondencia entre las páginas lógicas y los marcos de página física de la memoria.



Observemos que en la figura que ilustra el espacio de direcciones virtual, dejamos que el cúmulo crezca hacia arriba en la memoria, ya que se utiliza para la asignación dinámica de memoria. De forma similar, permitimos que la pila crezca hacia abajo en la memoria con las sucesivas llamadas a función.

El gran espacio vacío (o hueco) entre el cúmulo y la pila forma parte del espacio de direcciones virtual, pero sólo consumirá páginas físicas reales cuando crezcan el cúmulo o la pila. Los espacios de direcciones virtuales que incluyen este tipo de huecos se denominan ***espacios de direcciones dispersos***.



Resulta ventajoso utilizar un espacio de direcciones disperso, porque los huecos pueden rellenarse a medida que crecen los segmentos de pila o de cúmulo, o también si queremos montar dinámicamente una serie de bibliotecas (o posiblemente otros objetos compartidos) durante la ejecución del programa.

Además de separar la memoria lógica de la memoria física, la memoria virtual también permite que dos o más procesos compartan los archivos y la memoria mediante mecanismos de compartición de páginas. Esto proporciona las siguientes ventajas:

- Las bibliotecas del sistema pueden ser compartidas por numerosos procesos, mapeando el objeto compartido sobre un espacio de direcciones virtual. Aunque cada proceso considera que las bibliotecas compartidas forman parte de su propio espacio de direcciones virtual, las páginas reales de memoria física en las que residen las bibliotecas estarán compartidas por todos los procesos. Normalmente, las bibliotecas se mapean en modo sólo lectura dentro del espacio de cada proceso con el cual se monten.
- De forma similar, la memoria virtual permite a los procesos compartir memoria. Dos o más procesos pueden comunicarse utilizando memoria compartida. La memoria virtual permite que un proceso cree una región de memoria que se pueda compartir con otro proceso. Los procesos que compartan esta región la consideran parte de su espacio de direcciones virtual, aunque en realidad las páginas físicas reales de la memoria estarán compartidas entre los distintos procesos.
- La memoria virtual puede permitir que se compartan páginas durante la creación de procesos mediante la llamada al sistema fork (), acelerando así la tarea de creación de procesos.

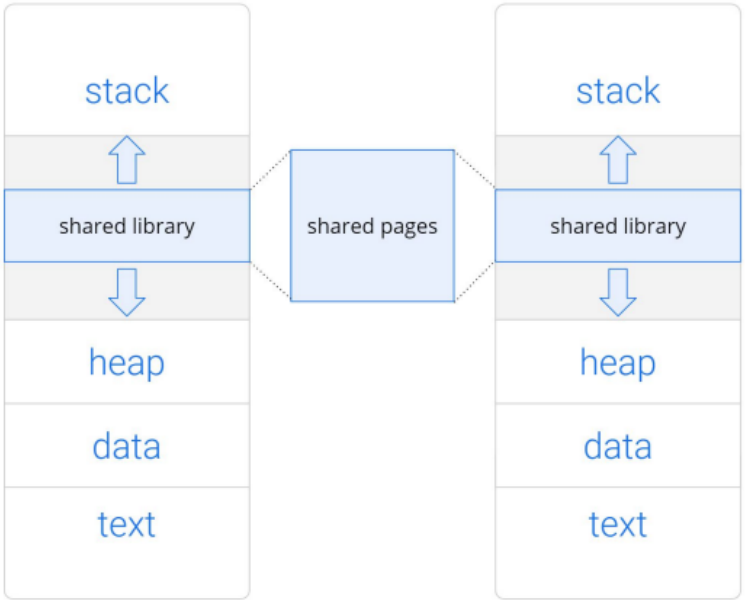


Ilustración 37 - Biblioteca compartida mediante memoria virtual.

Referencias y bibliografía de la sección.

- Tanenbaum 3-3.
- Silberschatz 9-1.

Paginación bajo demanda.

Consideremos cómo podría cargarse un programa ejecutable desde el disco a la memoria. Una opción consiste en cargar el programa completo en memoria física en el momento de ejecutar el programa. Sin embargo, esta técnica presenta el problema de que puede que no necesitemos inicialmente todo el programa en la memoria. Consideremos un programa que comience con una lista de acciones disponibles, de entre las cuales el usuario debe seleccionar una. Cargar el programa completo en memoria hace que se cargue el código ejecutable de todas las opciones, independientemente de que el usuario selecciona o no una determinada opción.

Una estrategia alternativa consiste en cargar inicialmente las páginas únicamente cuando sean necesarias. Esta técnica se denomina ***paginación bajo demanda*** y se utiliza comúnmente en los sistemas de memoria virtual.

Con la memoria virtual basada en paginación bajo demanda, sólo se cargan las páginas cuando así se solicita durante la ejecución del programa; de este modo, las páginas a las que nunca se acceda no llegarán a cargarse en la memoria física.

Un sistema de paginación bajo demanda es similar a un sistema de paginación con intercambio en el que los procesos residen en memoria secundaria (usualmente en disco). La siguiente imagen ilustra esta idea.

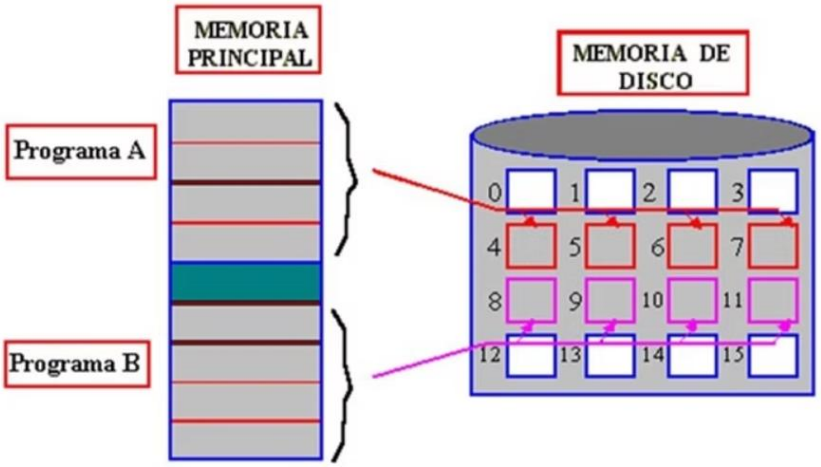


Ilustración 38 - Transferencia de una memoria paginada a un espacio contiguo de disco.

Cuando queremos ejecutar un proceso, realizamos un intercambio para cargarlo en memoria. Sin embargo, en lugar de intercambiar el proceso completo con la memoria, lo que hacemos en este caso es utilizar un ***intercambiador perezoso***[30]. El intercambiador perezoso jamás intercambia una página con la memoria a menos que esa página vaya a ser necesaria. Puesto que ahora estamos contemplando los procesos como secuencias de páginas, en lugar de como un único espacio de direcciones contiguas de gran tamaño, la utilización del término intercambiador es técnicamente incorrecta. Un intercambiador manipula procesos completos, mientras que un paginador sólo se ocupa de las páginas individuales de un proceso. Por tanto, es preferible el término ***paginador*** en lugar de intercambiador, cuando hablemos de la paginación bajo demanda.

Conceptos básicos.

Cuando hay que cargar un proceso, el paginador realiza una estimación de qué páginas serán utilizadas antes de descargar de nuevo el proceso. En lugar de cargar el proceso completo, el paginador sólo carga en la memoria las páginas necesarias; de este modo, evita cargar en la memoria las páginas que no vayan a ser utilizadas, reduciendo así el tiempo de carga y la cantidad de memoria física necesaria.

Con la paginación bajo demanda necesitamos algún tipo de soporte ***hardware*** para distinguir entre las páginas que se encuentran en memoria y las páginas que residen en el disco. Podemos usar para este propósito el esquema basado en un ***bit válido-inválido***. Por ejemplo, la ***MMU*** intercepta la dirección y verifica el bit válido-inválido. Si es 0, dispara la excepción.

1. **La CPU pide un dato:** El procesador intenta leer una dirección.
2. **La MMU busca en la TLB:** Se fija en el búfer rápido para ahorrar tiempo.
3. **Consulta a la Tabla (RAM):** Si no estaba en el búfer, la MMU mira la Tabla de Páginas en la memoria RAM.
4. **La MMU inspecciona el bit válido-inválido:**
   - **Si el bit es 1:** El dato ya está en la RAM. La ejecución continúa normalmente.
   - **Si el bit es 0:** Se genera un **Fallo de Página**. Esto le indica al sistema que el dato **está en el disco y es necesario cargarlo en la RAM**.

Sin embargo, esta vez, cuando se configura este bit como “válido”, la página asociada será legal y además se encontrará en memoria. Si el bit se configura como “inválido”, querrá decir que o bien la página no es válida (es decir, no se encuentra en el espacio lógico de direcciones del proceso) o es válida, pero está actualmente en el disco. La entrada de la tabla de páginas correspondiente a una página que se cargue en memoria se configurará de la forma usual, pero la entrada de la tabla de páginas correspondiente a una página que no se encuentre

[30] Un "intercambiador perezoso", en el contexto de la memoria virtual, es un sistema que no carga una página de memoria en memoria principal a menos que sea estrictamente necesario.

actualmente en la memoria se marcará simplemente como inválida o contendrá la dirección de la página dentro del disco. Esta situación se ilustra en la siguiente figura.

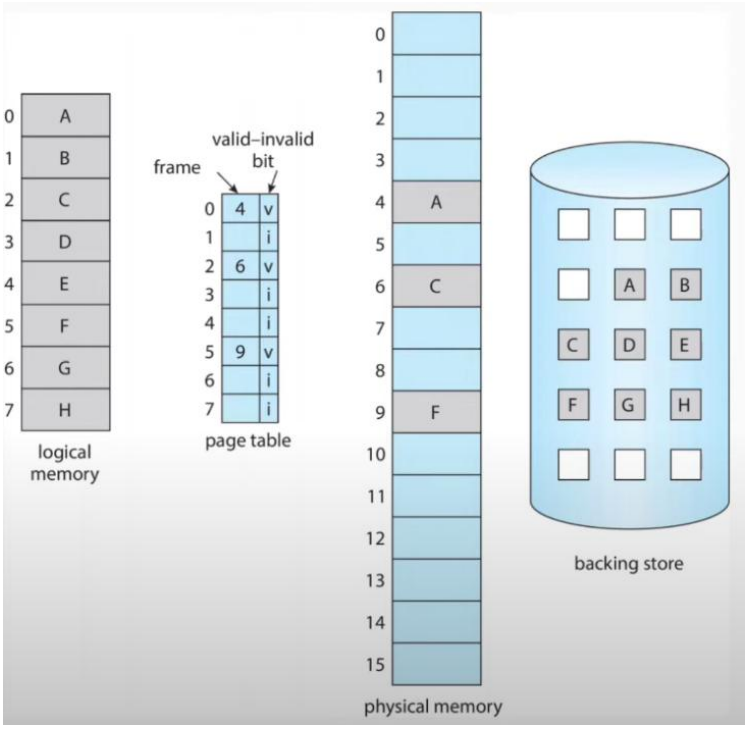


Ilustración 39 - Tabla de páginas cuando algunas páginas no se encuentran en memoria principal.

Observemos que marcar una página como inválida no tendrá ningún efecto si el proceso no intenta nunca acceder a dicha página. Por lo tanto, si nuestra estimación inicial es correcta y cargamos en la memoria todas las páginas que sean verdaderamente necesarias (y sólo esas páginas), el proceso se ejecutará exactamente igual que si hubiéramos cargado en memoria todas las páginas.

Mientras que el proceso se ejecute y acceda a páginas que sean residentes en memoria, la ejecución se llevará a cabo normalmente. Pero si el proceso trata de acceder a una página que no haya sido cargada en memoria, el acceso a una página marcada como inválida provoca una interrupción de fallo de página. El hardware de paginación, al traducir la dirección mediante la tabla de páginas, detectará que el bit de página inválida está activado, provocando una interrupción dirigida al sistema operativo. Esta interrupción se produce como resultado de que el sistema operativo no ha cargado anteriormente en memoria la página deseada. El procedimiento para tratar este fallo de página es muy sencillo.

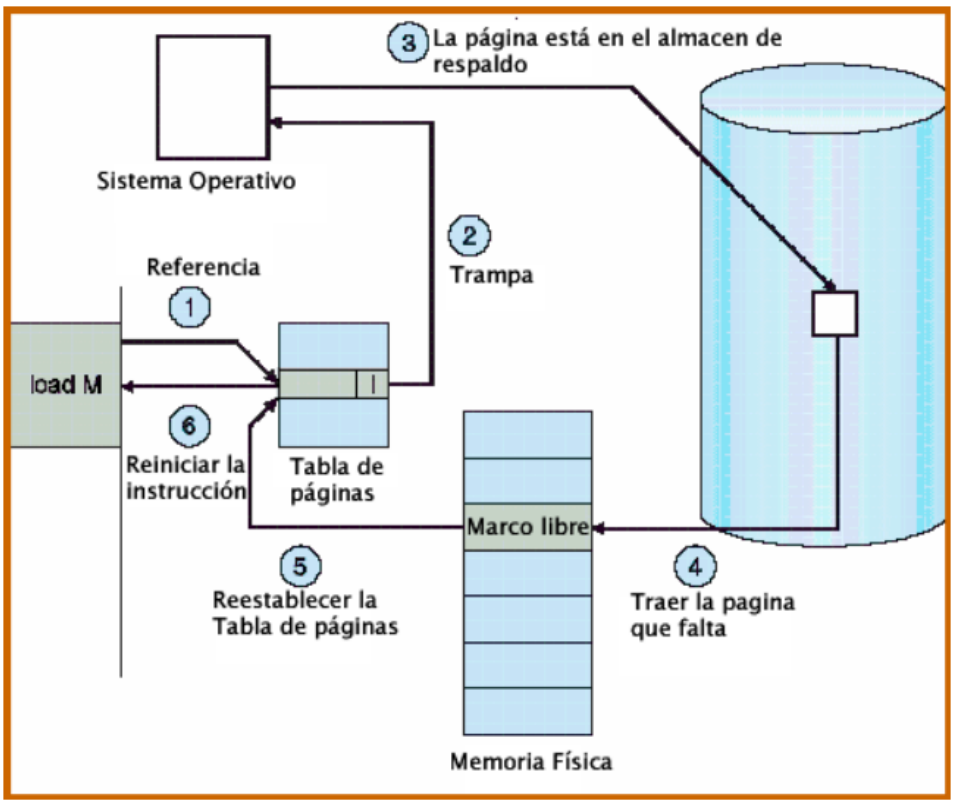


Ilustración 40 - Pasos para tratar un fallo de página.

1. Comprobamos una tabla interna (que usualmente se mantiene con el bloque del control del proceso) correspondiente a este proceso, para determinar si la referencia era un acceso de memoria válido o inválido.
2. Si la referencia era inválida, terminamos el proceso. Si era válida pero esa página todavía no ha sido cargada, la cargamos en memoria.
3. Buscamos un marco libre (por ejemplo, tomando uno de la lista de marcos libres).
4. Ordenamos una operación de disco para leer la página deseada en el marco recién asignado.
5. Una vez completada la lectura de disco, modificamos la tabla interna que se mantiene con los datos del proceso y la tabla de páginas para indicar que dicha página se encuentra ahora en memoria.
6. Reiniciamos la instrucción que fue interrumpida. El proceso podrá ahora acceder a la página como si siempre hubiera estado en memoria.

En el caso extremo, podríamos empezar a ejecutar un proceso sin ninguna página en memoria. Cuando el sistema operativo hiciera que el puntero de instrucciones apuntara a la primera instrucción del proceso, que se encontraría en una página no residente en memoria, se produciría inmediatamente un fallo de página. Después de cargar dicha página en memoria, el proceso continuaría ejecutándose, generando los fallos de página necesarios hasta que todas las páginas requeridas se encontraran en memoria. A partir de ahí, podría ejecutarse sin ningún fallo de página adicional. Este esquema sería una paginación bajo demanda pura, nunca cargar una página en memoria hasta que sea requerida.

En teoría, algunos programas podrían acceder a varias nuevas páginas de memoria con cada ejecución de una instrucción (una página para la instrucción y muchas para los datos), posiblemente generando múltiples fallos de página por cada instrucción. Esta situación provocaría una degradación inaceptable en el rendimiento del sistema. Afortunadamente, el análisis de la ejecución de los procesos muestra que este comportamiento es bastante improbable. Los programas tienden a tener lo que se denomina ***localidad de referencia***[31], por lo que las prestaciones que pueden obtenerse con el mecanismo de paginación bajo demanda son razonables.

El hardware necesario para soportar la paginación bajo demanda es el mismo que para los mecanismos de paginación e intercambio:

- **Una tabla de páginas.** Esta tabla permite marcar una entrada como inválida mediante un bit válido-inválido o mediante un valor especial de una serie de bits de protección.
- **Memoria secundaria.** Esta memoria almacena aquellas páginas que no están presentes memoria principal. La memoria secundaria es usualmente un disco de alta velocidad. Se le conoce como ***dispositivo de intercambio*** y la sección de la disco utilizada para este propósito se llama ***espacio de intercambio***.

Un requisito fundamental para la paginación bajo demanda es la necesidad de poder reiniciar cualquier instrucción después de un fallo de página. Puesto que guardamos el estado del proceso interrumpido en el momento de producirse el fallo de página, debemos poder reiniciar el proceso en exactamente el mismo lugar y el mismo estado, salvo porque ahora la página deseada se encontrará en memoria y será accesible. En la mayoría de los casos, este requisito resulta fácil de satisfacer.

Rendimiento de la paginación bajo demanda.

La paginación bajo demanda puede afectar significativamente al rendimiento de un sistema informático.

Mientras no tengamos fallos de página, el tiempo de acceso efectivo será igual al tiempo de acceso a memoria. Sin embargo, si se produce un fallo de página, deberemos primero leer la página relevante desde el disco y luego acceder a la página deseada.

Para calcular el tiempo de acceso efectivo, debemos conocer cuánto tiempo se requiere para dar servicio a un fallo de página. Cada fallo de página hace que se produzca la siguiente secuencia:

1. Interrupción al sistema operativo.
2. Guardar los registros de usuario y el estado del proceso.
3. Determinar que la interrupción se debe a un fallo de página.

[31] La "localidad de referencia" en el contexto de la paginación bajo demanda se refiere al principio de que los procesos tienden a acceder a un conjunto limitado de páginas de memoria virtual a la vez, en lugar de a todas las páginas de forma aleatoria. Esto significa que, durante un período de tiempo, un proceso solo necesita tener una pequeña parte de sus páginas en la memoria física, lo que permite que la paginación bajo demanda funcione eficientemente.

4. Comprobar que la referencia de página era legal y determinar la ubicación de la página en el disco.
5. Ordenar una lectura desde el disco para cargar la página en un marco libre:
    a. Esperar en la cola correspondiente a este dispositivo hasta que se dé servicio a la solicitud de lectura.
    b. Esperar el tiempo de búsqueda y/o latencia del dispositivo.
    c. Comenzar la transferencia de la página hacia un marco libre.
6. Mientras estamos esperando, asignar la CPU a algún otro usuario (planificación de la CPU, que será opcional).
7. Recibir una interrupción del subsistema de E/S de disco (E/S completada).
8. Guardar los registros y el estado del proceso para el otro usuario (si se ejecuta el paso 6).
9. Determinar que la interrupción corresponde al disco.
10. Corregir la tabla de páginas y otras tablas para mostrar que la página deseada se encuentra ahora en memoria.
11. Esperar a que se vuelva a asignar la CPU a este proceso.
12. Restaurar los registros de usuario, el estado del proceso y la nueva tabla de páginas y reanudar la ejecución de la instrucción interrumpida.

No todos estos pasos son necesarios en todos los casos. Por ejemplo, estamos suponiendo que, en el paso 6, la CPU se asigna a otro proceso mientras que tiene lugar la E/S. Esta operación permite que los mecanismos de multiprogramación mantengan una alta tasa de utilización de la CPU, pero requiere un tiempo adicional para reanudar la rutina de servicio del fallo de página después de completarse la transferencia de E/S.

Resulta crucial mantener una tasa de fallos de página baja en los sistemas de paginación bajo demanda. Si no se hace así, el tiempo efectivo de acceso se incrementa, ralentizando enormemente la ejecución de los procesos.

Un aspecto adicional de la paginación bajo demanda es la gestión y el uso global del espacio de intercambio. Las operaciones de E/S de disco dirigidas al espacio de intercambio son generalmente más rápidas que las correspondientes al sistema de archivos, porque el espacio de intercambios se asigna en bloques mucho mayores y no se utilizan mecanismos de búsqueda de archivos. El sistema puede, por tanto, conseguir una mayor tasa de transferencia para paginación copiando la imagen completa de un archivo en el espacio de intercambio en el momento de iniciar el proceso y luego realizando una paginación bajo demanda a partir del espacio de intercambio.

Otra opción consiste en demandar las páginas inicialmente desde el sistema de archivos, pero irlas escribiendo en el espacio de intercambio a medida que se las sustituye. Esta técnica garantiza que sólo se lean desde el sistema de archivos las páginas necesarias y que todas las operaciones subsiguientes de paginación se lleven a cabo desde el espacio de intercambio.

Referencias y bibliografía de la sección.

- Silberschatz 9-2.